



UNIVERSIDADE FEDERAL DE GOIÁS  
INSTITUTO DE INFORMÁTICA

FERNANDO SOUZA FURTADO CARRILHO

**Inferência de Intenções de Usuário em  
Linguagens Específicas de Domínio:**  
**Um Método baseado em LLMs e Engenharia Dirigida por  
Modelos**

Goiânia  
2026

UNIVERSIDADE FEDERAL DE GOIÁS  
INSITUTO DE INFORMÁTICA

**AUTORIZAÇÃO PARA PUBLICAÇÃO DE DISSERTAÇÃO  
EM FORMATO ELETRÔNICO**

Na qualidade de titular dos direitos de autor, **AUTORIZO** o Insituto de Informática da Universidade Federal de Goiás – UFG a reproduzir, inclusive em outro formato ou mídia e através de armazenamento permanente ou temporário, bem como a publicar na rede mundial de computadores (*Internet*) e na biblioteca virtual da UFG, entendendo-se os termos “reproduzir” e “publicar” conforme definições dos incisos VI e I, respectivamente, do artigo 5º da Lei nº 9610/98 de 10/02/1998, a obra abaixo especificada, sem que me seja devido pagamento a título de direitos autorais, desde que a reprodução e/ou publicação tenham a finalidade exclusiva de uso por quem a consulta, e a título de divulgação da produção acadêmica gerada pela Universidade, a partir desta data.

**Título:** Inferência de Intenções de Usuário em Linguagens Específicas de Domínio: – Um Método baseado em LLMs e Engenharia Dirigida por Modelos

**Autor(a):** Fernando Souza Furtado Carrilho

Goiânia, 20 de Abril de 2026.

---

Fernando Souza Furtado Carrilho – Autor

---

Fábio Moreira Costa – Orientador

FERNANDO SOUZA FURTADO CARRILHO

# **Inferência de Intenções de Usuário em Linguagens Específicas de Domínio:**

**Um Método baseado em LLMs e Engenharia Dirigida por  
Modelos**

Dissertação apresentada ao Programa de Pós-Graduação do  
Insituto de Informática da Universidade Federal de Goiás,  
como requisito parcial para obtenção do título de Mestre  
em Ciência da Computação.

**Área de concentração:** Sistemas de Computação.

**Orientador:** Prof. Fábio Moreira Costa

Goiânia  
2026

FERNANDO SOUZA FURTADO CARRILHO

# **Inferência de Intenções de Usuário em Linguagens Específicas de Domínio:**

## **Um Método baseado em LLMs e Engenharia Dirigida por Modelos**

Dissertação defendida no Programa de Pós-Graduação do Instituto de Informática da Universidade Federal de Goiás como requisito parcial para obtenção do título de Mestre em Ciência da Computação, aprovada em 20 de Abril de 2026, pela Banca Examinadora constituída pelos professores:

---

**Prof. Fábio Moreira Costa**  
Instituto de Informática – UFG  
Presidente da Banca

---

**Prof. Dr. Membro da Banca 1**  
Instituto de Informática – UFG

---

**Prof. Dr. Membro da Banca 2**  
Instituto de Informática – UFG

---

**Prof. Dr. Membro Externo**  
Faculdade de Fora – FF

Todos os direitos reservados. É proibida a reprodução total ou parcial do trabalho sem autorização da universidade, do autor e do orientador(a).

**Fernando Souza Furtado Carrilho**

Graduou-se em Bacharelado em Engenharia de Computação pelo Campus Trindade do Instituto Federal de Ciência, Tecnologia e Educação Goiano - IF Goiano. Durante sua graduação, foi membro do Conselho Superior da Instituição, monitor de Física II do curso e fez intercâmbio em Engenharia Informática no Instituto Politécnico de Bragança - IPB, Portugal. Atualmente desenvolve pesquisa no Programa de Pós-Graduação em Ciência da Computação no Instituto de Informática na UFG para problemas de leitura e interpretação de domínio específico via LLM.

Dedico este trabalho à Deus, pela oportunidade e força, à minha falecida mãe pelo incentivo aos estudos e à mim pela dedicação e comprometimento.

---

## Agradecimentos

---

Agradeço primeiramente a Deus, por me dar saúde, resiliência e clareza mental para atravessar os momentos mais desafiadores desta jornada. Ao meu orientador, Prof. Dr. Fábio Moreira Costa, pela confiança, paciência e por me guiar pelos complexos caminhos da Inteligência Artificial neuro-simbólica. Suas orientações foram fundamentais para transformar as ideias sobre extração de intenções e DSLs em uma pesquisa sólida. À Universidade Federal de Goiás (UFG) e ao Programa de Pós-Graduação em Ciência da Computação, por fornecerem o ambiente acadêmico de excelência e a infraestrutura necessários para o desenvolvimento desta dissertação. O presente trabalho foi realizado com apoio do Conselho Nacional de Desenvolvimento Científico e Tecnológico (CNPq)

“Os limites da minha linguagem significam os limites do meu mundo.”

**Ludwig Wittgenstein,**  
*Tractatus Logico-Philosophicus.*



---

## Resumo

---

Carrilho, Fernando Souza Furtado. **Inferência de Intenções de Usuário em Linguagens Específicas de Domínio**. Goiânia, 2026. 101p. Dissertação de Mestrado. Insituto de Informática, Universidade Federal de Goiás.

A crescente integração de Modelos de Linguagem de Grande Escala (LLMs) em interfaces computacionais viabilizou a interpretação de intenções humanas complexas por parte das máquinas a partir de comandos informais em linguagem natural. No entanto, em domínios de missão crítica, a natureza estocástica inerente a esses modelos gera riscos operacionais, como alucinações e violações de regras de negócio. Esta pesquisa de mestrado propõe a arquitetura CML (*Conceptual Model Learning*), uma infraestrutura híbrida projetada para garantir o fechamento semântico mútuo - correspondência determinística e fiel entre a intenção original do usuário e a execução da máquina - na automação de sistemas críticos. A solução utiliza a flexibilidade semântica dos LLMs como um motor de tradução, porém confina sua execução ao rigor formal da Engenharia Dirigida por Modelos (MDE) e à ancoragem factual de grafos de conhecimento. Através de um *pipeline* que integra gramáticas formais a um processo de decodificação restrita, o sistema assegura que a saída gerada pela IA seja validada por um Esquema de Controle e formatada por um Esquema de Dados. A viabilidade da arquitetura é avaliada em um estudo de caso no domínio de agricultura de precisão, onde comandos informais de usuários não especializados são convertidos em instruções de pulverização e voo que, além de determinísticas, preservam a estrita correspondência com a intenção original do usuário e operam em conformidade com as regras do modelo descrito pelo usuário especializado. Espera-se que os resultados evidenciem a neutralização de erros sintáticos e uma rigorosa conformidade com as invariantes do domínio, provendo uma camada de governança robusta para a automação dirigida por modelos.

### Palavras-chave

Modelos de Linguagem de Grande Escala (LLM), Linguagens de Domínio Específico (DSL), Grafos de Conhecimento, Engenharia Dirigida por Modelos (MDE), Sistemas Críticos.

---

## Abstract

---

Carrilho, Fernando Souza Furtado. **User Intent Inference in Domain-Specific Languages: A Method Based on LLMs and Model-Driven Engineering.** Goiânia, 2026. 101p. MSc. Dissertation. Insituto de Informática, Universidade Federal de Goiás.

The growing integration of Large Language Models (LLMs) into computational interfaces has enabled machines to interpret complex human intentions from informal natural language commands. However, in mission-critical domains, the stochastic nature inherent to these models introduces operational risks, such as hallucinations and business rule violations. This master's research proposes the CML (Conceptual Model Learning) architecture, a hybrid infrastructure designed to ensure mutual semantic closure — a deterministic and faithful correspondence between the user's original intention and the machine's execution — in the automation of critical systems. The solution leverages the semantic flexibility of LLMs as a translation engine, yet confines their execution to the formal rigor of Model-Driven Engineering (MDE) and the factual grounding of knowledge graphs. Through a pipeline that integrates formal grammars into a constrained decoding process, the system ensures that the AI-generated output is validated by a Control Schema and formatted by a Data Schema. The feasibility of the architecture is evaluated in a case study within the precision agriculture domain, where informal commands from non-specialized users are converted into spraying and flight instructions that, besides being deterministic, preserve strict correspondence with the user's original intention and operate in compliance with the rules of the model described by the specialized user. The results are expected to evidence the neutralization of syntactic errors and strict compliance with domain invariants, providing a robust governance layer for model-driven automation.

### Keywords

Large Language Models (LLM), Domain-Specific Languages (DSL), Knowledge Graphs, Model-Driven Engineering (MDE), Critical Systems.

---

# Sumário

---

|                                                                                        |           |
|----------------------------------------------------------------------------------------|-----------|
| Lista de Figuras                                                                       | 12        |
| Lista de Tabelas                                                                       | 13        |
| Lista de Algoritmos                                                                    | 14        |
| Lista de Códigos de Programas                                                          | 15        |
| <b>1 Introdução</b>                                                                    | <b>16</b> |
| 1.1 Contexto e Motivação                                                               | 16        |
| 1.2 Objetivos                                                                          | 18        |
| 1.3 Metodologia                                                                        | 19        |
| 1.4 Contribuição Esperada                                                              | 21        |
| 1.5 Organização do Texto                                                               | 21        |
| 1.6 Plano de Trabalho, Resultados Esperados e Cronograma                               | 22        |
| 1.6.1 Estágio Atual da Pesquisa                                                        | 22        |
| 1.6.2 Resultados Esperados                                                             | 23        |
| 1.6.3 Plano de Trabalho                                                                | 23        |
| 1.6.4 Cronograma de Execução                                                           | 24        |
| <b>2 Fundamentação Teórica</b>                                                         | <b>25</b> |
| 2.1 Engenharia Dirigida por Modelos (MDE) e Linguagens de Domínio Específico (DSL)     | 25        |
| 2.2 Modelos de Linguagem de Grande Escala e o Processamento Semântico                  | 27        |
| 2.3 Engenharia de Prompts Otimizada e Grammar Prompting                                | 29        |
| 2.4 Grafos de Conhecimento e Retrieval-Augmented Generation (GraphRAG)                 | 30        |
| 2.5 Sistemas Multiagentes na Garantia da Confiabilidade Semântica                      | 31        |
| 2.6 Conclusão do Capítulo                                                              | 32        |
| <b>3 Engenharia de Prompts Dirigida por Modelos: Revisão Sistemática da Literatura</b> | <b>34</b> |
| 3.1 O Problema de Pesquisa                                                             | 35        |
| 3.2 Metodologia e Protocolo da Revisão                                                 | 36        |
| 3.2.1 Questões de Pesquisa (QPs)                                                       | 37        |
| 3.2.2 Estratégia de Busca e Bases de Dados                                             | 37        |
| 3.2.3 Critérios de Seleção (Inclusão e Exclusão)                                       | 37        |
| 3.3 Resultados e Discussão                                                             | 39        |
| 3.3.1 Fluxo de Seleção dos Estudos Primários                                           | 39        |
| 3.3.2 Mapeamento Temático e Eixos de Análise                                           | 39        |
| Eixo 1: Otimização Algorítmica e Espaço Latente (Gradient-Based Optimization)          | 39        |
| Eixo 2: Governança Semântica via Grafos de Conhecimento e Ontologias                   | 40        |

|                                                                            |           |
|----------------------------------------------------------------------------|-----------|
| Eixo 3: Engenharia de Software Multiagente e Gramáticas Formais em NL2Code | 41        |
| 3.3.3 Síntese e Respostas às Questões de Pesquisa                          | 42        |
| 3.4 Conclusão do Capítulo                                                  | 43        |
| <b>4 Trabalhos Relacionados</b>                                            | <b>45</b> |
| 4.1 Interpretação de Intenção Semântica com LLMs                           | 45        |
| 4.1.1 Avanços na Literatura                                                | 45        |
| 4.1.2 Análise Crítica e Lacunas                                            | 46        |
| 4.2 DSLs e a Engenharia Dirigida por Modelos com IA                        | 47        |
| 4.2.1 Avanços na Literatura                                                | 47        |
| 4.2.2 Análise Crítica e Lacunas                                            | 48        |
| 4.3 Integração de Grafos de Conhecimento com LLMs                          | 48        |
| 4.3.1 Avanços na Literatura                                                | 49        |
| 4.3.2 Análise Crítica e Lacunas                                            | 49        |
| 4.4 Grammar-Guided Prompting e Decodificação Restrita                      | 50        |
| 4.4.1 Avanços na Literatura                                                | 50        |
| 4.4.2 Análise Crítica e Lacunas                                            | 51        |
| 4.5 Máquinas Virtuais de Domínio Específico (DSVM)                         | 52        |
| 4.5.1 Avanços na Literatura                                                | 52        |
| 4.5.2 Análise Crítica e Lacunas: A Ponte para o CML                        | 53        |
| 4.6 Síntese e Posicionamento: O Diferencial da Arquitetura CML             | 53        |
| 4.7 Conclusão do Capítulo                                                  | 56        |
| <b>5 Proposta do modelo da arquitetura SPC-CML</b>                         | <b>57</b> |
| 5.1 Visão Geral da Arquitetura                                             | 57        |
| 5.2 Fundamentação                                                          | 58        |
| 5.2.1 O Modelo: Esquemas de Controle e Dados                               | 59        |
| 5.2.2 Grafo de Conhecimento (GraphRAG)                                     | 60        |
| 5.3 O Pipeline de Processamento de Intenções                               | 60        |
| 5.4 Cenário                                                                | 61        |
| 5.4.1 Modelagem do Conhecimento: A DSL Agrícola e Subgrafos Ontológicos    | 62        |
| 5.4.2 O Fluxo de Execução em Tempo Real                                    | 63        |
| 5.4.3 Análise do Fechamento Semântico no Caso de Uso                       | 65        |
| 5.5 Status de Desenvolvimento e Implementação Atual                        | 66        |
| 5.5.1 Componentes e Artefatos Concluídos                                   | 66        |
| 5.5.2 Análise do Log de Execução e Consolidação Contextual                 | 68        |
| 5.5.3 Componentes Pendentes e Próximos Passos de Engenharia                | 72        |
| 5.6 Conclusão do Capítulo                                                  | 73        |
| <b>6 Conclusão e Próximos Passos</b>                                       | <b>74</b> |
| 6.1 Síntese das Contribuições Técnicas                                     | 74        |
| 6.2 Objetivos Alcançados e Maturidade da Pesquisa                          | 75        |
| 6.3 Resultados Esperados e Desafios de Implementação                       | 75        |
| 6.4 Cronograma para Defesa de Dissertação                                  | 76        |
| 6.5 Conclusão do Capítulo                                                  | 77        |
| <b>Referências Bibliográficas</b>                                          | <b>78</b> |

|          |                                                                                                      |           |
|----------|------------------------------------------------------------------------------------------------------|-----------|
| <b>A</b> | <b>Arquitetura, Integração e Viabilidade: Sistema Model-Driven com Grafos de Conhecimento e LLMs</b> | <b>82</b> |
| A.1      | Validação da Arquitetura e Componentes de Terceiros                                                  | 83        |
| A.2      | Componentes a Implementar                                                                            | 83        |
| A.2.1    | Body: Semantic Routing Engine                                                                        | 84        |
| A.3      | Plataforma, Linguagens e Frameworks                                                                  | 84        |
| A.4      | Estratégia do Pipeline CML                                                                           | 85        |
| A.5      | Plano de Testes e Avaliação                                                                          | 85        |
| A.6      | Análise de Viabilidade                                                                               | 86        |
| A.7      | Cronograma                                                                                           | 86        |
| A.8      | Justificativa da Escolha Tecnológica                                                                 | 86        |
| A.9      | A Instanciação da Linguagem Específica de Domínio                                                    | 87        |
| A.9.1    | A Arquitetura Conceitual da DSL                                                                      | 87        |
| A.9.2    | Ferramentas e Frameworks Selecionados                                                                | 87        |
| A.9.3    | Passo-a-Passo: Especificação CML para o Agrodrone                                                    | 87        |
| <b>B</b> | <b>Apostila Técnica - Arquitetura Neuro-Simbólica</b>                                                | <b>90</b> |
| B.1      | Visão Geral do Cenário AgroDrone                                                                     | 91        |
| B.2      | Setup do Ambiente                                                                                    | 91        |
| B.2.1    | Pré-requisitos                                                                                       | 91        |
| B.2.2    | Extensões do VS Code                                                                                 | 91        |
| B.3      | Criação do Projeto                                                                                   | 91        |
| B.3.1    | Configuração e Instalação                                                                            | 91        |
| B.4      | Estrutura do Projeto                                                                                 | 92        |
| B.5      | Criando Linguagem/Gramática DSL                                                                      | 93        |
| B.6      | Criando o Modelo da DSL Agrícola                                                                     | 96        |
| B.6.1    | Exemplo de Modelo no uso da DSL)                                                                     | 96        |
| B.7      | Langium - Pipeline de Parsing AgroDrone                                                              | 98        |
| B.8      | Geração de Embeddings Locais                                                                         | 98        |
| B.9      | Persistência no Knowledge Graph (Neo4j)                                                              | 98        |
| B.10     | Motor de Decisão (LLM) + Telemetria                                                                  | 98        |
| B.11     | Repositório do Projeto no GitHub                                                                     | 100       |

---

## Lista de Figuras

---

|      |                                                                                                                                                   |    |
|------|---------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 5.1  | Arquitetura de Integração CML: Fluxo neuro-simbólico da DSL para o LLM Alvo, suportado por Grafos de Conhecimento e Decodificação Restrita.       | 58 |
| 5.2  | Topologia do Grafo de Conhecimento CML instanciado a partir da DSL Agrícola, evidenciando subgrafos por cultura e invariantes sistêmicas globais. | 62 |
| 5.3  | Fluxo de Execução CML: Da assimilação de telemetria e fala coloquial à execução especializada de <i>hardware</i> .                                | 64 |
| 5.4  | Instanciação gráfica e propriedades dos nós de cultivo no banco de dados Neo4j.                                                                   | 67 |
| 5.5  | Mapeamento de relacionamentos topológicos entre talhões e culturas agrícolas no Neo4j.                                                            | 68 |
| 5.6  | Nós de substâncias químicas e parametrização de dosagens limite no grafo.                                                                         | 69 |
| 5.7  | Grafo de restrições relacionando a aplicação de defensivos às respectivas culturas-alvo.                                                          | 69 |
| 5.8  | Visualização das regras de transição de estado e restrições de segurança injetadas no grafo.                                                      | 70 |
| 5.9  | Estruturação de regras globais e subgrafos de segurança ativados via vetores de índice.                                                           | 70 |
| 5.10 | Painel de controle integrado evidenciando a correlação de regras recuperadas e prontas para a etapa de prompt.                                    | 71 |
| 5.11 | Log de execução do terminal demonstrando a captura de inputs, indexação vetorial e extração de regras no Neo4j.                                   | 71 |
| B.1  | Pipeline de parse com tratamento de referências cruzadas no modelo Agrícola.                                                                      | 99 |

---

## Lista de Tabelas

---

|     |                                                                       |    |
|-----|-----------------------------------------------------------------------|----|
| 1.1 | Cronograma de atividades para o semestre final (2026-2)               | 24 |
| 4.1 | Comparativo entre a Literatura Correlata e a Metodologia CML Proposta | 55 |
| 6.1 | Cronograma de atividades para o semestre final (2026-2)               | 76 |

---

## **Lista de Algoritmos**

---



---

## **Lista de Códigos de Programas**

---

---

# Introdução

---

## 1.1 Contexto e Motivação

A adoção de Modelos de Linguagem de Grande Escala (LLMs) desencadeou uma reestruturação profunda nas metodologias de Engenharia de Software e Processamento de Linguagem Natural (NLP). Conforme evidenciado por revisões sistemáticas recentes [Husein, Aburajouh e Catal 2025], a aplicação de LLMs em tarefas complexas, como a geração e o autocompletamento de código, atingiu patamares sem precedentes. Esse avanço foi viabilizado por arquiteturas baseadas em *transformers* e por técnicas de sintonia fina com eficiência de parâmetros (*PEFT - Parameter-Efficient Fine-Tuning*) [Bai et al. 2025], que permitem adaptar grandes modelos fundacionais utilizando uma fração mínima de poder computacional sem perder o conhecimento pré-treinado [Yan et al. 2025].

A expressividade semântica e a capacidade de adaptação dessas redes neurais permitiram a sua expansão para domínios técnicos altamente especializados. Observa-se a utilização bem-sucedida de IA generativa em cenários tão diversos quanto a síntese de programas para jogos eletrônicos [Eberhardinger et al. 2025], a sumarização iterativa de laudos radiológicos críticos [Ma et al. 2024], a análise de falhas em processos de manufatura aditiva 3D [Karabiyik 2025] e a previsão automatizada de carga energética estrutural em edifícios [Zhang et al. 2025]. Em contextos de engenharia de software no domínio de testes, a geração de entradas de texto automatizadas e sensíveis ao contexto em interfaces gráficas (GUIs) móveis também tem se beneficiado dessas abordagens generativas [Ma et al. 2024].

Apesar deste amplo sucesso empírico, a transição da pesquisa de laboratório para a produção em sistemas de missão crítica expõe vulnerabilidades estruturais nos LLMs. O fenômeno da alucinação — caracterizado pela geração de conteúdo linguisticamente fluente e sintaticamente plausível, porém factualmente incorreto ou desconexo das regras físicas e lógicas de um domínio — permanece como um dos maiores entraves para a confiabilidade sistêmica [Ji et al. 2023]. Adicionalmente, modelos puramente baseados em texto são suscetíveis a ataques adversariais sutis, demandando métodos contínuos de

adaptação de domínio e defesa via *prompt-tuning* para preservar a integridade das saídas [Chen et al. 2025].

Essa imprevisibilidade cria uma lacuna crítica entre a ambiguidade da linguagem natural e o determinismo exigido por sistemas críticos. Para compreender concretamente este problema, considere o cenário da automação de drones na agricultura de precisão. Um operador de campo pode emitir um comando verbal informal, como "vai ver se precisa veneno e aplica onde for preciso". Enquanto a linguagem natural humana é flexível, o sistema de controle de voo e pulverização de um drone exige coordenadas exatas, doses químicas matematicamente calculadas e a estrita obediência a regras de segurança — como a proibição de pulverizar sob ventos fortes ou aplicar o produto errado em uma cultura sensível.

O problema central reside no fato de que, se um LLM atuar de forma livre e puramente probabilística para interpretar e converter esse comando em ações de máquina, ele pode alucinar parâmetros ou ignorar completamente as restrições físicas da lavoura, o que resulta em aplicações químicas perigosas, infrações ambientais ou falhas sistêmicas catastróficas. Para mitigar esse risco, a abordagem geral de solução delineada nesta pesquisa baseia-se em não permitir que o LLM opere no vácuo de conhecimento ou com liberdade estrutural. O processo como um todo é concebido como um pipeline de restrições sucessivas.

Consoante, no cenário do drone, a solução opera da seguinte forma: primeiramente, o comando coloquial do operador ("aplica o veneno") é recebido e cruzado com os dados de telemetria em tempo real do drone (ex.: velocidade do vento e temperatura). Em seguida, essas informações são confrontadas com um Grafo de Conhecimento, que atua como a “verdade factual” da fazenda, recuperando regras estritas como "se o vento estiver acima de 10 km/h, a pulverização é proibida".

Logo, em vez de o LLM gerar um script de voo em texto livre, a sua saída é canalizada por um processo de decodificação restrita, do grammar prompting, governado por uma Linguagem de Domínio Específico (DSL). O modelo é matematicamente forçado a preencher apenas os campos válidos definidos pela DSL, traduzindo o comando inicial para uma instrução determinística e segura — que, no caso de ventos fortes, resultaria na geração de um comando exato de aborto de missão, para garantir o alinhamento total com a intenção e a segurança da operação.

Diante da imprevisibilidade inerente aos LLMs, a Engenharia de Prompts vem evoluindo de construções manuais empíricas para frameworks de otimização automatizada. Na literatura, pesquisadores buscam mitigar erros orientando a geração do modelo por meio de duas vertentes principais. A primeira envolve a otimização matemática dos comandos de entrada, utilizando técnicas como a busca baseada no próprio gradiente do modelo para refinar instruções automaticamente [Park e Kim 2026]

ou o preenchimento preditivo de contexto [Qin et al. 2025]. A segunda vertente apoia-se em técnicas de condicionamento via exemplos de referência - “*few-shot*” e abstração sem exemplos - “*zero-shot*”, utilizadas para guiar a IA em tarefas multimodais complexas — desde a geração de código até o reconhecimento de anomalias visuais [Shi et al. 2025, Nasiri et al. 2022, Zhang et al. 2025].

Embora essas abordagens reduzam significativamente a margem de erro ao “guiar” o raciocínio da IA, elas explicam-se por uma natureza fundamentalmente probabilística. Contudo, a dependência isolada dessa otimização estocástica — onde o modelo ainda opera calculando a probabilidade estatística da próxima palavra — tem se provado insuficiente para garantir o determinismo absoluto exigido quando há regras rígidas de segurança operacional envolvidas

## 1.2 Objetivos

Com base na lacuna identificada entre a expressividade da linguagem natural e o determinismo exigido por sistemas críticos, a pergunta de pesquisa que norteia esta dissertação é definida como: *Dado um modelo específico de domínio, é possível interpretar e converter de forma confiável a intenção de um usuário em ações sistêmicas determinísticas através de uma metodologia baseada em LLM*

Para responder a esta questão, definem-se os seguintes objetivos:

### Objetivo Geral

Propor, implementar e avaliar uma arquitetura de interpretação de comandos informais em linguagem natural, denominada CML (*Conceptual Model Learning*), que proteja sistemas críticos contra alucinações factuais, erros sintáticos e violações de regras de negócio geradas por LLMs, com o uso de métodos formais para garantir o fechamento semântico mútuo — conceituado nesta pesquisa como a correspondência rigorosa e determinística entre a intenção original do usuário e a execução final atuada pela máquina.

### Objetivos Específicos

1. Projetar uma linguagem específica de domínio (DSL) representativa, formalizada por notações estritas, para atuar como um contrato sintático que permita a validação pré-execução das saídas do modelo;
2. Desenvolver e integrar uma ontologia restritiva em um Grafo de Conhecimento para habilitar o mecanismo de geração aumentada por recuperação orientada a grafos (GraphRAG), a fim de fornecer ancoragem factual e impedir desvios lógicos das regras de negócio do domínio;
3. Construir um *pipeline* de multiagentes com *grammar prompting* iterativo — uma técnica de decodificação que restringe a geração de texto do LLM às regras de uma

gramática formal — para garantir que a intenção do usuário seja decodificada e traduzida em comandos estritamente válidos e conformes à gramática da DSL;

4. Validar empiricamente a abordagem proposta, ao mensurar a redução nas taxas de falha semântica e sintática em comparação com modelos puramente generativos e técnicas de *zero-shot prompting*, com o intuito de comprovar a viabilidade e a segurança operacional da arquitetura.

## 1.3 Metodologia

Do ponto de vista metodológico, esta pesquisa classifica-se, quanto à sua natureza, como aplicada, pois visa resolver um problema prático e latente na indústria de software: a falta de confiabilidade e a presença de alucinações na execução de comandos críticos por Modelos de Linguagem de Grande Escala (LLMs). Quanto aos objetivos, a pesquisa tem caráter exploratório e empírico. Exploratório por investigar a intersecção recente entre inteligência artificial generativa e a Engenharia Dirigida por Modelos (MDE) aliada a gramáticas formais; e empírico por submeter a arquitetura CML (Conceptual Model Learning) — o artefato concebido nesta pesquisa — a uma avaliação de viabilidade técnica por meio de um estudo de caso.

Para conduzir a investigação, adota-se o paradigma da *Design Science Research* (DSR), uma abordagem consagrada na Engenharia de Software focada na criação, implementação e avaliação de artefatos inovadores para a resolução de problemas complexos. Como alvo imediato de implementação e validação, esta pesquisa desenvolve um artefato de software funcional baseado na arquitetura CML (*Conceptual Model Learning*).

A construção e validação deste artefato fundamentam-se na convergência entre a inteligência artificial e os métodos formais de engenharia de software, apoiando-se na Engenharia Dirigida por Modelos (MDE) [France e Rumpe 2007] e mais especificamente em *Model-Driven Prompt Engineering* [Clarís e Cabot 2023]. Para alcançar os resultados esperados de forma sistemática e científica, a execução da pesquisa está dividida nas seguintes etapas sucessivas:

1. **Mapeamento Sistemático de Literatura e Definição do Domínio:** Condução de uma revisão sistemática da literatura (com uma metodologia consolidada [Husein, Aburajouh e Catal 2025]), apresentada no capítulo 3, para identificar as lacunas de confiabilidade semântica nos LLMs. Paralelamente, define-se o domínio de aplicação crítico, que será utilizado no estudo (operações de drones na agricultura de precisão), e os casos de uso que exigem garantia de segurança.
2. **Construção do Componente Sintático e Semântico (DSL):** Modelagem do domínio para definir a DSL e, então, implementá-la com o uso de um *framework* de

engenharia de linguagens (como o Langium). Esta etapa inclui a formalização de uma gramática rigorosa em notação *Backus-Naur Form* (BNF). O objetivo é criar o mecanismo de validação sintática pré-execução que viabilize o *parsing* semântico seguro — compreendido aqui como o processo determinístico de tradução de instruções em linguagem natural para uma representação formal, estruturada e diretamente executável pela máquina [Sun et al. 2020, Zhang et al. 2025].

3. **Desenvolvimento do Componente Semântico (Ontologia em Grafo):** Mapeamento das regras de negócio, restrições físicas e lógicas do domínio em um Grafo de Conhecimento, usando uma ferramenta como o Neo4j. Esta etapa estabelece a base para a técnica de *GraphRAG* (Geração Aumentada por Recuperação baseada em Grafos) ao indexar os dados textuais em nós topológicos e propriedades consultáveis, criando o mecanismo de recuperação factual necessário para fornecer um contexto validado ao modelo e mitigar alucinações [Lewis et al. 2020, Bahr et al. 2025, Shi et al. 2025].
4. **Controle de Fluxo e Integração Multiagente:** Implementação do *pipeline* central da arquitetura CML. Nesta fase, controla-se a passagem de dados entre o motor de LLM, a gramática (através de *Grammar Prompting* [Wang et al. 2023]) e o grafo de conhecimento. Para garantir a robustez e a revisão cruzada dos comandos antes da execução física, instanciam-se módulos especializados atuando como um sistema multiagente: enquanto um agente foca na desambiguação e roteamento da intenção do usuário no grafo, outro atua exclusivamente na validação e formatação da saída frente as regras da DSL (decodificação restrita), isolando responsabilidades para mitigar falhas [Bai et al. 2025, Chen et al. 2026].
5. **Avaliação Experimental e Análise de Resultados:** Submissão do *framework* CML a um conjunto de testes empíricos com uso de base de comandos de entrada propositalmente ambíguos. O experimento medirá as taxas de falha semântica e de erros de sintaxe — definidos neste escopo como qualquer saída gerada que viole a árvore gramatical BNF imposta pela DSL. A avaliação utilizará como linha de base de desempenho (*baseline*) modelos LLMs puros operando sob *zero-shot prompting* (abordagem em que o modelo é instruído a resolver a tarefa diretamente, sem o fornecimento de exemplos ou contextos prévios no *prompt*). Os dados coletados serão analisados estatisticamente para comprovar a eficácia do fechamento semântico, aqui conceituado como a garantia de que as saídas da inteligência artificial estejam absoluta e deterministicamente contidas dentro das fronteiras lógicas, físicas e gramaticais do domínio modelado.

O cumprimento dessas etapas metodológicas garantirá que o *framework* proposto transite com rigor científico desde a sua fundamentação teórica até a comprovação de sua viabilidade técnica em sistemas de missão crítica.

## 1.4 Contribuição Esperada

Este trabalho culmina na especificação, implementação e avaliação de uma extensão arquitetural fundamentada na abordagem CML (*Conceptual Model Learning*). A contribuição central desta aplicação prática para a área de engenharia de *software* reside em fornecer uma camada de tradução segura e determinística entre a linguagem natural e os sistemas de automação, demonstrando como os princípios do CML podem governar o uso de inteligência artificial generativa por meio de regras rígidas de modelagem de domínio.

A principal contribuição esperada é o estabelecimento de um elo híbrido — fundamentado na integração entre inteligência generativa e métodos formais — e determinístico entre o comando informal humano e a execução mecânica autônoma, tendo como caso de uso prático as operações de *drones* na agricultura de precisão. A arquitetura desenvolvida adota o paradigma CML para controlar a fluidez dos modelos de linguagem, restringindo-os sintaticamente através de Linguagens de Domínio Específico (DSL). Adicionalmente, o sistema atua validando as ações propostas pelo modelo — ou seja, verificando ativamente se a saída gerada pela inteligência artificial não viola nenhuma regra de negócio ou restrição ambiental — por meio de consultas dinâmicas a um Grafo de Conhecimento, que serve como a fonte de verdade (*Ground Truth*) do sistema.

Desse modo, a contribuição científica deste trabalho direciona-se primordialmente à área de Sistemas Autônomos e Ciber-físicos. Espera-se provar que a aplicação do CML na interpretação de comandos de usuários em sistemas de missão crítica impede que o fluxo opere de forma puramente probabilística (sujeito a falhas lógicas e alucinações), encapsulando a inferência em um artefato de *software* estritamente verificável, seguro e limitado pelas leis do ambiente operacional.

## 1.5 Organização do Texto

Para uma apresentação lógica, sistemática e progressiva da pesquisa, o restante deste documento de qualificação está estruturado em cinco capítulos principais, organizados da seguinte forma:

O **Capítulo 2 (Revisão Teórica)** estabelece o arcabouço conceitual, computacional e estatístico (necessário para a compreensão técnica da distribuição de probabilidades de *tokens* e do mascaramento de *logits* nas redes neurais) para a sustentação do trabalho. O capítulo aprofunda-se nos fundamentos da Engenharia Dirigida por Modelos (MDE), no comportamento probabilístico dos Grandes Modelos de Linguagem (LLMs), nas técnicas de decodificação restrita por gramáticas formais (*Grammar Prompting*) e nas propriedades de indexação de Grafos de Conhecimento.

O **Capítulo 3 (Revisão da Literatura)** mapeia o estado da arte e o panorama teórico e tecnológico recente que envolve a intersecção entre inteligência artificial generativa e interfaces de automação. Este capítulo discute a evolução dos Grandes Modelos de Linguagem (LLMs) em âmbito geral e a aplicação prática dessas redes neurais no enriquecimento de *prompts*, abstração de modelos específicos de domínio e obtenção de contextos semânticos.

O **Capítulo 4 (Trabalhos Relacionados)** expõe a Revisão Sistemática da Literatura (RSL). Esse capítulo analisa criticamente as soluções híbridas existentes na literatura para a interpretação de intenções, evidenciando as lacunas de validação estrita baseada em modelos — ou seja, a ausência de garantias sólidas de correção sintática e factual nas abordagens atuais — que justificam o desenvolvimento desta dissertação.

O **Capítulo 5 (Proposta)** detalha formalmente a Arquitetura CML (*Conceptual Model Learning*), especificando os seus componentes internos — o Esquema de Controle, o Esquema de Dados, o *AST-to-Graph Mapper* e o *Semantic Routing Engine*. O capítulo também apresenta a implementação prática da arquitetura em um contexto aplicado à pulverização agrícola com Veículos Aéreos Não Tripulados (VANTs), denominado cenário *Agrodrone*. Em subseção separada da implementação do sistema, o capítulo detalha o *design* do plano de experimentos focado na avaliação empírica do artefato.

Por fim, o **Capítulo 6 (Conclusão)** sintetiza os principais aprendizados obtidos até esta fase de qualificação, avalia os objetivos alcançados, discute os desafios técnicos de codificação remanescentes e apresenta o cronograma detalhado de atividades programadas para o semestre subsequente rumo à defesa final da dissertação.

## 1.6 Plano de Trabalho, Resultados Esperados e Cronograma

Uma vez estabelecidos os objetivos e a metodologia desta pesquisa, esta seção delinea o planejamento operacional para a fase final do mestrado. Ela detalha o atual estágio de maturidade do projeto, as frentes de desenvolvimento tecnológico que compõem o plano de trabalho, os resultados quantitativos e qualitativos almejados, bem como o cronograma de execução.

### 1.6.1 Estágio Atual da Pesquisa

Até o momento deste exame de qualificação, a pesquisa alcançou a maturidade conceitual necessária para o início da implementação em escala, com os seguintes marcos já plenamente concluídos:



- **Mapeamento do Referencial Teórico:** Conclusão da revisão bibliográfica conectando a Engenharia Dirigida por Modelos (MDE), Processamento de Linguagem Natural (PLN) e sistemas distribuídos [Clarísó e Cabot 2023].
- **Formalização Sistemática da Arquitetura:** O desenho da infraestrutura SPC-CML foi completamente estabelecido em três camadas de contenção interdependentes (Ancoragem Factual, Coerção Sintática e Separação de Esquemas), garantindo independência de modelo subjacente (*LLM-agnostic*).
- **Modelagem do Domínio Crítico (Agrodrone):** A materialização do estudo de caso permitiu validar a metassintaxe da DSL no ecossistema Langium, cobrindo cenários complexos de pulverização e restrições ambientais aplicadas a culturas agrícolas.

## 1.6.2 Resultados Esperados

A transição da presente qualificação para a dissertação final focará rigorosamente na validação empírica do modelo. Em termos de resultados práticos, espera-se comprovar que a Arquitetura SPC-CML é capaz de atingir uma **taxa de erro sintático de 0%** na geração de comandos (sustentada teoricamente pela aplicação de *grammar prompting* [Wang et al. 2023]).

Qualitativamente, espera-se demonstrar uma manutenção estrita da fidelidade semântica em relação à intenção original do usuário, convertendo jargões e comandos ambíguos em instruções determinísticas seguras e superando de forma significativa as abordagens baseadas exclusivamente em *prompt engineering* heurístico.

## 1.6.3 Plano de Trabalho

Para alcançar os resultados almejados e finalizar a arquitetura de contenção neuro-simbólica, as prioridades de implementação para o semestre final foram organizadas em quatro frentes técnicas:

1. **Módulo AST-to-Graph Mapper:** Construção e automação da ponte entre a Árvore de Sintaxe Abstrata (AST), gerada pela DSL no Langium, e a inserção dinâmica de nós no *Knowledge Graph* (Neo4j), assegurando que o grafo reflita fielmente as regras de domínio.
2. **Otimização do Semantic Routing Engine:** Refinamento dos modelos de *embedding* e das estratégias de roteamento para garantir alta acurácia na ativação dos subgrafos de restrição a partir da linguagem coloquial de operadores não especializados.
3. **Retrieval do Grammar Prompting:** Estabelecimento do mecanismo de recuperação e composição dinâmica dos artefatos de *prompt*. O foco será orquestrar a extração

de parâmetros, contexto e regras gramaticais recuperados do grafo para construir *prompts* modulares [Wang et al. 2023] que alimentem o LLM.

4. **Extensão Adaptativa Impromptu:** Implementação da camada de compatibilidade final, responsável por garantir que as saídas (JSON estruturado e validado) sejam devidamente traduzidas em protocolos físicos de rádio-controle para a simulação de voo dos VANTs industriais.

1.6.4 Cronograma de Execução

O cronograma estabelecido na Tabela 1.1 detalha as fases de desenvolvimento das frentes técnicas, a coleta de métricas empíricas e a redação final do texto, projetadas para o semestre letivo de encerramento do mestrado (2026-2).

Tabela 1.1: Cronograma de atividades para o semestre final (2026-2)

| ID | Atividade                               | Mês 1 | Mês 2 | Mês 3 | Mês 4 | Mês 5 | Mês 6 |
|----|-----------------------------------------|-------|-------|-------|-------|-------|-------|
| 1  | Codificação do Mapper AST-Grafo         | X     | X     |       |       |       |       |
| 2  | Automação do Grammar Builder (BNF)      |       | X     | X     |       |       |       |
| 3  | Integração via LangChain e Fast API     |       |       | X     | X     |       |       |
| 4  | Coleta de Métricas e Testes de Estresse |       |       |       | X     | X     |       |
| 5  | Redação da Dissertação Final            |       |       |       | X     | X     | X     |
| 6  | Defesa Pública de Mestrado              |       |       |       |       |       | X     |

## Fundamentação Teórica

---

Para investigar a viabilidade de abstrair e validar a intenção de um usuário por meio de metodologias baseadas em Modelos de Linguagem de Grande Escala (LLMs), é imperativo estabelecer as bases teóricas que governam a interação entre a inteligência artificial generativa e os formalismos de software. Esta fundamentação organiza-se em cinco eixos: os princípios da Engenharia Dirigida por Modelos, os mecanismos e limitações dos LLMs, a restrição estrutural via Engenharia de Prompts e Grammar Prompting, a ancoragem factual através de Grafos de Conhecimento, e o papel dos sistemas multiagentes na garantia da confiabilidade sintática e semântica.

Este último eixo conceitual justifica-se como o mecanismo de controle de fluxo responsável por segmentar a inferência da linguagem natural em tarefas especializadas e independentes (como a classificação isolada de intenções e a auditoria sintática de saídas), mitigando falhas de processamento monolítico e atuando como uma camada de supervisão ativa para a garantia da confiabilidade semântica global do ecossistema.

Logo, para o fechamento sintático, no “grammar prompting” com a decodificação restrita, um LLM para criar a gramática prevista para a intenção do usuário, outra valida se o resultado confere à gramática prevista, na saída em tempo de execução, o que confere um sistema multiagente. Assim, garantida a semântica, ante do passo final, pelo “grammar prompting”, a confiabilidade semântica pode ser entendida pela arquitetura proposta pelo artigo, por ser guiada pelas restrições do modelo.

### 2.1 Engenharia Dirigida por Modelos (MDE) e Linguagens de Domínio Específico (DSL)

Engenharia Dirigida por Modelos (MDE - *Model-Driven Engineering*) fundamenta-se na premissa de que abstrações estruturadas e sintaticamente rigorosas (modelos) devem ser os artefatos primários no desenvolvimento e na execução de sistemas complexos. Conforme delineado no *roadmap* seminal de [France e Rumpe 2007], a MDE visa elevar o nível de abstração para além do código-fonte tradicional, utilizando

modelos bem definidos para lidar com a complexidade inerente aos domínios de aplicação. Nesse paradigma, as linguagens específico de domínio desempenham um papel estrutural: elas fornecem um vocabulário restrito e uma gramática formal que capturam exclusivamente as regras e entidades de um nicho específico, o que evita a expressividade desnecessária — e potencialmente perigosa — das linguagens de propósito geral. Em sistemas de missão crítica, a alta expressividade de linguagens tradicionais torna-se perigosa porque a sua Turing-completude impede a verificação estática completa de invariantes de segurança, abrindo margem para estados imprevistos, caminhos de execução ambíguos ou falhas lógicas catastróficas durante a interpretação de comandos operacionais [Hopcroft, Motwani e Ullman 2006, Mernik, Heering e Sloane 2005].

A convergência entre os preceitos da MDE e o potencial da IA generativa representa uma importante vertente no estado da arte da automação e especificação na engenharia de software contemporânea. Diante disso, [Clarísó e Cabot 2023] introduzem formalmente o conceito de *Model-Driven Prompt Engineering*, ao argumentar que, no desenvolvimento de sistemas que utilizam modelos de linguagem para tarefas de engenharia de sistemas, a construção de prompts não deve se apoiar em processos heurísticos, manuais ou puramente empíricos, mas sim ser guiada de forma sistemática por metamodelos estritos.

Os autores tipificam essa necessidade ao isolar duas fragilidades técnicas e problemas de engenharia distintos: por um lado, o desafio de otimização e design na criação de instruções robustas e contextualizadas para tarefas complexas; por outro, o problema de portabilidade e acoplamento tecnológico, decorrente da fragilidade de sistemas cujos prompts manuais tornam-se excessivamente dependentes de sintaxes proprietárias de APIs de LLMs concorrentes.

Para mitigar ambas as fragilidades, a Engenharia Dirigida por Modelos (MDE) atua como uma camada de isolamento metodológico. Em relação ao primeiro problema (otimização estrutural), a geração de prompts deixa de ser um esforço empírico de tentativa e erro; o contexto injetado no LLM passa a ser sistematicamente derivado dos metamodelos do sistema, garantindo que todas as regras de negócio e propriedades de domínio sejam incluídas na instrução de forma autônoma e estruturada. Quanto ao segundo problema (acoplamento tecnológico), a utilização da MDE cria uma abstração agnóstica em relação ao modelo fundacional subjacente. Caso o sistema precise migrar de um LLM proprietário (como o GPT-4) para um modelo de código aberto (como LLaMA), a lógica de negócios e as regras de segurança permanecem inalteradas na abstração, exigindo apenas a atualização das regras de transformação (model-to-text), o que garante a portabilidade da arquitetura.

Ao estabelecer essa esteira de tradução controlada pelas regras da MDE, obtém-se a evidência técnica necessária para garantir que a interação com a IA seja rastreável

e auditável. Toda saída em linguagem natural gerada pelo LLM é obrigatoriamente submetida a um analisador sintático (parser) que atua revertendo o processo: ele mapeia o texto gerado de volta para uma árvore de sintaxe abstrata (AST) compatível com a gramática da linguagem de domínio. Se o modelo de linguagem estocástico alucinar comandos, inventar parâmetros ou desviar do formato esperado, o mapeamento para a árvore AST falha imediatamente. Esse mecanismo de interrupção fornece a prova formal e empírica de violação semântica ou sintática antes que qualquer comando crítico seja enviado aos atuadores físicos do sistema.

Sob essa perspectiva, a DSL transcende a função tradicional de mera linguagem de documentação ou geração de código estático, passando a operar como uma especificação interpretável em tempo de execução. Para garantir que a saída do modelo nunca force a falha da AST, a arquitetura proposta atua preventivamente: ela acopla as regras gramaticais da DSL (em notação formal Backus-Naur Form - BNF) diretamente ao algoritmo de decodificação do LLM. Por meio de técnicas de restrição estrutural (*Grammar Prompting*), ocorre o mascaramento dinâmico da distribuição de probabilidade (*logits*) durante a geração de cada palavra; ou seja, a rede neural é matematicamente impedida de amostrar sequências de tokens que violem as ramificações permitidas pela gramática da DSL. A linguagem assume, portanto, o papel de um contrato de execução determinístico. Esse arranjo materializa as diretrizes gerais da abordagem proposta ao assegurar que a estocasticidade inerente aos modelos generativos seja mitigada, garantindo que qualquer intenção manifestada pelo usuário seja filtrada, validada e rigorosamente contida dentro das capacidades operacionais seguras e previsíveis mapeadas para o sistema autônomo.

## 2.2 Modelos de Linguagem de Grande Escala e o Processamento Semântico

Os modelos de linguagem de grande escala, fundamentados na arquitetura *transformers*, impactaram significativamente o processamento de linguagem natural ao empregar mecanismos de autoatenção (*self-attention*) para mapear dependências textuais complexas em sequências de dados [Husein, Aburajouh e Catal 2025, Yan et al. 2025]. Revisões sistemáticas recentes evidenciam que os LLMs atingiram uma eficácia notável em tarefas de tradução de linguagem natural para código (NL2Code) e no autocompletamento de código [Husein, Aburajouh e Catal 2025]. Este processo de autocompletamento consiste na predição iterativa de *tokens* subsequentes — como palavras-chave, variáveis e nomes de funções —, o que agiliza a escrita de software e reduz a carga cognitiva do desenvolvedor [Husein, Aburajouh e Catal 2025]. Suas aplicações expandiram-se para áreas de complexidade técnica, como a geração de código-fonte em ambientes de jogos

[Eberhardinger et al. 2025] e a geração automática de impressões radiológicas a partir da sumarização de laudos médicos brutos [Ma et al. 2024].

Apesar dessa alta expressividade na geração de conteúdo, a natureza inerentemente probabilística e estocástica dos LLMs introduz um risco crítico em domínios fechados: o fenômeno da alucinação [Ji et al. 2023]. Como detalhado por [Ji et al. 2023] em sua pesquisa sobre o tema, a alucinação em geração de linguagem natural ocorre quando o modelo produz textos que, embora sintaticamente fluentes e contextualmente plausíveis, carecem de factualidade ou desviam-se das premissas lógicas do domínio de entrada. Na engenharia de sistemas de missão crítica, a tolerância a falhas é mínima. No domínio de aplicação, incluem o diagnóstico autônomo de falhas em processos de manufatura aditiva 3D [Karabiyik 2025], bem como a automação de voo de drones agrícolas, que constitui o foco primário desta pesquisa. Nestes contextos cibernético-físicos, uma instrução parcialmente correta traduz-se inexoravelmente em uma operação insegura e perigosa.

Para além das falhas internas caracterizadas pela alucinação espontânea, esses modelos também enfrentam vulnerabilidades externas quando submetidos a processos de adaptação para novos domínios. Modelos puramente dependentes do *fine-tuning* tradicional — processo que reajusta massivamente os pesos originais da rede neural com novos conjuntos de dados para adaptar o modelo a uma tarefa específica — apresentam fragilidades a ataques adversariais textuais [Li et al. 2025]. Para mitigar essa fragilidade, demanda-se o uso de técnicas de adaptação estrutural, a exemplo do *prompt-tuning*.

Esta técnica atua otimizando comandos contínuos diretamente no espaço vetorial sem alterar os parâmetros do modelo pré-treinado, o que preserva a integridade da saída semântica contra perturbações não intencionais na entrada de dados (*input*) [Li et al. 2025]. Em paralelo, a adoção de técnicas de sintonia fina com eficiência de parâmetros, conhecidas sob a sigla em inglês PEFT (*Parameter-Efficient Fine-Tuning* — ajuste fino com eficiência de parâmetros), mitiga os elevados custos computacionais ao congelar a maior parte da rede e atualizar apenas subconjuntos específicos de *tokens* adaptativos [Yan et al. 2025].

Contudo, embora o *prompt-tuning* e o PEFT solucionem problemas computacionais e de alinhamento semântico, eles não resolvem a imprevisibilidade estrutural. Este problema conceitua-se nesta pesquisa como a incapacidade estocástica de garantir que as saídas textuais obedeçam estrita e invariavelmente a contratos sintáticos rígidos, como gramáticas formais. Essa lacuna lógica explicita que refinar os parâmetros de um modelo não substitui a necessidade de implementar restrições simbólicas externas voltadas para a governança das saídas.

## 2.3 Engenharia de Prompts Otimizada e Grammar Prompting

Para conter a fluidez excessiva dos LLMs, a engenharia de *prompts* evoluiu de abordagens baseadas em tentativa e erro para métodos de otimização automatizada. Trabalhos recentes procuram otimizar matematicamente as instruções de entrada para orientar a geração do modelo. A técnica AIPO, por exemplo, utiliza o conceito de ascensão de gradiente — um algoritmo de otimização que ajusta iterativamente os parâmetros de entrada para maximizar uma função de recompensa do modelo — com o intuito de refinar instruções automaticamente [Park e Kim 2026]. Similarmente, [Qin et al. 2025] demonstram a eficácia do preenchimento automático de contexto (CaFGD) empregando a descida de gradiente, uma técnica que minimiza uma função de perda para encontrar e estabilizar os *tokens* de gatilho ideais no *prompt*. Em contextos de aprendizagem *few-shot* — um paradigma de inferência onde o modelo é adaptado a uma nova tarefa fornecendo-lhe um pequeno número de exemplos de referência —, estratégias como *Composable Meta Prompts* (CMP) mostraram ser vitais para a adaptação em domínios com dados escassos [Chen et al. 2026]. Para a arquitetura CML proposta nesta investigação, o estudo destas técnicas é essencial para demonstrar que, embora mitiguem falhas, estas continuam a depender de otimização probabilística, o que justifica a necessidade de uma abordagem estrutural mais restritiva para sistemas críticos.

Contudo, no contexto específico da utilização de LLMs para a tradução de comandos em linguagem natural para linguagens específico de domínio, a otimização estocástica revela-se insuficiente para garantir o determinismo sintático exigido. A solução metodológica para este problema de imprevisibilidade estrutural emerge com a técnica de *grammar prompting*, introduzida por [Wang et al. 2023]. Esta técnica postula que, em vez de depender puramente da capacidade do LLM de imitar probabilisticamente a estrutura de um código com base em exemplos textuais, o sistema deve fornecer a representação estrita da linguagem alvo através da notação de Backus-Naur (BNF). Durante a inferência, a decodificação é restringida: o LLM é matematicamente forçado a selecionar apenas os *tokens* seguintes que mantenham a cadeia de texto em conformidade com as regras de produção da gramática. Isto resolve o problema da alucinação sintática, garantindo a geração de saídas que um sistema robótico ou compilador consiga processar sem falhas.

Esta abordagem converge diretamente com o *parsing* semântico neural — o processo computacional de mapear linguagem natural abstrata para representações lógicas formais e executáveis —, que constitui a tarefa basilar de interpretação pretendida por este trabalho. No desenvolvimento de agentes autónomos orientados à geração de código, como o *Spec2RTL-Agent* [Yan et al. 2025], a criação de lógicas complexas de *hardware* a partir de especificações ambíguas requer exatamente este tipo de constrição



sintática. Consequentemente, o *grammar prompting* representa o mecanismo de coerção sintática central desta arquitetura: ele garante que o modelo nunca gerará uma cadeia de caracteres (*string*) ou instrução de código que o analisador (*parser*) da DSL não consiga compreender. Conforme delineado na introdução, este elemento é o que viabiliza o fechamento semântico mútuo, isolando a operação física do *drone* das falhas estocásticas da inteligência artificial.

## 2.4 Grafos de Conhecimento e Retrieval-Augmented Generation (GraphRAG)

Enquanto a MDE e a técnica de *grammar prompting* asseguram a integridade sintática — isto é, a aderência estrita à estrutura formal da linguagem alvo —, grafos de conhecimento (KGs - *Knowledge Graphs*) são empregados nesta pesquisa para garantir a integridade semântica e factual. A técnica de geração aumentada por recuperação (RAG), formalizada primeiramente por [Lewis et al. 2020], impactou o processamento de tarefas intensivas em conhecimento ao permitir que redes neurais acessassem memórias não paramétricas (bancos de dados externos) para fundamentar suas respostas, mitigando de forma significativa as alucinações mapeadas por [Ji et al. 2023].

A evolução da técnica RAG baseada puramente em similaridade de vetores textuais densos para a recuperação estruturada em grafos (GraphRAG) apresenta-se como uma abordagem robusta para evitar falhas lógicas e de contexto em sistemas industriais. A pesquisa de [Bahr et al. 2025] valida esta premissa ao demonstrar que os KGs superam abordagens convencionais na Análise de Modos de Falha e Efeitos (FMEA) — uma metodologia sistemática de engenharia focada na identificação, avaliação e prevenção de falhas potenciais em processos de manufatura ou produtos. O uso do GraphRAG neste contexto é superior pois preserva e recupera relacionamentos determinísticos vitais (como a restrição “A Condição A causa a Falha B”), os quais são frequentemente perdidos na recuperação estocástica puramente vetorial. O uso de grafos de conhecimento, portanto, fornece ao LLM uma base ontológica rica, ao fundir *prompts* dinâmicos com regras estritas de domínio — técnica exemplificada com sucesso na previsão temporal de parâmetros na construção civil [Zhang et al. 2025] e no diagnóstico analítico de falhas em equipamentos de energia elétrica [?] —, o modelo de Inteligência Artificial não precisa deduzir as restrições operacionais do mundo real; ele as consulta diretamente da base de dados.

Para a arquitetura aprimorada CML (*Conceptual Model Learning*) proposta nesta dissertação, o entendimento e a aplicação dessa dinâmica são essenciais. Na literatura, os métodos descritos por [Shi et al. 2025] ilustram a eficácia de se integrar a recuperação



no grafo de conhecimento a processos de *Chain-of-Thought* (Raciocínio em Cadeia) para gerar instruções de manutenção industrial passo a passo, evidenciando como os grafos fornecem guias lógicos seguros para orientar a tomada de decisão da IA. Similarmente, [Feng et al. 2025] demonstram a sinergia entre grafos e LLMs para moldar *prompts* na área de geração e *design* de interfaces (UI), mapeando os intrincados relacionamentos entre requisitos informais do usuário e especificações técnicas precisas. Aderindo a esses princípios, a modelagem de domínio em uma ferramenta orientada a grafos (como o Neo4J) atua na arquitetura CML como um mecanismo de validação semântica e factual estrita. A sua finalidade operacional não é barrar a intenção inicial humana, mas sim proteger o ecossistema contra as interpretações errôneas e as alucinações produzidas pelo LLM na tentativa de traduzir essa intenção. Dessa forma, impede-se a geração de comandos que, embora possam ser gramaticalmente e sintaticamente válidos para a DSL, violariam as invariantes físicas, restrições químicas ou lógicas de segurança do ambiente operacional de missão crítica.

## 2.5 Sistemas Multiagentes na Garantia da Confiabilidade Semântica

O emprego de Sistemas Multiagentes (MAS – *Multi-Agent Systems*) baseia-se na necessidade de mitigar a vulnerabilidade de ponto único de falha associada ao processamento monolítico LLMs [Bai et al. 2025]. Em tarefas de alta complexidade cognitiva, a delegação de pipelines extensos a uma única instância inferencial eleva a probabilidade de propagação de erros estocásticos em cascata [Bai et al. 2025]. A acumulação de desvios probabilísticos ao longo do contexto de geração pode comprometer as etapas subsequentes de síntese sintática caso ocorra uma falha na recuperação semântica inicial. Dessa forma, a descentralização do fluxo computacional por meio da segregação de responsabilidades funcionais constitui um mecanismo para assegurar a estabilidade e o isolamento de falhas lógicas no sistema [Bai et al. 2025].

Sob a perspectiva da engenharia de software distribuída, um agente é definido como um módulo de computação autônomo e assíncrono, caracterizado por especialização funcional e delimitação de escopo operacional [Bai et al. 2025]. Na infraestrutura de *back-end*, esses componentes operam como unidades de controle dedicadas, distanciando-se de interfaces de conversação genéricas [Chen et al. 2026]. A literatura demonstra que a segmentação do fluxo inferencial em nós especializados confere maior resiliência a arquiteturas complexas frente a inconsistências de dados e falsos positivos [Chen et al. 2026]. Adicionalmente, o fracionamento do raciocínio automatizado em etapas lógicas sequenciais, integradas a bases de conhecimento externas, atua diretamente na redução de aluci-

nações semânticas [Shi et al. 2025].

A evolução recente no domínio de geração automatizada de código (NL2Code) - geração automatizada de código a partir de linguagem natural, constitui o paradigma fundamental para a tradução de intenções humanas em artefatos de *software* executáveis [Bai et al. 2025, Husein, Aburajouh e Catal 2025]- evidencia uma transição estrutural de abordagens baseadas em comandos isolados (*single-prompt*) para frameworks multifásicos colaborativos [Bai et al. 2025]. A organização de tarefas em estágios sequenciais de definição de escopo, escrita sintática e auditoria independente maximiza a segurança e a correte formal do artefato gerado [Bai et al. 2025]. Para cenários de missão crítica com severas restrições de conformidade, a implementação de ciclos iterativos contendo etapas de planejamento, execução e reflexão gramatical automatizada provê garantias adicionais contra desvios de especificação [Yu et al. 2025].

A integração deste paradigma à arquitetura baseada em CML fundamenta o mecanismo de controle de fluxo e governança de dados proposto. Ao estabelecer papéis isolados para os microsserviços do sistema — dividindo a execução entre um componente restrito ao roteamento no Grafo de Conhecimento (*Semantic Routing Engine*) e outro focado no mapeamento sintático para a Linguagem de Domínio Específico (*AST Mapper*) —, instituem-se pontos de controle invariantes baseados nos padrões de verificação multiestágio descritos na literatura [Bai et al. 2025]. A troca de mensagens estruturadas entre os módulos especializados viabiliza a auditoria estática de representações intermediárias antes da consolidação da Árvore de Sintaxe Abstrata (AST) [Yu et al. 2025]. Este encaimento modular restringe a propagação de inconsistências probabilísticas, impedindo que falhas lógicas alcancem os atuadores físicos do sistema autônomo [Chen et al. 2026].

## 2.6 Conclusão do Capítulo

Este capítulo estabeleceu a fundamentação teórica para sustentar o desenvolvimento de arquiteturas de software voltadas à abstração, inferência e validação de intenções por meio do acoplamento entre inteligência artificial generativa e métodos formais. Neste contexto de pesquisa, os métodos formais são definidos estritamente como a aplicação de gramáticas regulares (utilizando a notação *Backus-Naur Form* - BNF) para a verificação estática de sintaxe e o emprego de ontologias estruturadas em grafos para a validação matemática de regras lógicas do domínio, enquadrando-se na pesquisa como a camada que assegura a previsibilidade e a segurança das saídas operacionais [Wang et al. 2023, Bahr et al. 2025].

A discussão foi estruturada a partir de cinco eixos conceituais complementares que operam de forma integrada. Inicialmente, estabeleceu-se a engenharia dirigida por modelos e as linguagens específico de domínio não apenas como ferramentas des-

critivas, mas como infraestruturas de restrição que substituem a expressividade de linguagens de propósito geral por contratos de execução de escopo estritamente fechado [Clariso e Cabot 2023, Morris et al. 2015]. Na sequência, contrapôs-se a alta capacidade de processamento semântico dos LLMs à sua vulnerabilidade inerente à alucinação [Ji et al. 2023]. Para mitigar essa estocasticidade, detalhou-se o *grammar prompting*, conceituado como o mecanismo de mascaramento dinâmico de *logits* que impõe a coerção sintática em nível de *tokens* durante a fase de decodificação do modelo [Wang et al. 2023].

Complementando a integridade estrutural, demonstrou-se a relevância da técnica *GraphRAG* (geração aumentada por recuperação baseada em grafos), definida como o processo de indexação e busca topológica que fornece à rede neural uma fonte de verdade externa, auditável e imutável [Bahr et al. 2025]. Por fim, a orquestração do ecossistema foi justificada pelos sistemas multiagentes, caracterizados computacionalmente como uma arquitetura distribuída de módulos autônomos e especializados que executam a validação cruzada do código gerado, suprimindo o risco de pontos únicos de falha [Bai et al. 2025]. Coletivamente, as definições teóricas desses cinco eixos estabelecem os requisitos de controle estrutural e de governança de dados necessários para a consolidação da arquitetura baseada em CML proposta nesta dissertação.

Uma vez delineadas as fronteiras teóricas e a instrumentação conceitual que justificam a necessidade de um ecossistema metodológico híbrido para governar o comportamento dos LLMs, torna-se imperativo examinar como esses conceitos vêm sendo efetivamente combinados e validados pela comunidade científica internacional. Com esse propósito, o Capítulo 3 apresenta Revisão Sistemática da Literatura (RSL). Essa transição permitirá consolidar o entendimento do estado da arte prático e identificar os padrões arquiteturais de integração vigentes — conceituados neste trabalho como as topologias recorrentes de *software* que a indústria e a academia aplicam para integrar redes neurais estocásticas a bases de conhecimento determinísticas [Husein, Aburajouh e Catal 2025]. O mapeamento destas topologias evidenciará as lacunas de governança formal ainda não resolvidas pela literatura, legitimando cientificamente a solução tecnológica desenvolvida nesta pesquisa.

## Engenharia de Prompts Dirigida por Modelos: Revisão Sistemática da Literatura

---

A arquitetura proposta nesta qualificação encontra sua fundamentação teórica e prática nos resultados desta Revisão Sistemática da Literatura (RSL). Caracterizada como um estudo de natureza exploratório-descritiva, esta RSL tem como tema central norteador a Engenharia de Prompts Dirigida por Modelos (*Model-Driven Prompt Engineering*) [Clariso e Cabot 2023]. O objetivo precípua deste estudo consiste em investigar, identificar e analisar abordagens que utilizam modelos conceituais, ontológicos ou de conhecimento como base para a geração automática de *prompts* aplicados a Modelos de Linguagem de Grande Escala (LLMs).

Busca-se compreender como esses modelos de referência são empregados para estruturar, guiar ou automatizar a criação de instruções, avaliando as técnicas, *frameworks*, benefícios e limitações relatados na literatura científica atual. Para assegurar a validade, o alcance e o alinhamento dos achados, o pipeline de pesquisa seguiu rigorosamente as diretrizes metodológicas consolidadas por [Kitchenham e Charters 2007] para a Engenharia de Software. Essa abordagem garantiu a execução de um processo intrinsecamente rigoroso, transparente, imparcial e auditável (reprodutível), mitigando de forma ativa os vieses de seleção e estabelecendo o rigor científico indispensável para delimitar e fundamentar a interseção entre métodos formais e inteligência artificial generativa.

No contexto da ciência da computação, métodos formais constituem um conjunto de técnicas fundamentadas em formalismos matemáticos discretos e lógicos para a especificação, desenvolvimento e verificação de propriedades invariantes em sistemas de software e hardware [France e Rumpe 2007]. Sob o escopo desta pesquisa, os formalismos matemáticos não são aplicados diretamente à verificação estatística interna dos pesos dos LLMs, mas sim materializados como restrições gramaticais externas baseadas em especificações estritas de Linguagens de Domínio Específico (DSLs) [Mernik, Heering e Sloane 2005, Hopcroft, Motwani e Ullman 2006]. Essas estruturas axiomáticas atuam como barreiras de controle lógico para neutralizar a estocasticidade inerente às redes neurais autorregressivas [Wang et al. 2023].

## 3.1 O Problema de Pesquisa

A recente expansão das capacidades computacionais dos LLMs permitiu que esses modelos transcendam tarefas de conversação genérica e passem a ser integrados a domínios complexos de engenharia. A literatura recente demonstra a viabilidade de aplicação dessas redes neurais autoregressivas em cenários de alta exigência lógica e técnica, que incluem a síntese automatizada de hardware em níveis de transferência de registradores (RTL) a partir de especificações complexas [Yu et al. 2025], o diagnóstico preditivo de falhas em subestações e equipamentos de potência [Wen et al. 2025], a modelagem preditiva de cargas energéticas estruturais estruturadas [Zhang et al. 2025], a geração contextualizada de dados de entrada para automação de testes funcionais de interfaces gráficas móveis [Liu et al. 2023] e a confecção de laudos estruturados orientados por grafos de conhecimento industrial [Shi et al. 2025].

Contudo, a transição desses modelos conexionistas para ambientes operacionais críticos expõe uma vulnerabilidade estrutural severa: a ausência de garantias lógicas sobre as saídas geradas. Devido à sua arquitetura probabilística, os LLMs são propensos a sofrer o fenômeno da alucinação, produzindo saídas linguisticamente fluentes e coerentes sob o ponto de vista formal, mas que carecem de factualidade, violam regras lógicas de negócio ou introduzem quebras severas de restrições de domínio [Ji et al. 2023, Bahr et al. 2025].

Estudos de revisão sistemática focados no cenário de geração e complementação de código de software (*code completion*) [Husein, Aburajouh e Catal 2025, Umama et al. 2025] indicam que, embora os modelos preditivos de tokens demonstrem alta capacidade contextual, o desenvolvimento na área ainda se encontra fragmentado e suscetível à ambiguidade da linguagem natural. A prática predominante de *prompt engineering* puramente manual e heurística — baseada em tentativa e erro humana (*ad-hoc prompting*) — mostra-se dispendiosa, inconsistente e incapaz de oferecer o determinismo necessário para sistemas automatizados de engenharia.

Essa fragilidade ocorre porque o espaço latente do modelo falha em capturar rigidamente restrições estruturais abstratas quando guiado unicamente por strings livres. O espaço latente constitui o ambiente matemático contínuo, vetorial e de alta dimensionalidade onde as camadas ocultas das arquiteturas baseadas em *transformers* projetam e manipulam representações densas (*embeddings*) dos tokens com base em co-ocorrências estatísticas aprendidas no pré-treinamento [Yan et al. 2025, Sun et al. 2020]. Por operar exclusivamente através de distribuições de probabilidade e proximidade vetorial para calcular o próximo token, o espaço latente carece nativamente de mecanismos discretos capazes de forçar o cumprimento absoluto de invariantes lógicas, resultando em desvios dedutivos ou erros de compilação (*parsing errors*) em tarefas de precisão [Qin et al. 2025, Wang et al. 2023].

Diante desse gargalo, surge a necessidade premente de substituir o design empírico de instruções por mecanismos híbridos de governança de software dirigidos por modelos. O problema de pesquisa desta revisão concentra-se na Engenharia de Prompts Dirigida por Modelos (*Model-Driven Prompt Engineering*) [Clarísó e Cabot 2023], investigando de que forma modelos conceituais, ontologias, grafos de conhecimento e gramáticas formais são empregados como estruturas estáveis para guiar e restringir a síntese de instruções de forma computacionalmente blindada.

Essa investigação estabelece a justificativa direta e o alicerce para a arquitetura CML (*Conceptual Model Learning*) proposta nesta dissertação. A Arquitetura CML resolve de forma sistemática a volatilidade do espaço latente documentada pela literatura ao acoplar modelos conceituais abstratos à interpretação em tempo de execução (*runtime models*) [Morris et al. 2015, Allison et al. 2014]. Através de uma separação rígida de responsabilidades entre um esquema de controle (ancorado em uma gramática BNF restrita) e um esquema de dados (alimentado por subgrafos factuais de um grafo de conhecimento), a abordagem CML substitui as heurísticas empíricas por um pipeline de decodificação restrita via *grammar prompting* [Wang et al. 2023]. A RSL permite conceber os princípios necessários para assegurar a tradução robusta e determinística de intenções de usuário em artefatos válidos e seguros para domínios de alta complexidade.

## 3.2 Metodologia e Protocolo da Revisão

O planejamento e a condução desta revisão sistemática da literatura (RSL) foram estruturados rigorosamente a partir de um processo metodológico composto por três fases interdependentes, adaptadas das diretrizes consolidadas por [Kitchenham e Charters 2007] para a Engenharia de Software:

1. **Fase 1 - Formulação do Protocolo de Pesquisa:** Etapa de planejamento onde foram definidas as questões norteadoras, as estratégias de indexação e os critérios de aceitação dos estudos. Como componente indissociável da formulação deste protocolo, foi definida que a extração de dados utilizaria um método de síntese temática parametrizada, mapeando sistematicamente as primitivas arquiteturais de cada estudo (como o tipo de modelo axiomático, a linguagem-alvo e as barreiras lógicas de contenção).
2. **Fase 2 - Execução da Busca e Filtragem dos Estudos:** Etapa operacional dedicada à execução automatizada das strings lógicas nas bases de dados e à aplicação sequencial dos filtros de inclusão e exclusão sobre o montante recuperado.
3. **Fase 3 - Extração Analítica e Síntese de Evidências:** Etapa analítica onde os 20 artigos finais aceitos foram integralmente decodificados com base na matriz

analítica estabelecida na Fase 1, permitindo agrupar as soluções da literatura e identificar lacunas formais que justificam parte do desenvolvimento desta pesquisa.

### 3.2.1 Questões de Pesquisa (QPs)

Para orientar a varredura da literatura e garantir que a síntese de evidências responda diretamente ao problema de pesquisa delimitado, foram formuladas três questões de pesquisa:

- **QP1:** Quais abordagens e técnicas formais estão sendo utilizadas para gerar ou otimizar automaticamente *prompts* com base em modelos conceituais, ontológicos ou estruturas de conhecimento?
- **QP2:** Como o uso de modelos formais e grafos de conhecimento impacta a capacidade dos LLMs de gerar código (NL2Code) ou executar tarefas de raciocínio lógico em domínios específicos?
- **QP3:** Quais são os principais benefícios e limitações relatados na literatura ao restringir o espaço de busca dos LLMs através de *prompts* dirigidos por modelos em comparação com abordagens puramente generativas?

### 3.2.2 Estratégia de Busca e Bases de Dados

A formulação da estratégia de busca e a consequente execução das varreduras lógicas foram conduzidas nos indexadores científicos de maior impacto e relevância internacional na área de Ciência da Computação e sub-área: *ACM Digital Library*, *IEEE Xplore* e *ScienceDirect (Elsevier)*.

Visando capturar com precisão a interseção entre automação de instruções, engenharia de modelos e modelos de linguagem, foi elaborado a seguinte *string* de busca booleana aplicável aos campos de título, resumo (*abstract*), corpo (*body*) e palavras-chave (*keywords*) dos artigos indexados nas bases acima mencionadas:

```
("automatic prompt generation" OR "auto prompt" OR "prompt  
generation" OR "auto-generated prompts" OR "self-prompting"  
OR "prompt synthesis")  
AND ("model-based" OR "model-driven")  
AND ("large language model" OR "LLM")
```

### 3.2.3 Critérios de Seleção (Inclusão e Exclusão)

A triagem dos estudos recuperados baseou-se em critérios estritos de inclusão e exclusão, previamente delimitados no protocolo para evitar vieses *ad-hoc* de seleção.

**Critérios de Inclusão (CI):**

1. **CI1 - Criação de *prompt* a partir de modelo:** O estudo deve propor um arcabouço onde a formulation do *prompt* seja derivada sistematicamente de uma estrutura abstrata, ontologia, metamodelo ou modelo conceitual pré-definido.
2. **CI2 - Geração de Código por LLM (NL2Code):** O estudo aplica ou avalia a técnica no contexto de tradução de linguagem natural para artefatos de código, garantindo aderência ao mapeamento de intenções linguísticas em artefatos executáveis ou expressões formais com comportamento puramente determinístico, como scripts de DSL ou código estruturado compilável.
3. **CI3 - Automação da Síntese de Prompts:** O estudo detalha algoritmos, sistemas multiagentes ou métodos computacionais para a geração autônoma de instruções, eliminando o fator puramente empírico manual.
4. **CI4 - Refinamento Dirigido de Prompts:** O estudo aborda técnicas exclusivamente autônomas e algorítmicas de otimização iterativa, calibração ou *prompt-tuning* (como otimização por descida de gradiente ou sintonia em tempo de execução baseada em feedback do compilador), operando sem intervenção ou ajuste heurístico manual humano.

#### **Critérios de Exclusão (CE):**

1. **CE1 - Obsolescência Tecnológica (Janela Temporal):** Publicações com data de corte superior a 8 anos. *Justificativa:* A arquitetura *transformer* e os modelos massivos autoregressivos de bilhões de parâmetros definem um ecossistema contemporâneo muito dinâmico; estudos anteriores não refletem as propriedades emergentes dos LLMs modernos.
2. **CE2 - Barreiras Linguísticas:** Estudos que não estejam redigidos integralmente na língua inglesa, inviabilizando a validação por pares nos principais fóruns globais.
3. **CE3 - Desvio de Escopo Algorítmico:** Trabalhos baseados em algoritmos clássicos de aprendizado de máquina ou inteligência artificial simbólica pura que usem o termo “prompt” com semânticas distintas.
4. **CE4 - Aplicações Conversacionais Genéricas:** Pesquisas nas quais o LLM é avaliado apenas em tarefas criativas, resumos literários ou conversação livre (*chatbots*), cujos domínios carecem de requisitos estritos de precisão lógica, consistência factual ou conformidade formal com regras de execução estruturadas e computacionalmente verificáveis.
5. **CE5 - Engenharia de Prompts Heurística/Manual:** Estudos baseados exclusivamente em abordagens empíricas de tentativa e erro humana (*ad-hoc prompting*), sem a presença de um modelo formal ou matemático de controle.



## 3.3 Resultados e Discussão

Esta seção consolida o fluxo de triagem dos dados coletados, apresenta o mapeamento taxonômico da literatura e responde empiricamente às questões de pesquisa delineadas.

### 3.3.1 Fluxo de Seleção dos Estudos Primários

A execução da *string* de busca resultou em uma amostra bruta inicial de 519 estudos primários, indexados da seguinte forma: 235 oriundos da *IEEE Xplore*, 165 recuperados da *ACM Digital Library* e 119 localizados na *ScienceDirect*.

Após a remoção automática de duplicatas, os títulos e resumos foram submetidos aos Critérios de Exclusão (CE), descartando trabalhos desalinhados. Os artigos remanescentes foram lidos na íntegra e confrontados com os Critérios de Inclusão (CI). Ao término desta filtragem rigorosa, um portfólio definitivo de 20 artigos qualificados foi retido para o processo de extração e síntese de evidências.

### 3.3.2 Mapeamento Temático e Eixos de Análise

A análise cruzada do portfólio evidenciou um movimento de consolidação metodológica. A literatura científica deixou de tratar o *prompt* como uma “frase estática” e passou a adotá-lo como uma camada de interface dinâmica inserida em arquiteturas de software híbridas, orientadas a modelos e guiadas por restrições formais [Clariso e Cabot 2023]. O estado da arte organiza-se em três eixos fundamentais:

#### Eixo 1: Otimização Algorítmica e Espaço Latente (Gradient-Based Optimization)

A literatura demonstra uma forte tendência em substituir o design de prompts baseado em heurísticas humanas por algoritmos matemáticos que operam sobre o espaço vetorial dos modelos. Em [Park e Kim 2026], os autores propuseram o framework AIPO (*Automatic Instruction Prompt Optimization*), o qual utiliza técnicas inspiradas em ascensão de gradiente (*Gradient Ascent*) executadas pelo próprio modelo para calibrar instruções intrínsecas. Entende-se por instruções intrínsecas as representações contínuas de prompts digitais ou vetores virtuais otimizados diretamente nas camadas de incorporação (*embeddings*) do modelo, maximizando a acurácia de tarefas matemáticas e de raciocínio sem a necessidade de intervenção manual na linguagem natural.

Complementarmente, em [Qin et al. 2025] foi introduzido o framework CaFGD (*Context Auto-Filling via Gradient Descent*), comprovando que a busca e inserção automatizada de tokens gatilho baseada no cálculo da média dos gradientes reduz drástica e sistematicamente a instabilidade semântica frente a entradas incertas dos usuários. No

escopo deste trabalho, os frameworks de otimização contínua baseados em gradientes (AIPO e CaFGD) possuem relevância metodológica estrita como linhas de base (*base-lines*) para análise comparativa. Eles servem para contrapor abordagens de otimização numérica no espaço latente versus o mecanismo de coerção discreta e sintática proposto pela Arquitetura CML.

Ademais, a automação algorítmica do design de prompts pode ser combinada a mecanismos de autocorreção de código em execução para otimizar a geração de scripts específicos, como demonstrado no domínio de predição de carga energética [Zhang et al. 2025]. Nessa abordagem, o ecossistema é estruturado como um *loop* fechado em que o LLM atua acoplado a um interpretador ou motor de simulação externo. Esse ambiente dinâmico devolve mensagens de erro de execução (como falhas de compilação ou violações de limites numéricos), guiando o modelo em um processo de refinamento automatizado. Tal refinamento caracteriza-se como a reescrita iterativa e autônoma do prompt e do script gerado com base no *feedback* do compilador, mitigando erros lógicos e garantindo a viabilidade sintática do programa. No contexto desta dissertação, o princípio desse *loop* de retroalimentação é altamente complementar, pois ilustra como a validação empírica pós-geração pode atuar em sinergia com as restrições estruturais estabelecidas *a priori* pelas gramáticas de domínio.

## Eixo 2: Governança Semântica via Grafos de Conhecimento e Ontologias

O segundo eixo mapeado aborda a restrição factual e o alinhamento de domínio através do uso de modelos estruturados em grafos, mitigando o problema das alucinações por meio do fechamento semântico. O fechamento semântico estabelece-se aqui como a técnica de delimitação lógica que restringe as fronteiras de interpretação do LLM a um conjunto finito e verificável de estados válidos, mapeados a partir de regras invariantes do domínio. Consoante, o estudo de [Karabiyik 2025] demonstrou a eficácia desse acoplamento ao contexto de recuperação dos modelos no domínio industrial ao desenvolver o *Prompt Evaluation Framework* (PEF) para análise de falhas em manufatura aditiva, integrando visão computacional e LLMs estruturados por regras para isolar falhas de impressão 3D.

Para além da correção de código em tempo de execução, a estruturação de contexto por meio de modelos ontológicos mostra-se crucial para restringir a inferência dos LLMs. Grafos de conhecimento definem-se como redes topológicas estruturadas que representam semanticamente um domínio de discurso por meio de entidades (nós) e relacionamentos tipados (arestas), fornecendo uma base axiomática e determinística de consulta. A integração desses grafos ao raciocínio em cadeia tem sido explorada para guiar modelos, passo a passo, na geração automatizada de artefatos complexos, a exemplo de documentos estruturados de manutenção industrial via fusão de *Graph Retrieval-Augmented*

*Generation* (GraphRAG) e *Chain-of-Thought* (CoT) [Shi et al. 2025]. Essa capacidade de ancoragem ontológica atua diretamente na prevenção de desvios dedutivos e alucinações, como comprovado no desenvolvimento de sistemas analíticos visuais para o diagnóstico de anomalias em subestações de energia [Wen et al. 2025]. A mesma lógica de tradução guiada por grafos aplica-se na transformação de abstrações em estruturas concretas, sendo utilizada em conjunto com a Engenharia Kansei - abordagem metodológica que traduz sentimentos, impressões e demandas cognitivas do usuário em especificações técnicas e métricas de design de produto - para mapear demandas psicológicas e subjetivas em parâmetros técnicos para a geração de interfaces de usuário [Feng et al. 2025].

No contexto desta pesquisa, o uso de representações baseadas em grafos para balizar o raciocínio do LLM alinha-se diretamente à arquitetura proposta para o ecossistema do projeto. Assim como os estudos supracitados utilizam bases de conhecimento para ancorar o modelo e traduzir linguagens abstratas em artefatos técnicos escritos, este trabalho adota a persistência em grafos (via Neo4j) e a roteirização semântica para garantir que a intenção orgânica do usuário seja mapeada de forma precisa e determinística para os construtos formais da DSL).

### **Eixo 3: Engenharia de Software Multiagente e Gramáticas Formais em NL2Code**

No âmbito específico da geração de código (NL2Code), a literatura foca em arquiteturas multiagentes descentralizadas e no controle sintático por restrições gramaticais. O estudo de [Bai et al. 2025] aborda as ambiguidades crônicas de descrições humanas em linguagem natural introduzindo uma arquitetura de colaboração multiagente. Descrições humanas caracterizam-se como requisitos de software fornecidos por usuários finais desprovidos de jargão técnico, marcados por alta polissemia, vagueza semântica e ausência de especificações formais de restrição. O pipeline fragmenta o problema em quatro macroestágios determinísticos (definição de papéis, otimização das demandas, escrita de código e revisão técnica), gerando um ganho de robustez em comparação com abordagens monolíticas.

A automação em nível de especificação complexa — mapeando requisitos de engenharia intrincados diretamente para linguagens estruturadas — apresenta alta pertinência a esta dissertação. Esse cenário é evidenciado no framework Spec2RTL-Agent, proposto por [Yu et al. 2025], que utiliza uma equipe de agentes baseados em LLMs para processar documentos densos de especificações técnicas de hardware e gerar código RTL com baixo índice de erro, sem necessidade de supervisão humana contínua. Por fim, [Eberhardinger et al. 2025] demonstraram que a aplicação de algoritmos de busca estocástica local, como o *hill-climbing*, onde as mutações estruturais do programa são coordenadas logicamente por LLMs, supera o desempenho de modelos massivos não controlados em tarefas complexas de síntese de código de jogos (Python/Java).

Essas técnicas de restrição sintática e coordenação multiagente (Spec2RTL-Agent e o método baseado em *hill-climbing* são diretamente relevantes para o arcabouço desta pesquisa, pois validam empiricamente o argumento de que a imposição de restrições arquiteturais e de controle externo sobre a geração de *tokens* é mais eficaz para a corretude de programas do que o simples aumento na escala de parâmetros do modelo conexionista. Isso fundamenta e legitima a adoção do motor de coerção gramatical baseado em gramáticas BNF adotado no ecossistema CML.

### 3.3.3 Síntese e Respostas às Questões de Pesquisa

A consolidação analítica dos 20 estudos primários fornece as seguintes respostas formais às QPs estabelecidas no protocolo:

- **Resposta à QP1 (Técnicas de Automação):** O estado da arte apoia-se em três pilares para a automação e otimização de prompts: (i) *Otimização Matemática Baseada em Gradientes*, que mapeia tokens gatilho ideais no espaço latente do modelo, definido como o ambiente vetorial contínuo e multidimensional onde as camadas internas dos \*Transformers\* processam as distribuições probabilísticas de adjacência de tokens [Park e Kim 2026, Qin et al. 2025]; (ii) *Síntese Dirigida por Grafos de Conhecimento (GraphRAG)*, convertendo caminhos e ontologias relacionais estruturadas diretamente em contextos textuais delimitados de consulta factual [Shi et al. 2025, Feng et al. 2025]; e (iii) *Orquestração de Pipelines Multiagentes*, onde prompts especializados segmentam tarefas de raciocínio de forma modular, dividindo um problema complexo em subproblemas sequenciais gerenciados por papéis computacionais específicos [Bai et al. 2025, Yu et al. 2025].
- **Resposta à QP2 (Impacto de Modelos Formais e Grafos):** A introdução de modelos formais e grafos de conhecimento atua como um mecanismo de governança e barreira de contenção semântica, que se traduz na imposição em tempo de execução de filtros e restrições estruturais lógicas que impedem o modelo de atingir estados gramaticais inválidos ou respostas fora do escopo de validade do domínio. Em tarefas de raciocínio e geração de código (NL2Code), os Grafos de Conhecimento injetam a *ancoragem factual* e o contexto de mundo real que redes neurais puras não possuem, eliminando alucinações de domínio [Karabiyik 2025, Wen et al. 2025]. Simultaneamente, gramáticas e restrições formais eliminam falhas de compilação (*parsing errors*), forçando o LLM a atuar como um componente determinístico acoplado a um compilador ou interpretador [Eberhardinger et al. 2025].
- **Resposta à QP3 (Benefícios e Limitações):** Os **benefícios** proeminentes incluem a garantia de repetibilidade sistêmica, alta estabilidade sintática na geração de código [Eberhardinger et al. 2025], portabilidade de domínio [Clarísó e Cabot 2023] e

drástica redução de custos operacionais, uma vez que a injeção dinâmica de conhecimento estruturado baseado em modelos elimina a necessidade de processos caros de ajuste fino (*fine-tuning*) ou re-treinamento de infraestrutura [Zhang et al. 2025]. Em contrapartida, as **limitações** críticas envolvem a latência computacional acumulada em arquiteturas multiagentes iterativas [Bai et al. 2025] e a forte dependência de uma engenharia de dados prévia: se o grafo de conhecimento ou o metamodelo inicial contiver falhas de modelagem ou inconsistências lógicas, o LLM propagará o erro de forma determinística, tornando a curadoria do modelo formal o novo gargalo de desenvolvimento de sistemas inteligentes [Shi et al. 2025].

Os achados desta Revisão Sistemática da Literatura validam empiricamente a hipótese que sustenta esta pesquisa, conforme evidenciado no Eixo 3 através das avaliações comparativas de [Eberhardinger et al. 2025]: o desenvolvimento de sistemas baseados em LLMs para cenários críticos não deve depender do aumento indefinido do tamanho ou da escala de parâmetros dos modelos, mas sim do design de arquiteturas híbridas e integradas, que combinem a flexibilidade cognitiva generativa com a previsibilidade matemática de engenharia orientada a modelos.

### 3.4 Conclusão do Capítulo

Este capítulo apresentou, de forma estruturada e metodologicamente rigorosa, uma Revisão Sistemática da Literatura (RSL) sobre o panorama emergente da Engenharia de Prompts Dirigida por Modelos. Amparada pelas diretrizes consolidadas de Kitchenham e Charters [Kitchenham e Charters 2007], a investigação partiu de um universo bruto de 519 estudos primários indexados nas bases ACM Digital Library, IEEE Xplore e ScienceDirect, culminando na seleção e extração analítica de 20 artigos. A síntese dessas evidências empíricas permitiu mapear o estado da arte em três eixos tecnológicos fundamentais: a otimização matemática baseada em gradientes, a governança semântica via grafos de conhecimento (GraphRAG) e a especificação sintática por meio de gramáticas formais e pipelines multiagentes. As respostas fundamentadas às três questões de pesquisa (QPs) ratificaram a viabilidade e a urgência de abordagens de orquestração híbrida, demonstrando que a robustez e a previsibilidade em sistemas de missão crítica dependem do acoplamento entre modelos conceituais determinísticos e a capacidade generativa dos LLMs.

Torna-se imperativo, contudo, esclarecer ao leitor a distinção de escopo existente entre as contribuições deste capítulo e o conteúdo a ser apresentado no Capítulo 4 (*Trabalhos Relacionados*). Esta RSL operou sob uma perspectiva macroscópica e metodológica, mapeando de forma exaustiva as tendências de pesquisa, os limites paradigmáticos e o

portfólio consolidado da literatura internacional sobre a fusão entre Engenharia Dirigida por Modelos (MDE) e modelos de linguagem, seguindo filtros formais de busca e critérios rígidos de exclusão por pares.

Por outro lado, o próximo capítulo, intitulado **Trabalhos Relacionados**, estabelecerá um exame microscópico, tecnológico e taxonômico focado unicamente em ferramentas de software, *toolkits*, interpretadores em tempo de execução e frameworks arquiteturais de decodificação restrita (como *Outlines*, *Guidance* e compiladores de DSLs) que resolvem problemas concretos de correspondência sintática. Essa transição permitirá contrastar diretamente a mecânica operacional da Arquitetura CML com as iniciativas de engenharia de software de implementação mais próximas de seu escopo técnico, evidenciando as lacunas funcionais e os gargalos de acoplamento que este trabalho visa preencher.

## Trabalhos Relacionados

---

Este capítulo apresenta uma análise crítica e sistemática dos estudos que fundamentam o estado da arte nas áreas de interseção entre a Inteligência Artificial Generativa e a Engenharia de Software formal. O objetivo primordial desta revisão é situar a proposta desta dissertação no cenário científico contemporâneo, com o intuito de mapear as principais frentes de investigação que buscam mitigar a lacuna entre a expressividade estocástica da linguagem natural e o determinismo operacional exigido por sistemas governados por modelos de domínio.

### 4.1 Interpretação de Intenção Semântica com LLMs

A capacidade de extrair e operacionalizar a intenção de um usuário a partir de descrições em linguagem natural tem se consolidado como o principal vetor de pesquisa e adoção dos LLMs. Desde as abordagens iniciais focadas estritamente na predição estática de próximos *tokens* até o desenvolvimento de modelos instrucionais ajustados por Aprendizado por Reforço com *Feedback* Humano (*Reinforcement Learning from Human Feedback* - RLHF) [Ouyang et al. 2022], a literatura recente documenta uma mudança de paradigma orientada à transição de fluxos textuais ambíguos para comandos estritamente estruturados e linguagens de execução. Esse cenário redefine os mecanismos de interação humano-computador, estabelecendo camadas intermediárias de tradução semântica que convertem a flexibilidade da linguagem orgânica em especificações técnicas determinísticas.

#### 4.1.1 Avanços na Literatura

No domínio da engenharia de *software*, a automação desse processo de tradução tem focado na síntese e completude de artefatos computacionais. Investigando este cenário, revisões sistemáticas demonstram que a aplicação de arquiteturas baseadas em *transformer* reduz a carga cognitiva dos desenvolvedores ao antecipar *tokens* funcionais e blocos de código inteiros com base no contexto do arquivo sob análise



[Husein, Aburajouh e Catal 2025]. Para mitigar a ambiguidade inerente aos requisitos iniciais, abordagens multiagente têm sido introduzidas para segmentar a geração de código (tarefas *NL2Code*) em etapas sequenciais e especializadas, englobando a otimização de demandas e a revisão pareada, o que eleva substancialmente as taxas de sucesso na primeira inferência (*pass@1*) [Bai et al. 2025]. Embora o encadeamento extensivo de inferências lineares corra o risco de acumular derivas semânticas e degradar o raciocínio preditivo, a decomposição estruturada de tarefas atua como um mecanismo de controle, preservando a consistência lógica e a conformidade do programa gerado frente às especificações do domínio.

Essa necessidade de ancorar a flexibilidade gerativa a restrições de domínios específicos expandiu-se para cenários industriais e analíticos que exigem alta precisão sintática e tolerância zero a inconsistências. Na área médica, o sistema *ImpressionGPT* exemplifica essa tendência ao empregar algoritmos de otimização iterativa e *prompts* dinâmicos para processar achados textuais livres e consolidar impressões clínicas altamente fidedignas [Ma et al. 2024]. De forma análoga, no campo de testes automatizados de interfaces móveis, modelos de linguagem são integrados ao contexto de elementos de interface gráfica (GUI) para deduzir e preencher campos de formulários a partir de intenções inferidas contextualmente [Liu et al. 2023]. Adicionalmente, no gerenciamento de infraestruturas cibernético-físicas, *pipelines* baseados em GPT são aplicados na predição de carga energética predial por meio do acoplamento de LLMs a bases de conhecimento externas e rotinas de autocorreção de código em tempo de execução [Zhang et al. 2025].

Em comum, tais soluções compartilham o princípio de que a inferência puramente estocástica de um LLM deve ser balizada por componentes exógenos de validação — sejam eles *prompts* dinâmicos, bases de conhecimento ou *loops* de retroalimentação baseados em interpretadores. Essa evolução pavimenta o caminho para metodologias ainda mais rigorosas, baseadas na Engenharia Dirigida por Modelos (MDE) [France e Rumpe 2007], as quais buscam substituir o refinamento reativo por restrições gramaticais aplicadas preventivamente no momento da decodificação [Wang et al. 2023].

### 4.1.2 Análise Crítica e Lacunas

Apesar do notável sucesso empírico documentado, os trabalhos fundamentados na geração de linguagem natural probabilística partilham uma limitação arquitetural: a dependência da inferência de intenção puramente estatística. Modelos neurais operam otimizando distribuições de probabilidade para a predição do próximo *token* ( $P(w_t | w_{1:t-1})$ ), maximizando a fluência sequencial, o que não garante a validade formal ou o respeito às regras de negócio subjacentes [Ji et al. 2023]. Sem âncoras determinísticas, o modelo frequentemente sofre de “desvio semântico” (*semantic drift*), gerando artefatos que são



superficialmente plausíveis, mas funcionalmente incorretos [Ji et al. 2023].

Nos fluxos de trabalho contemporâneos voltados à geração de código, nota-se uma alteração substancial no paradigma de desenvolvimento: o profissional passa a atuar predominantemente na verificação e correção do código estocástico gerado pela máquina, aumentando a carga de validação empírica [Bai et al. 2025, Husein, Aburajouh e Catal 2025]. Nessas abordagens, a conformidade do código é avaliada *a posteriori*, através de testes unitários ou interpretadores externos. Essa premissa de validação baseada em tentativa e erro iterativa apresenta riscos em sistemas críticos — como na robótica industrial de precisão ou na automação de processos industriais severos —, cenários onde as ações executadas interagem com o mundo físico, carecem de mecanismos de reversão rápida (*rollback*) e toleram zero margem para falhas operacionais. A limitação desse paradigma generativo puro reside na ausência de mecanismos de restrição *a priori*: o LLM atua sem um modelo conceitual rigoroso que circunscreva computacionalmente o espaço de comandos válidos antes da execução [Wang et al. 2023].

## 4.2 DSLs e a Engenharia Dirigida por Modelos com IA

Para prover o rigor ausente nas abordagens probabilísticas, a literatura de engenharia de *software* tem revisitado as premissas da Engenharia Dirigida por Modelos (MDE) [France e Rumpe 2007], integrando-as ao *design* de Linguagens de Domínio Específico (DSLs). Uma DSL fornece um modelo formal e regras de formação estrutural precisas (abstração sintática e semântica), definindo o vocabulário e o escopo exato que um sistema necessita para operar, atuando como um contrato formal entre as diretrizes de controle e a execução da máquina [Mernik, Heering e Sloane 2005].

### 4.2.1 Avanços na Literatura

A convergência entre MDE e IA generativa tem sido progressivamente estabelecida na literatura, destacando-se a formalização do *Model-Driven Prompt Engineering* proposta por [Clarísó e Cabot 2023]. Os autores argumentam que os *prompts* devem transitar de heurísticas empíricas livres para artefatos gerados sistematicamente a partir de metamodelos validados. A utilização de metamodelos garante que os comandos submetidos ao LLM contenham representações independentes de plataforma, reduzindo ambiguidades e estabelecendo rastreabilidade estrutural entre a intenção do usuário e a arquitetura do sistema [Clarísó e Cabot 2023].

No campo do processamento de linguagens específicas, observou-se a viabilidade de aplicar o *parsing* semântico neural para mapear comandos ambíguos em formas

lógicas rigorosas, mesmo em ambientes com escassez de dados [Sun et al. 2020]. Acompanhando essa evolução temporal, a literatura recente aprimorou a restrição gerativa com abordagens de otimização automática de instruções [Park e Kim 2026] e sintonia fina condicionada ao contexto (*prompt-tuning*) [Qin et al. 2025]. Tais métodos ajustam os pesos de atenção do modelo para maximizar a conformidade da saída frente a vocabulários técnicos complexos.

## 4.2.2 Análise Crítica e Lacunas

A integração de princípios de MDE na comunicação com modelos de linguagem introduz uma camada formal valiosa na interação [Clariso e Cabot 2023]. Contudo, as abordagens propostas nesta área concentram-se predominantemente no estágio de construção prévia da requisição (*design* estático do *prompt*). Elas não utilizam a gramática final como um mecanismo direto de restrição computacional e validação sintática durante o momento em que o modelo decodifica a resposta.

Nos trabalhos discutidos, observa-se a tendência de utilizar o LLM como um tradutor de texto livre para código em DSLs, operando de forma isolada do motor de processamento da linguagem. Ferramentas consolidadas para a criação de DSLs operam processando o código-fonte gerado para construir Árvores de Sintaxe Abstrata (AST)<sup>1</sup> [Hopcroft, Motwani e Ullman 2006]. Atualmente, a literatura evidencia uma desconexão sistêmica: o modelo neural gera a cadeia de texto sem validação em tempo real contra as regras gramaticais que compõem a AST final. Consequentemente, o modelo permanece suscetível a gerar alucinações de parâmetros estruturais não previstos pela gramática, ocasionando erros de compilação do lado do interpretador final. Para assegurar a correção sintática estrutural, estudos indicam que a própria gramática de destino deve restringir o espaço de decodificação (*decoding*) do LLM, permitindo apenas a seleção de palavras que formem constructos válidos [Wang et al. 2023].

## 4.3 Integração de Grafos de Conhecimento com LLMs

Para aprimorar o alinhamento das saídas gerativas aos fatos técnicos, métricas físicas e regras lógicas de um domínio específico, a literatura consolidou a adoção da técnica de Geração Aumentada por Recuperação (*Retrieval-Augmented Generation* - RAG) [Lewis et al. 2020]. Uma extensão metodológica fundamental desse conceito é o *GraphRAG*. Enquanto os bancos de dados vetoriais tradicionais do RAG realizam buscas baseadas na similaridade matemática de fragmentos textuais (frequentemente agrupando

---

<sup>1</sup>Estruturas de dados em forma de árvore que representam a estrutura sintática hierárquica do código-fonte de acordo com a gramática da linguagem estabelecida.

correlações semânticas espúrias), o *GraphRAG* processa a recuperação da informação por meio da navegação topológica em grafos de conhecimento [Bahr et al. 2025]. Nesses grafos, as entidades e os seus relacionamentos lógicos são formalmente delimitados em nós e arestas.

### 4.3.1 Avanços na Literatura

A integração de KGs em arquiteturas de recuperação de informações mitiga desvios dedutivos na modelagem de sistemas estruturados, superando as abordagens puramente baseadas em bancos de dados vetoriais em tarefas como a Análise de Modos de Falha e Efeitos (FMEA) [Bahr et al. 2025]. A adoção de grafos impõe direcionamento causal no contexto do *prompt*, ancorando as respostas do LLM às restrições normativas do domínio.

Expandindo esta capacidade estrutural, desenvolveu-se um arcabouço de raciocínio suportado por *GraphRAG* e pela técnica de indução *Chain-of-Thought* (CoT)<sup>2</sup>, projetado para gerar instruções detalhadas de manutenção mecânica, validando a coerência gerativa contra as relações hierárquicas dos equipamentos [Shi et al. 2025]. A utilização preditiva e interpretativa de grafos estende-se também a cenários multidisciplinares: na engenharia civil pesada, bases de conhecimento otimizam a fusão de parâmetros cruzados para o planejamento a longo prazo [Zhang et al. 2025]; na geração de interfaces, a topologia formal garante que as abstrações emocionais do usuário analisadas via Engenharia Kansei<sup>3</sup> sejam convertidas em especificações validadas de *design* visual [Feng et al. 2025]; por fim, o *Prompt4VFD* — um sistema analítico visual voltado ao diagnóstico automatizado de falhas em equipamentos elétricos — ilustra o uso de KGs acoplado ao raciocínio de análise para garantir diagnósticos precisos de anomalias em subestações de energia [Wen et al. 2025].

### 4.3.2 Análise Crítica e Lacunas

A literatura atual endossa a eficiência dos grafos de conhecimento na contenção de alucinações epistêmicas e na garantia de conformidade às regras intrínsecas de um domínio. Entretanto, observa-se uma limitação nos fluxos operacionais documentados por trabalhos dedicados a regras de domínio, como os de [Bahr et al. 2025] e [Shi et al. 2025]:

---

<sup>2</sup>A técnica de *Chain-of-Thought* induz o modelo a expor os seus passos lógicos intermediários antes de processar uma resposta final, promovendo um raciocínio mais estruturado na geração de texto longo [Wei et al. 2022].

<sup>3</sup>A Engenharia Kansei é uma abordagem metodológica que traduz sentimentos, impressões e demandas cognitivas subjetivas do usuário em especificações técnicas explícitas e métricas de *design* de produto.

após a correta extração das relações lógicas, a saída do sistema frequentemente é materializada em prosa ou parágrafos livres de documentação.

Esse comportamento evidencia uma desassociação entre a consistência semântica e a estruturação computacional. No mecanismo do *GraphRAG*, a injeção do subgrafo de contexto (a secção específica do grafo que se relaciona à consulta do usuário) fornece as restrições ao modelo, impedindo-o de tomar uma decisão logicamente inviável — como, por exemplo, sugerir um tratamento agroquímico que seja biologicamente incompatível com o atual estágio fenológico da plantação<sup>4</sup>. Contudo, essa injeção apenas restringe *o que* será respondido do ponto de vista do fato, falhando em impor o formato estrito de *como* essa informação deve ser computacionalmente serializada.

Como consequência, mesmo deliberando perfeitamente a lógica, o modelo encapsula a informação validada de modo textual, impedindo sua ingestão direta e segura por atuadores mecânicos, Controladores Lógicos Programáveis (CLPs) ou camadas de comunicação em sistemas cibernético-físicos. A literatura de automação de código investiga amplamente a conversão estrutural baseada em requisitos, contudo, nota-se a urgência de unificar conceitualmente ambas as abordagens. Faz-se necessário acoplar a garantia fática oriunda do grafo topológico com a restrição gramatical absoluta provida pelas ferramentas de MDE. A convergência entre o embasamento em dados formais de negócio (*GraphRAG*) e a limitação sintática do espaço de geração (*grammar prompting*) constitui a principal lacuna atacada por esta dissertação.

## 4.4 Grammar-Guided Prompting e Decodificação Restrita

Com o objetivo de mitigar as inconsistências estruturais inerentes à geração probabilística de texto e assegurar que a saída do modelo respeite estritamente as regras de formação da linguagem alvo, a literatura consolidou abordagens de Decodificação Restrita (*Constrained Decoding*) [Wang et al. 2023]. O foco central dessa abordagem é garantir a conformidade sintática computacional antes da execução de um comando gerado, destacando-se, nesse escopo, a técnica de *Grammar-Guided Prompting*.

### 4.4.1 Avanços na Literatura

O estudo de [Wang et al. 2023] estabeleceu os alicerces teóricos do *grammar prompting*. Esta técnica evolui além do paradigma *few-shot* — abordagem em que um

---

<sup>4</sup>Estágio fenológico refere-se à fase de desenvolvimento biológico do organismo (como maturação ou floração da planta), fator crítico que dita a viabilidade técnica da aplicação de insumos.

pequeno conjunto de pares de exemplos de entrada e saída é fornecido no *prompt* na expectativa de que a rede neural imite o padrão demonstrado. Em vez de depender da imitação heurística, o *grammar prompting* atua diretamente no motor de geração. Durante a etapa de decodificação, o sistema embute as regras sintáticas exatas de uma linguagem, formalizadas matematicamente em *Backus-Naur Form* (BNF), no avaliador probabilístico do LLM. Esse processo utiliza o *logits masking*, uma técnica que intervém dinamicamente no cálculo de probabilidades da rede, anulando a chance de seleção de *tokens* inválidos. Se a escolha de uma determinada palavra violar a árvore gramatical BNF em tempo de execução, a probabilidade de emissão desse caractere é forçada a zero. Consequentemente, o mecanismo restringe o espaço de busca do modelo, garantindo 100% de conformidade sintática nas saídas geradas.

O impacto deste controle estrutural é considerável na geração de código para sistemas de baixo nível e infraestruturas físicas. O estudo de [Yu et al. 2025] demonstrou, através da arquitetura *Spec2RTL-Agent*, que a conversão autônoma de especificações ambíguas para código de circuitos integrados (RTL) depende da orquestração de múltiplos agentes de IA especializados (com papéis distintos, como planejamento, codificação e validação) operando sob rigorosas restrições estruturais regidas por gramáticas. Essa mesma postura de contenção do espaço probabilístico é observada em estudos recentes sobre robustez e defesa contra ataques adversariais (*adversarial defense*) em tarefas de processamento de linguagem através de meta-prompts [Shi et al. 2025] e do refinamento guiado de instruções vetoriais [Li et al. 2025].

#### 4.4.2 Análise Crítica e Lacunas

A mecânica de *grammar prompting*, proposta por [Wang et al. 2023], provê uma camada de proteção eficiente para sistemas automatizados e interpretadores de código que consomem a saída do LLM, garantindo que o *script* derivado esteja livre de vírgulas ausentes, formatações incorretas ou infrações sintáticas que gerariam erros de compilação.

Contudo, a dependência isolada da decodificação restrita precisa de uma limitação analítica fundamental: a garantia sintática não assegura a viabilidade semântica ou a adequação prática de uma ação. Um comando estruturalmente perfeito pode, funcionalmente, representar um risco sistêmico ao obter uma decisão semanticamente incorreta. No contexto de um interpretador BNF puro desenhado para a aviação agrícola, um comando DSL como `Aplicar_Produto(Defensivo="Paraquat", Alvo="Talhao_1", Vento="25km/h")` pode ser processado sem qualquer erro morfológico. Semântica e operacionalmente, no entanto, é uma instrução inadequada que resultará em deriva química severa por ignorar fatores climáticos. A principal lacuna da aplicação exclusiva de gramáticas formais reside no fato de que elas estruturam rigorosamente o código, mas não

possuem mecanismos para validar as premissas de negócio e as regras físicas do domínio, evidenciando que a contenção sintática deve ser acoplada a componentes ontológicos para atender arquiteturas complexas.

## 4.5 Máquinas Virtuais de Domínio Específico (DSVM)

Na busca por materializar as premissas da Engenharia Dirigida por Modelos (MDE) em motores de execução autônomos, desenvolveram-se as Linguagens de Modelagem Específicas de Domínio Interpretadas (i-DSMLs). Assim como as linguagens interpretadas tradicionais processam instruções diretamente em tempo de execução, as i-DSMLs adotam a perspectiva de que os modelos de domínio podem ser consumidos e executados dinamicamente, dispensando a geração prévia de código-fonte em linguagens de propósito geral (como C++ ou Java). Para suportar o processamento direto dessa semântica, formulou-se o paradigma arquitetônico das Máquinas Virtuais de Domínio Específico (DSVMs), que atua como o alicerce central para a interpretação de comandos da qual esta dissertação deriva a sua base de execução [Allison et al. 2014].

### 4.5.1 Avanços na Literatura

A fundação estrutural desse paradigma foi detalhada e validada, por [Allison et al. 2014], através do gerenciamento inteligente de microrredes de energia (MGridVM). A pesquisa provou a viabilidade técnica de uma infraestrutura segmentada em quatro camadas: (1) Interface de Usuário, (2) Motor de Síntese (*Synthesis Engine* - SE), (3) Middleware de Controle e (4) *Broker* de Recursos [Allison et al. 2014]. O avanço arquitetural mais expressivo introduzido foi a modularidade estrita: a arquitetura do SE desvinculou o Modelo de Execução (MoE) abstrato e universal do Conhecimento Específico de Domínio (DSK). Essa separação permitiu que a máquina virtual se adaptasse a novos contextos simplesmente pela substituição do DSK — atuando como um módulo intercambiável de regras — sem exigir alterações no motor de processamento subjacente [Allison et al. 2014].

Fortalecendo as capacidades dinâmicas deste ecossistema, o desenvolvimento de middlewares adaptativos acoplados ao núcleo da Máquina Virtual de Comunicação (CVM) expandiu o potencial de roteamento da arquitetura [Morris et al. 2015]. Ao interagir com os comandos oriundos do Motor de Síntese, o middleware introduziu a capacidade de compor o DSK na forma de Modelos de Intenção (*Intent Models*). Operando em conjunto com Classificadores Específicos de Domínio (DSCs), a máquina de estados avalia e seleciona as ramificações operacionais ótimas para a resolução de uma tarefa, permitindo

que a plataforma ajuste a sua execução dinamicamente em resposta à disponibilidade de recursos e às condições do ambiente em tempo real [Morris et al. 2015].

### 4.5.2 Análise Crítica e Lacunas: A Ponte para o CML

A orquestração técnica estabelecida entre o Motor de Síntese — que extrai e tipifica intenções [Allison et al. 2014] — e o Middleware Adaptativo — que traça o caminho de execução mais eficiente [Morris et al. 2015] — comprovou ser possível dissociar a semântica de domínio da lógica de controle, permitindo a construção de ecossistemas de *software* altamente escaláveis.

Apesar da sólida sofisticação em modelagem, as implementações legadas de DSVM apresentam limitações operacionais vinculadas ao seu escopo original de *design*. Um dos desafios centrais dessas abordagens é a necessidade de especificação manual e rígida: a arquitetura exigia que os especialistas programassem ativamente o código das unidades de execução e estruturassem a entrada com base em técnicas formais, como a Diferenciação em Grafos e Árvores (*Graph Differencing*). Assumia-se, portanto, que a interação ocorreria exclusivamente através de operadores detentores de perícia técnica manipulando interfaces de modelagem visuais e predefinidas [Allison et al. 2014].

Como essas arquiteturas clássicas não foram projetadas para processar ambiguidades da linguagem orgânica humana, elas carecem de flexibilidade quando se exige que o sistema assimile requisições expressas em jargão natural por operadores não especializados. A arquitetura *Conceptual Model Learning* (CML), proposta no escopo deste trabalho, atua como uma evolução direta da camada de entrada e síntese das DSVMs para contornar essa lacuna. O CML acopla a inferência semântica de um LLM na porta de entrada do motor, substituindo a dependência estrita da diferenciação de grafos por traduções linguísticas orientadas por *prompt*. Adicionalmente, esta modernização adota um Grafo de Conhecimento integrado como substituto dinâmico para o antigo e estático DSK, utilizando as regras do *grammar prompting* como filtro restritivo. Essa fusão preserva a integridade de execução da DSVM original, mas confere-lhe a capacidade inédita de ingerir e validar intenções em linguagem livre de maneira autônoma.

## 4.6 Síntese e Posicionamento: O Diferencial da Arquitetura CML

A análise do estado da arte evidencia que as abordagens atuais para a automação dirigida por Modelos de Linguagem de Grande Escala (LLMs) tendem a tratar os desafios de geração de código e de raciocínio de forma compartimentada.



Trabalhos focados exclusivamente na geração livre de código mitigam a ambiguidade semântica na interpretação das requisições, mas permanecem suscetíveis a erros sintáticos no momento da compilação ou execução [Husein, Aburajouh e Catal 2025]. Por outro lado, a Engenharia Dirigida por Modelos [Clariso e Cabot 2023] e as Máquinas Virtuais de Domínio Específico (DSVMs) [Allison et al. 2014, Morris et al. 2015] estabelecem bases formais rigorosas de controle. Como tais arquiteturas foram concebidas priorizando a precisão de modelagem, a sua interface de entrada requer comandos estritamente estruturados. A evolução natural desse ecossistema consiste em acoplar a flexibilidade de interpretação de linguagem natural dos LLMs a essas infraestruturas formais preexistentes.

Adicionalmente, abordagens baseadas em *GraphRAG* [Bahr et al. 2025] fornecem a ancoragem factual necessária para evitar alucinações, mas frequentemente resultam na geração de respostas em linguagem natural livre, em vez de formatos estruturados diretamente consumíveis por interpretadores de máquina. Em contrapartida, técnicas de restrição na decodificação, como o *grammar prompting* [Wang et al. 2023], asseguram a validade sintática do código gerado, mas não possuem mecanismos intrínsecos para validar a coerência lógica e factual das ações propostas frente às restrições físicas e às regras de negócio de um domínio.

O presente trabalho visa preencher essa lacuna propondo a integração dessas diferentes abordagens em um fluxo de processamento unificado. A arquitetura *Conceptual Model Learning* (CML) propõe a ligação entre a ancoragem factual fornecida por grafos de conhecimento e a restrição sintática imposta pelo *grammar prompting*. Essa integração estabelece o que se define neste trabalho como Fechamento Semântico Mútuo: um mecanismo de validação cruzada no qual o modelo de linguagem é duplamente contido — restrito ontologicamente pelos fatos do domínio (via grafos) e contido sintaticamente pelas regras de formação de código (via gramática BNF). Dessa forma, assegura-se que apenas comandos simultaneamente válidos em formato e em lógica sejam enviados à máquina virtual de execução.

A Tabela 4.1 sintetiza o panorama bibliográfico, demonstrando graficamente a lacuna que este trabalho visa preencher e posicionando a proposta face aos estudos correlatos analisados.

Conforme sistematizado na Tabela 4.1, a contribuição principal deste trabalho não reside no aprimoramento isolado de modelos de linguagem convencionais, mas na concepção de uma arquitetura de integração. A proposta une a capacidade de abstração e processamento de linguagem livre dos LLMs à segurança estrutural exigida na execução de comandos de *hardware*. Ao modernizar a interface de entrada das DSVMs, a abordagem permite que operadores interajam com o sistema sem exigência de formalidade sintática rigorosa, enquanto os mecanismos internos da arquitetura garantem que essa in-



| Tabela 4.1: Comparativo entre a Literatura Correlata e a Metodologia CML Proposta |                                       |          |         |     |                                                                                                                                                 |
|-----------------------------------------------------------------------------------|---------------------------------------|----------|---------|-----|-------------------------------------------------------------------------------------------------------------------------------------------------|
| Referência                                                                        | Foco do Domínio / Aplicação           | MDE/ DSL | KG/ RAG | BNF | Lacuna / Limitação Central                                                                                                                      |
| Husein et al. (2025)                                                              | Revisão Sistemática (Code Completion) | Não      | Não     | Não | Foca em geração estocástica; alta dependência da capacidade inata do LLM.                                                                       |
| Allison et al. (2014)                                                             | Energia (MGridVM/DSVM)                | Sim      | Não     | Não | Propõe a arquitetura de síntese e separa o DSK, mas exige <i>inputs</i> estritamente estruturados (i-DSML).                                     |
| Morris et al. (2015)                                                              | Comunicação (CVM/DSVM)                | Sim      | Não     | Não | Middleware adaptável com validação formal estrita, mas sem capacidade de processar linguagem não estruturada.                                   |
| Bahr et al. (2025)                                                                | Análise FMEA (Falhas Industriais)     | Não      | Sim     | Não | Ancoragem factual em KGs, mas sem garantia formal sobre a formatação estrutural da saída.                                                       |
| Shi et al. (2025)                                                                 | Manuais Estruturados de Manutenção    | Não      | Sim     | Não | Integra CoT e Graph-RAG sem controle sintático restritivo (DSL/BNF) sobre o espaço de geração de <i>tokens</i> .                                |
| Wang et al. (2023)                                                                | Parsing Semântico (SMILES, GeoQuery)  | Sim      | Não     | Sim | Garante rigor de <i>parser/sintaxe</i> via <i>Grammar Prompting</i> , mas não valida premissas físicas ou operacionais.                         |
| Este Trabalho (CML)                                                               | Sistemas Críticos (Evolução da CVM)   | Sim      | Sim     | Sim | Integração de abstração e controle (Grafo + Gramática + LLM) conectando intenções informais a <i>scripts</i> validados sintática e logicamente. |

Fonte: Elaborado pelo autor (2026).

tenção seja traduzida, validada e executada de maneira determinística, pavimentando a transição para a proposta técnica explorada no próximo capítulo.

## 4.7 Conclusão do Capítulo

Este capítulo apresentou uma revisão crítica e comparativa dos trabalhos representativos que exploram a interseção entre a Inteligência Artificial Generativa e as práticas de Engenharia de *Software*. A investigação estruturou-se em cinco eixos fundamentais: a interpretação de intenções semânticas por LLMs; o emprego de Linguagens de Domínio Específico sob a ótica da Engenharia Dirigida por Modelos; a ancoragem factual de informações operacionais via *GraphRAG*; a restrição sintática de geração computacional por meio do *Grammar-Guided Prompting*; e o paradigma arquitetônico das Máquinas Virtuais de Domínio Específico (DSVM).

O exame dessas vertentes evidenciou que a literatura atual aborda os desafios de geração de código de maneira segmentada, em que as soluções voltadas a mitigar alucinações factuais e aquelas focadas em evitar quebras de compilação operam predominantemente sem coordenação. O posicionamento comparativo, consolidado na Tabela 4.1, permitiu delimitar as lacunas metodológicas e situar de forma objetiva a contribuição desta investigação frente às abordagens discutidas na literatura.

Uma vez mapeados os trabalhos correlatos e compreendidas as limitações das ferramentas quando aplicadas de modo isolado, estabelece-se o embasamento necessário para a adoção de uma arquitetura híbrida. Portanto, o Capítulo 5 destina-se à apresentação arquitetural da solução proposta, detalhando os seus componentes modulares, os fluxos de processamento de dados e os mecanismos de validação cruzada que viabilizam o Fechamento Semântico Mútuo — isto é, a restrição simultânea da geração por regras lógicas de domínio e por normas sintáticas — essencial para a operação segura em sistemas críticos.

---

## Proposta do modelo da arquitetura SPC-CML

---

O presente capítulo detalha a concepção, os componentes estruturais e o fluxo operacional do Compilador Semântico de Prompts (*Semantic Prompt Compiler* - SPC), desenvolvido como uma extensão e parte integrante da arquitetura *Conceptual Model Learning* (CML). Esta especificação, designada como subsistema SPC-CML, constitui o artefato metodológico central desta pesquisa. Conforme discutido na revisão da literatura, a automação de sistemas de missão crítica por meio de Modelos de Linguagem de Grande Escala (LLMs) enfrenta restrições severas devido à natureza estocástica e probabilística das respostas geradas pela inferência neural. Com o propósito de superar essa limitação, o SPC-CML introduz um modelo de fechamento semântico mútuo no escopo da plataforma CML, integrando a flexibilidade de interpretação da Inteligência Artificial Generativa ao rigor formal da Engenharia Dirigida por Modelos (MDE) e à consistência factual fornecida por Grafos de Conhecimento (KGs).

### 5.1 Visão Geral da Arquitetura

A arquitetura SPC-CML distingue-se das abordagens convencionais de engenharia de *prompts* baseadas estritamente na manipulação heurística de texto livre. A solução proposta atua como uma camada de *middleware* projetada para interceptar as intenções expressas pelo usuário em linguagem natural, submetendo-as a uma validação prévia frente às restrições físicas e regras operacionais do domínio antes de permitir a sua conversão final em comandos interpretáveis pelo sistema de destino.

O fluxo de processamento dessa arquitetura, representado na Figura 5.1, fundamenta-se na adoção de uma Linguagem Específico de Domínio (DSL) que opera como o contrato contratual e normativo do ecossistema. Essa especificação formal é decomposta e direcionada a dois motores complementares de verificação: as restrições relacionais e os fatos do domínio alimentam o grafo de conhecimento para garantir a conformidade lógica da deliberação, enquanto as regras gramaticais estruturam o motor de *grammar prompting*, assegurando de forma determinística a validade sintática do código gerado.

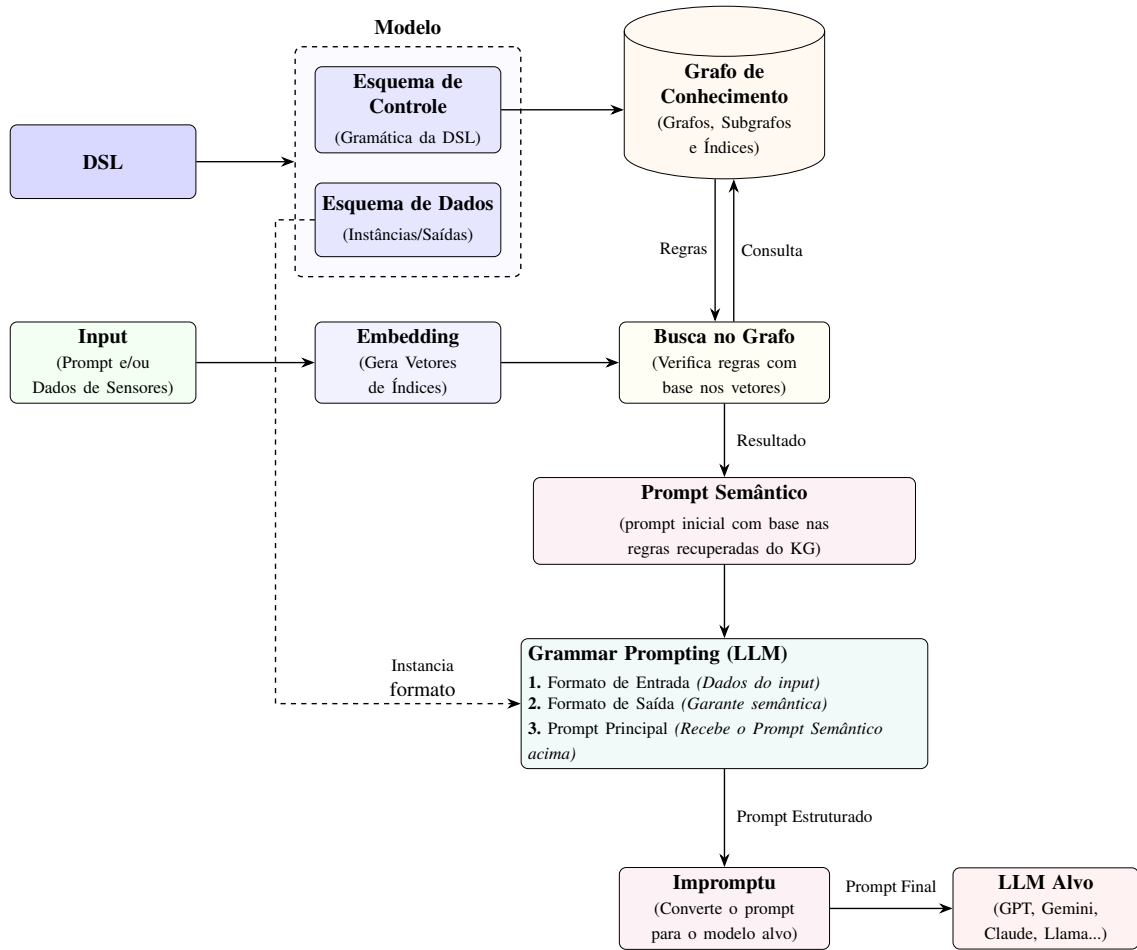


Figura 5.1: Arquitetura de Integração CML: Fluxo neuro-simbólico da DSL para o LLM Alvo, suportado por Grafos de Conhecimento e Decodificação Restrita.

## 5.2 Fundamentação

O núcleo estrutural da arquitetura proposta reside na formalização sistemática e *a priori* do conhecimento de domínio, mitigando de maneira objetiva a dependência exclusiva das capacidades probabilísticas inatas dos LLMs [Clarísó e Cabot 2023]. Em vez de tratar o modelo de linguagem autorregressivo como um componente autônomo de deliberação, a abordagem SPC-CML posiciona-o estritamente como um motor de tradução semântica. As saídas desse motor são delimitadas por restrições lógicas e gramaticais externas, fundamentando-se na premissa de que a confiabilidade em ambientes ciber-físicos não é um subproduto da escala da rede neural, mas sim um requisito assegurado por mecanismos determinísticos de controle.

Sob essa ótica, a arquitetura expande os preceitos da Engenharia de Prompts Dirigida por Modelos (*Model-Driven Prompt Engineering*) formalizados por [Clarísó e Cabot 2023]. A literatura sugere que a interação com modelos generativos deve

evoluir do *design* empírico (*ad-hoc prompting*) para a utilização de artefatos derivados de metamodelos validados. No escopo da arquitetura CML, essa abstração é instrumentalizada por meio de uma DSL — construída, por exemplo, sobre o *framework* Langium —, a qual estabelece um modelo independente de plataforma para governar a sintaxe permitida. Esse arcabouço previne que a ambiguidade inerente à linguagem natural do operador comprometa a previsibilidade sintática das instruções enviadas ao sistema de controle.

Complementarmente, a arquitetura adapta os princípios de isolamento de responsabilidades consolidados no paradigma das Máquinas Virtuais de Domínio Específico (DSVMs) [Allison et al. 2014]. Na concepção clássica de uma DSVM, o Motor de Execução abstrato é desvinculado do Conhecimento Específico de Domínio (DSK), garantindo a portabilidade e a integridade lógica do núcleo do sistema frente a alterações de contexto.

O *framework* SPC-CML reinterpreta essa separação ao segregar a capacidade universal de processamento de linguagem do LLM das regras estritas que regem a mecânica e a segurança do *hardware*. Para operacionalizar esse isolamento e atingir o Fechamento Semântico Mútuo, a DSL da arquitetura CML é particionada logicamente em dois esquemas, dividindo o domínio entre a representação teórica do ambiente e a parametrização das ações de atuação física.

### 5.2.1 O Modelo: Esquemas de Controle e Dados

Conforme ilustrado na Figura 5.1, a arquitetura categoriza a modelagem do domínio em duas frentes para assegurar o determinismo da resposta:

- **Esquema de Controle:** Define formalmente o domínio. Este esquema abriga a gramática em notação *Backus-Naur Form* (BNF), as definições ontológicas das entidades e as regras de negócio intrínsecas (os invariantes do sistema). Ele estabelece os componentes existentes no ambiente e delimita os relacionamentos operacionais permitidos entre eles.
- **Esquema de Dados:** Atua na gestão das **restrições de saída** e das **respostas permitidas** no processo de decodificação do LLM. Funciona como um modelo de validação empírica que especifica as instâncias finais e a tipagem exata dos dados (e.g., chaves, formatos de restrição, listas e parâmetros textuais) exigidos na resposta estruturada. Isso assegura que a ação gerada seja logicamente coerente com o domínio e, simultaneamente, formatada no padrão exato exigido pelo interpretador de destino.

### 5.2.2 Grafo de Conhecimento (GraphRAG)

As invariantes físicas, as restrições operacionais e as regras de negócio estabelecidas no Esquema de Controle são mapeadas de forma automatizada para um Grafo de Conhecimento, implementado sobre infraestruturas orientadas a grafos, como o Neo4j. Diferentemente dos repositórios vetoriais densos puros, o modelo relacional em grafos viabiliza a execução de consultas topológicas complexas e direcionadas [Bahr et al. 2025].

Os dados estruturais são persistidos na forma de nós — que representam entidades físicas ou conceituais, como culturas, substâncias químicas e sensores — e arestas direcionadas fortemente tipadas. Essas arestas explicitam dependências e exclusões mútuas, como a incompatibilidade físico-química entre dois compostos agrícolas, conforme detalhado na Seção 5.4 e ilustrado na Figura 5.4.

Esta modelagem permite a segregação do conhecimento em *clusters* ontológicos específicos. No cenário proposto na Seção 5.4, o grafo é segmentado em submódulos que isolam as particularidades de cada cultivo (Soja, Milho e Cana-de-Açúcar), sendo supervisionado por uma camada transversal de diretrizes globais de segurança aplicáveis ao *hardware* (e.g., a obrigatoriedade de descontaminação do tanque de pulverização).

Para recuperar essas diretrizes dinamicamente, a arquitetura SPC-CML emprega índices vetoriais extraídos via modelos de *embedding*. Quando o sistema processa a requisição do operador ou a telemetria dos sensores, o motor de busca realiza um mapeamento no espaço latente para identificar os nós e arestas correspondentes na ontologia. Essa similaridade matemática permite extrair o subgrafo de regras pertinente ao contexto imediato da atuação.

O bloco relacional resultante constitui a referência factual (*Ground Truth*) do ambiente. Esse insumo validado atua na prevenção de alucinações e serve de base para a etapa de *Grammar Prompting*, garantindo que a inteligência generativa opere sob o controle das restrições do domínio antes de emitir qualquer comando.

## 5.3 O Pipeline de Processamento de Intenções

O fluxo de processamento inicia-se com a recepção de uma entrada (*input*), que pode consistir na instrução em linguagem natural de um operador — independentemente de seu nível de especialização —, nos dados contínuos de telemetria dos sensores, ou na combinação de ambos. Para tratar essa requisição estocástica, o sistema SPC-CML opera em um ciclo contínuo estruturado em quatro estágios de refinamento e redução de incertezas:

1. **Embedding e Desambiguação Léxica:** A entrada em texto livre (e.g., “aplique onde for preciso”) é projetada em um espaço vetorial multidimensional. Este estágio

correlaciona o jargão informal às entidades formais da DSL, extraindo parâmetros operacionais sem depender de correspondências sintáticas exatas [Sun et al. 2020].

2. **Busca no Grafo e Formulação do Prompt Semântico:** Com base nos vetores gerados, o sistema consulta o Grafo de Conhecimento. Utilizando técnicas de *GraphRAG*, a arquitetura recupera os subgrafos com as restrições pertinentes à intenção extraída. Esse processo resulta no **Prompt Semântico**: uma estrutura que unifica o contexto da requisição do usuário às obrigações e proibições formais do domínio [Shi et al. 2025].
3. **Grammar Prompting (Decodificação Restrita):** O espaço de geração do modelo neural é ativamente contido. O *Prompt Semântico* é processado por um motor de *Grammar Prompting* que, alimentado pelo Esquema de Dados, força o LLM a aderir à metassintaxe da DSL. Qualquer *token* probabilístico cuja emissão viole as regras sintáticas predefinidas tem o seu valor mascarado matematicamente (*logits masking*) e é descartado pelo sistema [Wang et al. 2023].
4. **Módulo *Impromptu* (Extensão de Adaptação de Prompts):** Configurado como uma extensão prática de interoperabilidade — atuando externamente ao escopo principal de validação conceitual —, o módulo *Impromptu* processa o artefato gerado na etapa de *Grammar Prompting*. O componente funciona como um tradutor agnóstico que converte as instruções conceituais para a sintaxe nativa da inteligência artificial fundacional adotada (e.g., Gemini ou Claude), preservando propriedades como a modularidade e o encadeamento (*prompt chaining*) [Wang et al. 2023]. Por meio dessa etapa de especialização de código, o sistema ajusta automaticamente os operadores de relevância, os *prompts* negativos e os formatos de restrição de acordo com a documentação da API do modelo selecionado [Wang et al. 2023]. Esse desacoplamento estrutural garante que a arquitetura permaneça independente de provedores de IA, mitigando riscos de degradação oriundos de atualizações sistêmicas de terceiros [Wang et al. 2023].

## 5.4 Cenário

Para validar as definições arquiteturais propostas e demonstrar a aplicabilidade prática do modelo de Fechamento Semântico Mútuo, esta seção detalha a instanciização da arquitetura SPC-CML em um domínio de missão crítica: a automação de Veículos Aéreos Não Tripulados (VANTs) aplicados a rotinas de pulverização agrícola de precisão.

A operação de sistemas autônomos nesse ambiente exige elevados níveis de previsibilidade e conformidade regulatória. Desvios operacionais ou imprecisões na aplicação de defensivos químicos geram impactos severos, tais como a deriva aérea — que resulta no comprometimento de recursos hídricos e danos a cultivos adjacentes

—, a indução de fitotoxicidade<sup>1</sup> na cultura alvo e a violação de diretrizes ambientais. Alinhando-se aos princípios teóricos das Máquinas Virtuais de Domínio Específico (DSVM) [Allison et al. 2014], o Conhecimento Específico de Domínio (DSK) deve ser formalmente modelado e isolado do motor de execução. Esse isolamento assegura que o sistema opere estritamente sob premissas de segurança predefinidas, mitigando os riscos associados à ambiguidade ou variabilidade das instruções fornecidas pelo operador.

### 5.4.1 Modelagem do Conhecimento: A DSL Agrícola e Subgrafos Ontológicos

A etapa inicial do fluxo operacional do SPC-CML consiste na especificação da base de referência factual (*Ground Truth*) por especialistas do domínio técnico. Por meio da DSL estruturada no Esquema de Controle, são formalizadas as diretrizes normativas e operacionais que povoam o Grafo de Conhecimento (KG). Conforme ilustrado na Figura 5.2, essas regras são organizadas na forma de subgrafos de contexto, segmentados de acordo com as especificidades de cada cultivo, e supervisionadas por uma camada de restrições globais encarregada de regular o comportamento físico do dispositivo de atuação.

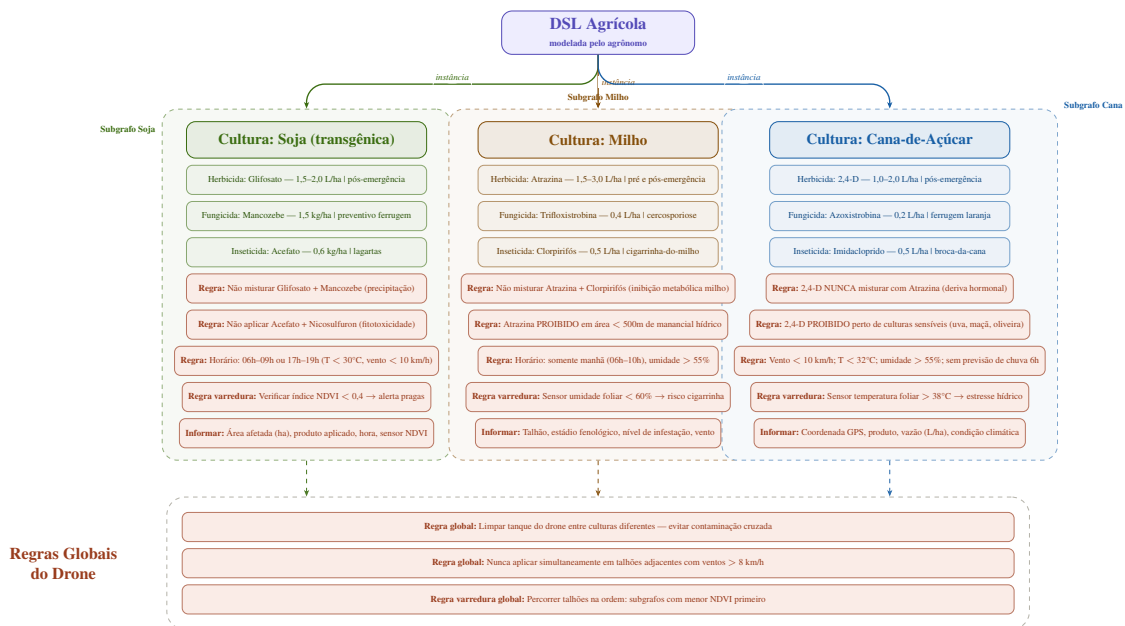


Figura 5.2: Topologia do Grafo de Conhecimento CML instanciado a partir da DSL Agrícola, evidenciando subgrafos por cultura e invariantes sistêmicas globais.

<sup>1</sup>Fitotoxicidade é o efeito tóxico ou prejudicial causado por uma substância química, agente biológico ou condição ambiental sobre uma planta, resultando em alterações fisiológicas, redução do crescimento ou danos aos tecidos vegetais.



A topologia do grafo, ilustrada na Figura 5.2, estabelece as restrições operacionais estritas do sistema:

- **Restrições Físico-Químicas (Nível de Nó):** Cada cultura possui parâmetros operacionais formalmente definidos. No cultivo da soja, restringe-se a mistura de Glifosato e Mancozebe devido ao risco de precipitação química. Na cana-de-açúcar, a aplicação do herbicida 2,4-D é bloqueada caso a temperatura ambiente ultrapasse 32°C ou existam culturas sensíveis em áreas adjacentes.
- **Regras de Monitoramento e IoT (Internet das Coisas):** O grafo correlaciona dados de telemetria a ações sistêmicas. A detecção de um Índice de Vegetação da Diferença Normalizada (NDVI) inferior a 0,4 na soja aciona um alerta de potencial infestação de pragas, enquanto uma temperatura foliar superior a 38°C na cana-de-açúcar é mapeada como indicador de estresse hídrico.
- **Regras Globais de Segurança (Nível de Aresta):** Consistem em invariantes topológicas que conectam diferentes subgrafos. Caso o plano de voo indique a transição do VANT de um talhão de soja para um de cana-de-açúcar, a ontologia impõe a **regra global de higienização do tanque** para prevenir contaminação cruzada. Além disso, a estrutura restringe aplicações simultâneas em talhões vizinhos se a velocidade do vento exceder 8 km/h.

A integração de grafos de conhecimento atua diretamente na mitigação da dificuldade inerente aos LLMs em manter coerência factual em contextos complexos [Bahr et al. 2025]. O modelo relacional atua como uma barreira preventiva, impedindo que a rede neural gere inferências não fundamentadas e delimitando o espaço de busca com base em diretrizes determinísticas e validadas.

### 5.4.2 O Fluxo de Execução em Tempo Real

A validação operacional da arquitetura SPC-CML evidencia-se no processamento e na conciliação entre o rigor formal do grafo e a variabilidade semântica inerente à linguagem natural humana no ambiente de campo. A Figura 5.3 ilustra o *pipeline* completo de resolução, mapeando as etapas desde a captação da instrução não estruturada do operador até a geração do comando de controle final interpretável pelos atuadores do VANT.

Conforme delineado na Figura 5.3, o ciclo operacional desenvolve-se por meio de estágios sequenciais de refinamento de intenção, estruturados sobre os preceitos do Middleware Adaptativo [Morris et al. 2015] e adaptados para viabilizar a conciliação entre fluxos de linguagem natural e restrições formais:

1. **Assimilação de Entrada (Ambiguidade e Telemetria):** O ecossistema processa inicialmente a instrução em linguagem natural e de teor coloquial fornecida por

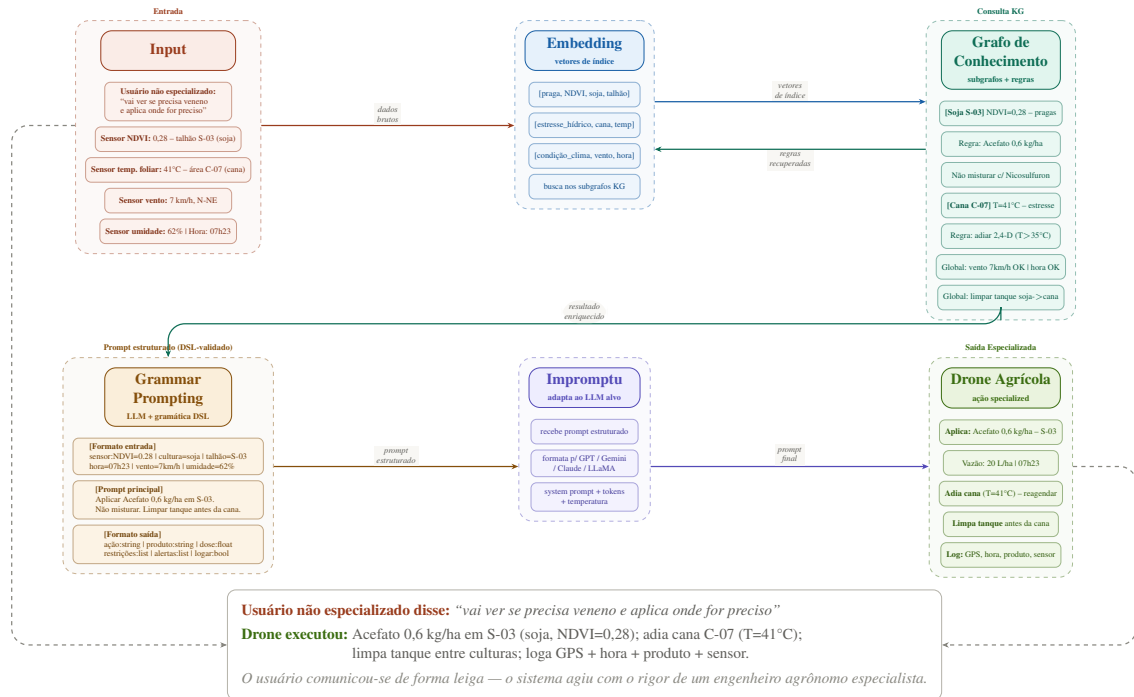


Figura 5.3: Fluxo de Execução CML: Da assimilação de telemetria e fala coloquial à execução especializada de *hardware*.

um operador em campo, tal como: “*Vai ver se precisa veneno e aplica onde for preciso*”. Simultaneamente, o sistema realiza a ingestão de dados de telemetria provenientes de dispositivos de Internet das Coisas (IoT), identificando variáveis contextuais imediatas, como o Índice de Vegetação da Diferença Normalizada (NDVI) em 0,28 no talhão de soja (S-03), a temperatura ambiente de 41°C no talhão de cana-de-açúcar (C-07) e a velocidade do vento a 7 km/h.

2. **Vetorização (*Embedding*):** O mecanismo de representação mapeia as expressões informais e ambíguas (e.g., “veneno”) em vetores numéricos densos no espaço latente. Esse processo extrai as chaves conceituais de busca — tais como praga, NDVI, estresse hídrico e velocidade do vento — necessárias para realizar a filtragem topológica na ontologia de referência.
3. **Consulta ao Grafo (*GraphRAG*):** Os índices vetoriais gerados são utilizados para consultar o Grafo de Conhecimento, permitindo que o subsistema de recuperação identifique as diretrizes estruturais correspondentes ao contexto de telemetria sob análise:

- Para o NDVI de 0,28 detectado no cultivo de soja, a regra ontológica recuperada prescreve a aplicação do insumo Acefato na dosagem de 0,6 kg/ha.
- Para a temperatura de 41°C na cana-de-açúcar, ativa-se a restrição de segurança térmica ( $T > 35^{\circ}\text{C}$ ), determinando a suspensão temporária da pulveri-

zação do composto 2,4-D.

- Em nível global, constata-se a viabilidade meteorológica quanto à velocidade do vento (7 km/h), porém impõe-se a obrigatoriedade de higienização do tanque de armazenamento na transição operacional entre cultivos distintos.

Essas informações integradas estruturam o **Prompt Semântico**, que unifica a requisição original do usuário ao contexto factual e às restrições normativas recuperadas do grafo.

4. **Decodificação Restrita (*Grammar Prompting*):** Para assegurar a conformidade sintática da resposta gerada, o espaço de busca do LLM é delimitado pelo Esquema de Dados da DSL. O motor de decodificação guiada por gramática aplica a técnica de mascaramento de *logits* (*logits masking*), anulando a probabilidade de emissão de caracteres inválidos e forçando o modelo a preencher exclusivamente as variáveis autorizadas pela sintaxe formal, tais como ação, produto, dosagem, restrições, alertas e registros de *log* [Wang et al. 2023].
5. **Adaptação e Especialização de Prompts (*Impromptu*):** Este estágio final recebe o artefato estruturado gerado pela decodificação restrita e emprega o módulo *Impromptu* para atuar como uma camada de tradução agnóstica de plataforma. Fundamentado em princípios de Engenharia Dirigida por Modelos (MDE), o componente converte as especificações conceituais da DSL para a sintaxe nativa e formato de API do modelo de linguagem de grande escala selecionado para execução no pipeline. Esse procedimento engloba o mapeamento de hiperparâmetros, a atribuição de pesos de relevância e a formatação de diretrizes negativas específicas do provedor tecnológico [Wang et al. 2023]. Esse desacoplamento preserva a independência lógica da arquitetura face a fornecedores externos, viabilizando migrações transparentes entre modelos e mitigando os riscos de degradação operacional decorrentes de atualizações nos sistemas de inteligência artificial.

### 5.4.3 Análise do Fechamento Semântico no Caso de Uso

A análise estrutural deste fluxo evidencia a mitigação das limitações inerentes aos modelos puramente generativos [Husein, Aburajouh e Catal 2025]. Conforme sintetizado na Figura 5.3, a arquitetura atua como uma interface de conversão que traduz instruções informais de usuários não especializados em comandos validados por diretrizes agrônômicas técnicas.

Sob a perspectiva da Engenharia de *Software*, a variabilidade estocástica da linguagem natural humana é mapeada para o determinismo operacional da máquina. O espaço de geração do LLM é delimitado, impedindo a formulação de planos de voo arbitrários ou a alteração não prevista dos formatos de saída. O modelo atua unicamente

como um mecanismo de tradução entre a requisição do operador e as regras formais do Grafo de Conhecimento, enquanto o Esquema de Dados garante que as respostas geradas restrijam-se ao conjunto de instâncias permitidas. Este ecossistema atende ao propósito fundamental estabelecido pela literatura de Engenharia Dirigida por Modelos (MDE) integrada a Máquinas Virtuais de Domínio [Clarísó e Cabot 2023, Allison et al. 2014]: assegurar que o modelo conceitual de domínio atue permanentemente como o elemento regulador central sobre as ações cibernético-físicas do sistema.

## 5.5 Status de Desenvolvimento e Implementação Atual

Com o objetivo de fundamentar a validação empírica desta pesquisa, esta seção apresenta o estágio atual de maturidade técnica da arquitetura SPC-CML. O desenvolvimento foi estruturado em fases, dividindo-se entre os componentes funcionais de modelagem e extração — que já se encontram consolidados — e as camadas de restrição lógica terminal, cuja codificação e validação compõem a etapa final deste projeto de mestrado.

### 5.5.1 Componentes e Artefatos Concluídos

Até o presente momento, a infraestrutura base da arquitetura SPC-CML encontra-se implementada e parcialmente funcional, operando de forma integrada nas etapas que antecedem a aplicação do *grammar prompting*. O *pipeline* inicial compreende os seguintes marcos de engenharia concluídos:

1. **Projeto e Compilação da DSL:** A sintaxe abstrata e concreta da linguagem específico de domínio foi integralmente desenvolvida utilizando o *framework* Langium. Foram validados os limites operacionais e as tipagens lógicas para o ecossistema do VANT agrícola (*AgroDrone*), estabelecendo a separação estrita entre o Esquema de Controle e o Esquema de Dados, conforme detalhado na documentação gramatical constante no Apêndice B (Seção B.5).
2. **Maapeamento Automático para o Banco em Grafos:** Desenvolveu-se um compilador interno responsável por processar as instâncias geradas pelo especialista do domínio (conforme modelo detalhado no Apêndice B, Seção B.6) e convertê-las dinamicamente em uma topologia relacional. Essa estrutura é exportada e persistida no banco de dados orientado a grafos Neo4j. A Figura 5.4 detalha a instanciação das propriedades dos nós de cultivo na interface gráfica do banco, enquanto a Figura 5.5 ilustra a conexão topológica global entre os talhões de atuação e as respectivas culturas.

A modelagem assegura que cada entidade cultural possua as suas próprias diretrizes e atributos parametrizados. Complementarmente, as propriedades físico-químicas

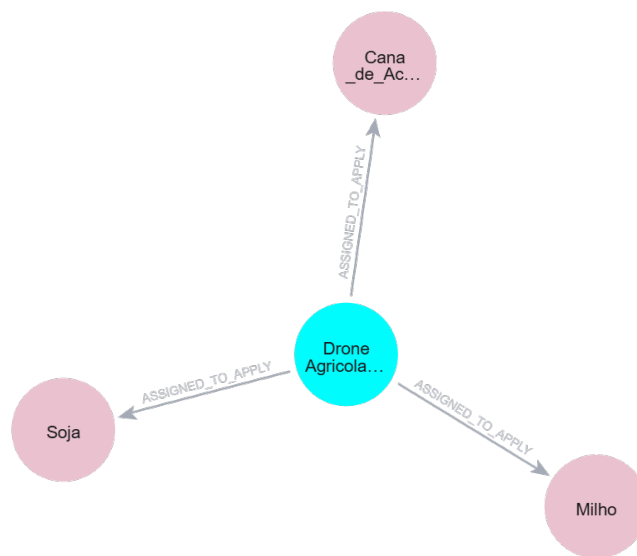


Figura 5.4: Instanciação gráfica e propriedades dos nós de cultivo no banco de dados Neo4j.

das substâncias (como herbicidas e inseticidas), definidas via DSL, são traduzidas em atributos associados a cada nó químico. O comportamento estrutural e as regras de restrição de uso dessas entidades no grafo podem ser examinados nas instâncias documentadas de forma isolada pela Figura 5.6 e correlacionadas na Figura 5.7.

3. **Mecanismo de Vetorização e Busca por Similaridade:** A etapa de processamento de entrada encontra-se operacional. O sistema assimila dados de sensores de telemetria integrados a instruções naturais do operador e, mediante modelos de *embedding*, converte a semântica da requisição em vetores numéricos. Esses vetores são comparados às propriedades do Neo4j para recuperar os subgrafos de restrições adequados ao contexto. A Figura 5.8 demonstra as invariantes operacionais isoladas, enquanto a Figura 5.9 evidencia a integração global dessas regras após o processo de desambiguação léxica.

Para fins de detalhamento técnico, a Figura 5.9 expõe a interface completa da consulta: o painel esquerdo apresenta o uso da linguagem *Cypher* (padrão estruturado de consultas para grafos) e o painel direito exibe os detalhes do nó selecionado, com destaque para a visualização explícita do vetor de índice (*embedding*).

A consolidação cruzada entre a telemetria em tempo real, as solicitações do usuário e o conjunto de proibições factuais do Neo4j encontra-se operante na camada de interface, conforme ilustrado no painel de monitoramento da Figura 5.10. O

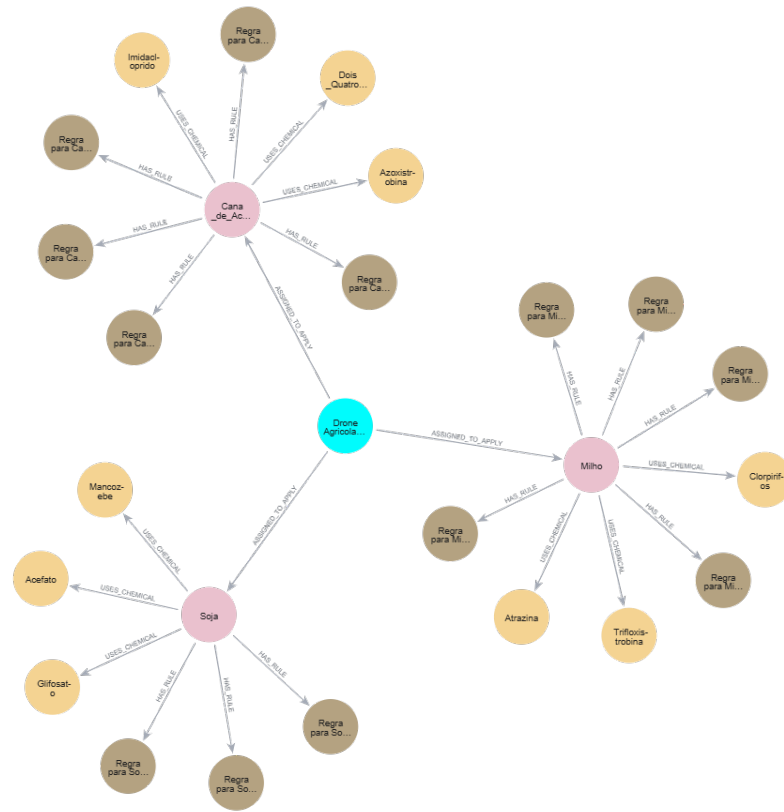


Figura 5.5: Mapeamento de relacionamentos topológicos entre talhões e culturas agrícolas no Neo4j.

sistema estrutura esse bloco de conhecimento factual, tornando-o disponível para ser consumido pela subsequente etapa de inferência.

### 5.5.2 Análise do Log de Execução e Consolidação Contextual

Para validar empiricamente o comportamento dinâmico do *pipeline* antes da fase de restrição sintática final, registrou-se a saída (*log*) gerada pela infraestrutura SPC-CML durante o processamento de uma requisição. Este ensaio comprova a integração entre a camada de sensoriamento (IoT), a desambiguação semântica da intenção humana e a recuperação topológica de diretrizes no Neo4j. A Figura 5.11 ilustra o terminal do sistema, evidenciando o fluxo de enriquecimento dos dados até a geração de um artefato contextual unificado.

O exame do registro exposto na Figura 5.11 reflete o encadeamento das camadas de entrada da arquitetura. O processamento inicia-se com a fusão das expressões do operador e dos parâmetros telemétricos (e.g., velocidade do vento, temperatura e NDVI). Subsequentemente, o módulo de *embedding* atua na conversão dos conceitos da linguagem natural em chaves taxonômicas ancoradas na DSL.

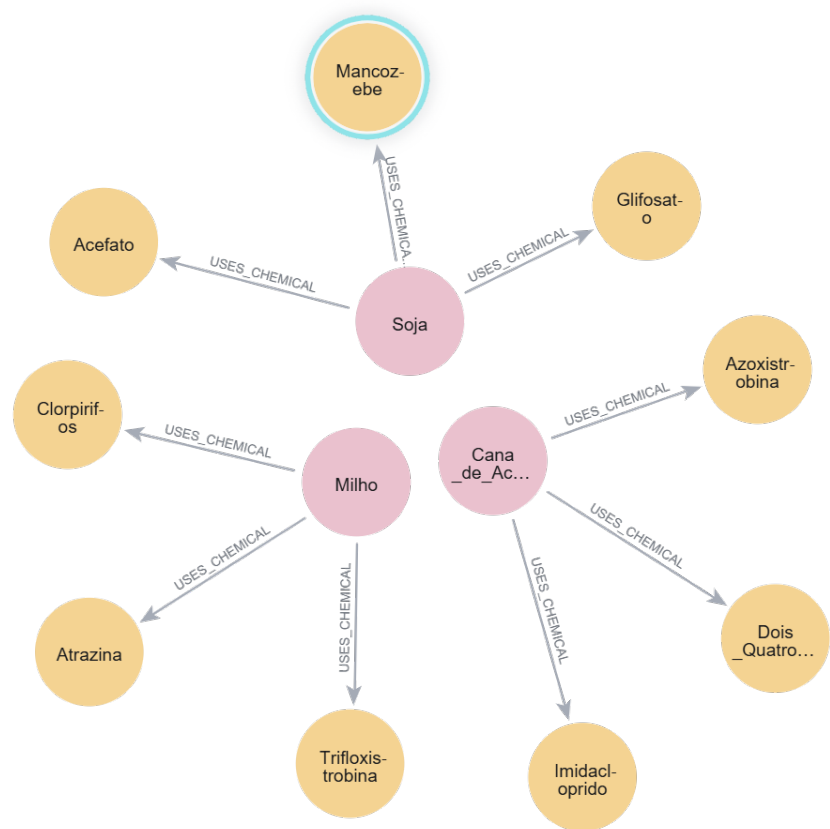


Figura 5.6: Nós de substâncias químicas e parametrização de dosagens limite no grafo.

|    |                |          |   |                              |
|----|----------------|----------|---|------------------------------|
| 1  | Milho          | HAS_RULE | → | Regra para Milho: Não mi...  |
| 2  | Milho          | HAS_RULE | → | Regra para Milho: Proibid... |
| 3  | Milho          | HAS_RULE | → | Regra para Milho: Condiç...  |
| 4  | Milho          | HAS_RULE | → | Regra para Milho: Se Sen...  |
| 5  | Milho          | HAS_RULE | → | Regra para Milho:            |
| 6  | Cana_de_Acucar | HAS_RULE | → | Regra para Cana_de_Acu...    |
| 7  | Cana_de_Acucar | HAS_RULE | → | Regra para Cana_de_Acu...    |
| 8  | Cana_de_Acucar | HAS_RULE | → | Regra para Cana_de_Acu...    |
| 9  | Cana_de_Acucar | HAS_RULE | → | Regra para Cana_de_Acu...    |
| 10 | Cana_de_Acucar | HAS_RULE | → | Regra para Cana_de_Acu...    |
| 11 | Soja           | HAS_RULE | → | Regra para Soja: Não mis...  |
| 12 | Soja           | HAS_RULE | → | Regra para Soja: Não mis...  |
| 13 | Soja           | HAS_RULE | → | Regra para Soja: Condiçõ...  |
| 14 | Soja           | HAS_RULE | → | Regra para Soja: Se Indic... |
| 15 | Soja           | HAS_RULE | → | Regra para Soja:             |

Figura 5.7: Grafo de restrições relacionando a aplicação de defensivos às respectivas culturas-alvo.

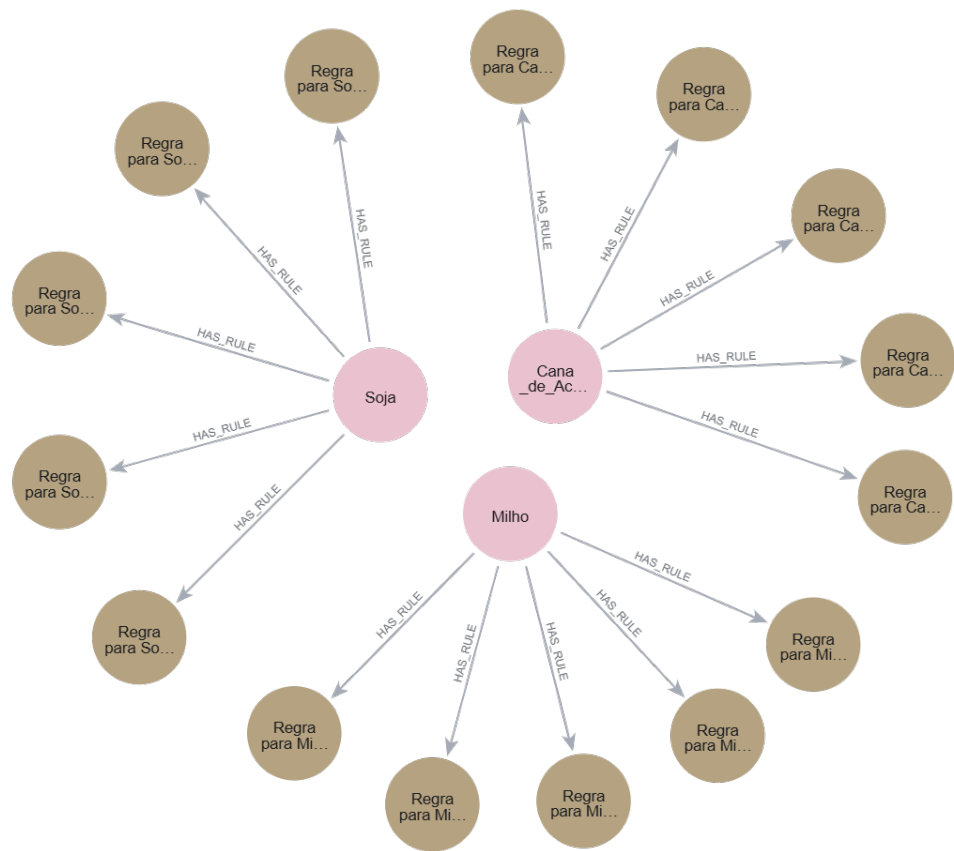


Figura 5.8: Visualização das regras de transição de estado e restrições de segurança injetadas no grafo.

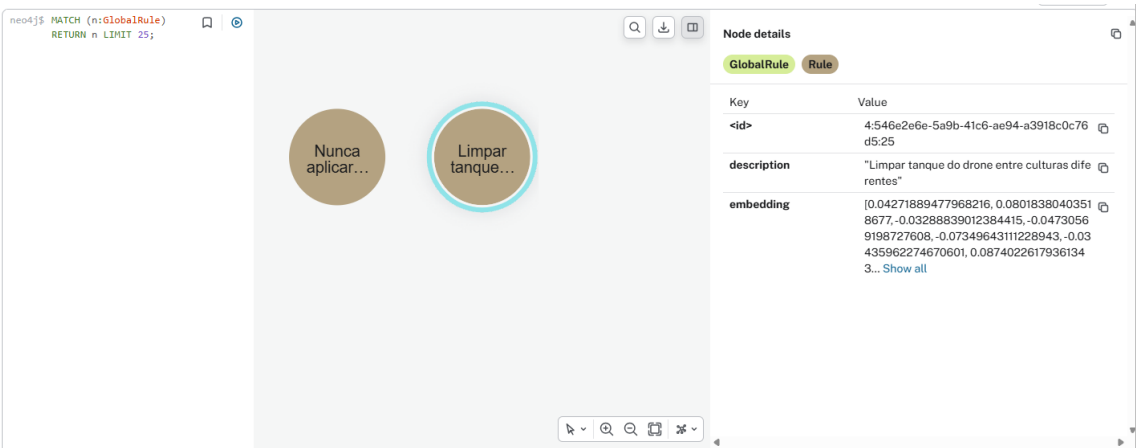


Figura 5.9: Estruturação de regras globais e subgrafos de segurança ativados via vetores de índice.



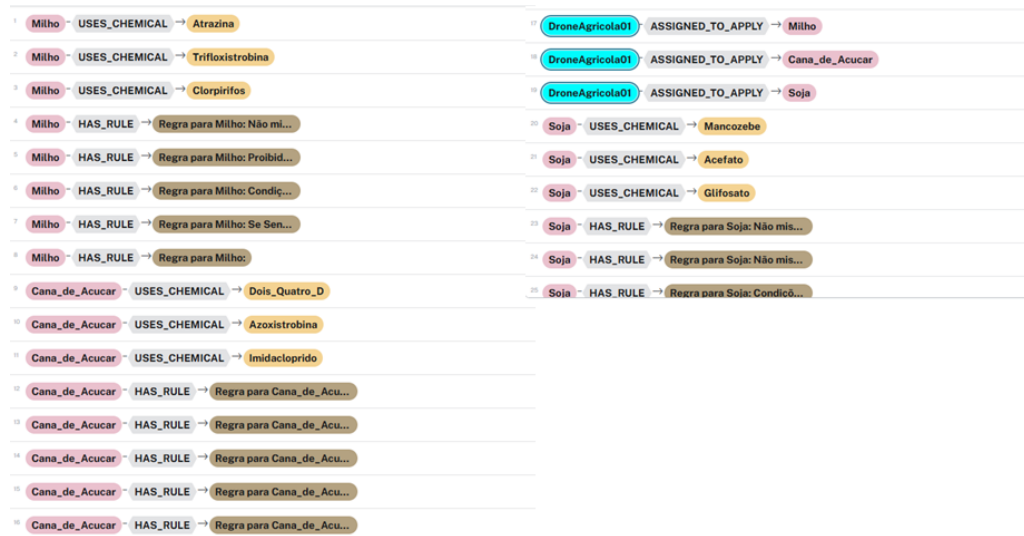


Figura 5.10: Painel de controle integrado evidenciando a correlação de regras recuperadas e prontas para a etapa de prompt.

```

PROBLEMS  OUTPUT  AZURE  TERMINAL  DEBUG CONSOLE  PORTS  GITLENS
carri@BOOK-1C24C8NDB0 MINGW64 ~/Workspace/dsl-msc (main)
$ npx tsx src/database/semantic-search.ts
🚀 RECEBENDO INPUT DO DRONE E DO USUÁRIO NÃO ESPECIALIZADO...
🌸 Enviando contexto para o LLM (Tomada de Decisão)...

***** PROMPT GERADO *****

Você é o sistema autônomo de um Drone Agrícola.
Sua missão é avaliar os dados dos sensores e decidir se o drone pode executar a ordem do usuário, baseando-se EXCLUSIVAMENTE
nas Regras Extraídas do Grafo de Conhecimento.

[DADOS DOS SENSORES]
Cultura: Cana_de_Acucar
Temperatura: 41.5°C
Vento: 5 km/h
Umidade: 48%
Pedido do operador: "Aplique veneno na cultura"

[REGRAS EXTRAÍDAS DO GRAFO PARA ESSA CULTURA]
- Nunca aplicar simultaneamente em talhões adjacentes com ventos > 8 km/h se culturas diferentes
- Regra para Cana_de_Acucar: Condições climáticas exigidas: Vento < 10 km/h; T < 32°C; umidade > 55%; sem previsão de chuva 8h

[FORMATO DE SAÍDA EXIGIDO - JSON]
Analise os dados. Retorne um JSON com:
"status": "PERMITIDO" ou "RECUSADO"
"motivo": Explicar o cruzamento lógico das regras com os sensores.
"acao_drone": O que o drone deve fazer agora (ex: retornar a base, aplicar produto X).

*****

✅ DECISÃO FINAL DO SISTEMA (Retorno do LLM):
{
  "status": "RECUSADO",
  "motivo": "A temperatura atual é 41.5°C, o que viola a Regra Climática da Cana que exige T < 32°C. Além disso, o sensor foliar indica e
  stresse hídrico. A aplicação nestas condições causaria fitotoxicidade grave ou perda do produto por evaporação rápida.",
  "acao_drone": "Abortar missão. Registrar log de estresse hídrico no sistema e retornar para a base."
}

carri@BOOK-1C24C8NDB0 MINGW64 ~/Workspace/dsl-msc (main)
$

```

Figura 5.11: Log de execução do terminal demonstrando a captura de inputs, indexação vetorial e extração de regras no Neo4j.

Através das representações vetoriais, o motor de busca realiza a varredura baseada em *GraphRAG*. O sistema extrai o subgrafo de contexto e identifica as premissas operacionais críticas — incluindo limites térmicos, zonas de exclusão e incompatibilidades químicas.

Cabe ressaltar uma delimitação pontual no estágio de implementação documentado pela Figura 5.11: o *log* demonstra a geração bem-sucedida do **Prompt Semântico**, consolidando a requisição do usuário e a fundamentação agrônômica. Contudo, o motor de *Grammar Prompting* baseado em BNF ainda não atua sobre essa composição. A inserção desta filtragem probabilística terminal, que visa garantir a aderência estrita à metassintaxe e coibir desvios estruturais na saída do LLM [Wang et al. 2023], configura-se como a meta primária de desenvolvimento das próximas fases da pesquisa, possibilitando a tradução desse compêndio textual em *scripts* de *hardware* computacionalmente válidos.

### 5.5.3 Componentes Pendentes e Próximos Passos de Engenharia

Apesar da infraestrutura funcional para acumulação de contexto e recuperação factual de dados, dois componentes da arquitetura SPC-CML encontram-se programados para implementação durante o período de finalização da pesquisa:

- **Implementação do Módulo de Grammar Prompting:** Este componente representa o núcleo técnico da validação da dissertação. Uma vez que o refinamento contextual e as restrições lógicas do grafo já estão devidamente parametrizados na etapa anterior, a pesquisa concentrar-se-á na codificação do motor de decodificação restrita. O objetivo consiste em integrar a gramática BNF nativa da DSL ao fluxo de inferência do modelo de linguagem. Isso exigirá a aplicação técnica do mascaramento probabilístico de *logits* em tempo de execução, assegurando a conformidade sintática da resposta gerada aos moldes preconizados por [Wang et al. 2023].
- **Acoplamento Adaptativo do Módulo Impromptu:** A camada secundária denominada *Impromptu* — responsável por formatar a instrução final validada de acordo com as especificações particulares de provedores de IA ou atuadores cibernéticos — figura como um elemento arquitetural complementar. Este módulo atua como uma camada de interoperabilidade de *software* e não constitui o escopo primário de validação lógica da arquitetura SPC-CML. Dessa forma, caso o cronograma de desenvolvimento permita a sua integração, ele ampliará a flexibilidade técnica da solução; do contrário, a prova de conceito do mestrado será considerada satisfatória mediante a entrega da etapa estrutural validada pelo *Grammar Prompting*.

## 5.6 Conclusão do Capítulo

Este capítulo detalhou a especificação formal, os componentes estruturais e o fluxo operacional da arquitetura SPC-CML, apresentando-a como uma camada intermediária (*middleware*) projetada para viabilizar o Fechamento Semântico Mútuo em sistemas de missão crítica. A partir da estruturação da Linguagem Específico de Domínio (DSL) em esquemas de controle e de dados, demonstrou-se como o modelo integra a flexibilidade interpretativa dos LLMs à ancoragem factual do *GraphRAG* e à restrição sintática do *grammar prompting*. A instanciação do arcabouço no cenário de automação de VANTs agrícolas ilustrou a viabilidade da solução, mapeando a conversão de instruções informais e ambíguas em comandos computacionais determinísticos, estritamente alinhados às restrições operacionais e regulatórias do domínio.

Com a arquitetura formalizada e o seu comportamento dinâmico delineado por meio do cenário de aplicação, estabelecem-se as bases conceituais e tecnológicas que sustentam a proposta desta pesquisa. Por conseguinte, o Capítulo 6 apresenta as considerações finais do documento. Nele, serão sintetizadas as contribuições metodológicas alcançadas até a presente etapa de qualificação, discutidas as delimitações do modelo atual e estruturados os próximos passos exigidos para a conclusão do desenvolvimento, integração dos módulos pendentes e validação empírica final da arquitetura SPC-CML.

## Conclusão e Próximos Passos

Este documento de qualificação consolidou a fundamentação teórica e a especificação arquitetural do modelo **SPC-CML**. Esta infraestrutura híbrida foi desenvolvida com o objetivo de conciliar a expressividade da linguagem natural com o rigor determinístico exigido por sistemas cibernético-físicos de missão crítica. Ao integrar a Engenharia Dirigida por Modelos (MDE) às capacidades dos Modelos de Linguagem de Grande Escala (LLMs), o trabalho propõe uma abordagem estruturada para mitigar a variabilidade inerente à Inteligência Artificial Generativa em ambientes operacionais complexos.

### 6.1 Síntese das Contribuições Técnicas

O desenvolvimento desta pesquisa evidenciou que a aplicação isolada de modelos generativos em processos de controle apresenta limitações operacionais significativas, sobretudo devido à ocorrência de alucinações sintáticas e semânticas [Husein, Aburajouh e Catal 2025]. O modelo proposto fundamenta-se no princípio do **Fechamento Semântico Mútuo**, estabelecendo que a utilidade e a segurança do LLM em tarefas críticas dependem da sua atuação como um tradutor semântico, rigorosamente contido por uma estrutura formal externa.

Nesse contexto, a arquitetura SPC-CML contribui para a literatura ao implementar três camadas interdependentes de controle estrutural:

1. **Ancoragem Factual via GraphRAG:** Distinguindo-se de abordagens de recuperação baseadas estritamente em vetores de similaridade, a utilização de Grafos de Conhecimento (como o Neo4j) viabiliza a extração de dados, relacionamentos e restrições transversais. Essa estrutura assegura que a intenção do usuário seja previamente filtrada pelas regras operacionais definidas por especialistas do domínio [Bahr et al. 2025].
2. **Restrição Sintática via Decodificação Guiada:** A integração de gramáticas formais (BNF) ao processo de inferência atua sobre a validade estrutural da saída. O emprego do mascaramento probabilístico de *logits* restringe o espaço de geração

do modelo, garantindo que o *script* resultante seja sintaticamente compilável pelo sistema de destino [Wang et al. 2023].

3. **Determinismo por Separação de Esquemas:** Inspirada nos princípios das Máquinas Virtuais de Domínio Específico (DSVMs) [Allison et al. 2014], a arquitetura dissocia o **Esquema de Controle** (que define a ontologia do sistema) do **Esquema de Dados** (que delimita as restrições de saída). Essa divisão blinda a lógica de execução do *hardware* frente à variabilidade das entradas em linguagem natural.

## 6.2 Objetivos Alcançados e Maturidade da Pesquisa

Nesta etapa de qualificação, a pesquisa atingiu a maturidade conceitual e estrutural necessária para fundamentar a fase de implementação e validação empírica. Os seguintes marcos foram estabelecidos:

- **Formalização Sistêmica da Arquitetura SPC-CML:** O delineamento dos módulos, interfaces de comunicação e fluxos de dados do *pipeline* foi concluído, assegurando que o sistema opere de maneira independente do provedor do modelo de linguagem (*LLM-agnostic*).
- **Modelagem de Domínio Crítico (AgroDrone):** A estruturação do estudo de caso permitiu validar a representação metassintática da DSL por meio do *framework* Langium, englobando variáveis de pulverização e restrições agronômicas aplicáveis a diferentes culturas (soja, milho e cana-de-açúcar).
- **Sistematização do Referencial Teórico:** A revisão da literatura articulou os domínios de MDE, processamento de linguagem natural e sistemas baseados em conhecimento, identificando a lacuna metodológica que a arquitetura CML visa preencher [Clarísó e Cabot 2023].

## 6.3 Resultados Esperados e Desafios de Implementação

A transição metodológica para a redação da dissertação concentrar-se-á na validação empírica da proposta. O objetivo principal é demonstrar que a arquitetura SPC-CML é capaz de garantir conformidade sintática absoluta na geração de código [Wang et al. 2023], bem como proporcionar melhorias substanciais na preservação da coerência semântica frente a comandos ambíguos, quando comparada a abordagens tradicionais de *prompt engineering*.

Para o último semestre de pesquisa, as prioridades de engenharia e implementação compreendem:

- **Módulo *AST-to-Graph Mapper*:** O desenvolvimento de um mecanismo automatizado para sincronizar a Árvore de Sintaxe Abstrata (gerada via Langium) com a inserção dinâmica de nós no Neo4j, garantindo a correspondência contínua entre o modelo de domínio e a topologia do grafo.
- **Otimização do *Semantic Routing Engine*:** O aprimoramento dos modelos de *embedding* com o intuito de maximizar a precisão na desambiguação léxica de termos informais e jargões técnicos, viabilizando a ativação correta dos subgrafos de restrição.
- **Recuperação Baseada em *Grammar Prompting*:** A implementação do sistema de composição dinâmica de *prompts*. Em alinhamento às técnicas documentadas na literatura [Wang et al. 2023], o desafio envolve coordenar a extração de parâmetros, regras gramaticais e contextos do grafo para a construção de instruções modulares, assegurando a precisão semântica prévia ao processamento do modelo neural.
- **Extensão Adaptativa *Impromptu*:** A codificação da camada de interoperabilidade responsável por converter o artefato estruturado gerado em protocolos de rádio-controle e rotinas compatíveis com os atuadores físicos de VANTs industriais.

## 6.4 Cronograma para Defesa de Dissertação

O planejamento para a conclusão do projeto, incluindo as etapas de desenvolvimento técnico e a redação do documento final, está sumarizado na Tabela 6.1, referente ao segundo semestre acadêmico de 2026.

Tabela 6.1: Cronograma de atividades para o semestre final (2026-2)

| ID | Atividade                                | Mês 1 | Mês 2 | Mês 3 | Mês 4 | Mês 5 | Mês 6 |
|----|------------------------------------------|-------|-------|-------|-------|-------|-------|
| 1  | Codificação do Mapper AST-Grafo          | X     | X     |       |       |       |       |
| 2  | Automação do Grammar Builder (BNF)       |       | X     | X     |       |       |       |
| 3  | Integração via LangChain e Fast API      |       |       | X     | X     |       |       |
| 4  | Coleta de Métricas e Testes de Validação |       |       |       | X     | X     |       |
| 5  | Redação da Dissertação Final             |       |       |       | X     | X     | X     |
| 6  | Defesa Pública de Mestrado               |       |       |       |       |       | X     |

Em suma, este trabalho propõe uma transição metodológica para a automação baseada em modelos de linguagem: em vez de depender do alinhamento comportamental probabilístico do LLM para a prevenção de falhas, a segurança e a consistência operacionais passam a ser garantidas formalmente por restrições arquiteturais. Essa abordagem visa converter a inteligência generativa em um componente de engenharia determinístico, apto a integrar ecossistemas de elevada criticidade.

## 6.5 Conclusão do Capítulo

Este capítulo finaliza o documento de qualificação ao sistematizar as contribuições da arquitetura SPC-CML, detalhar o estágio atual de maturidade da pesquisa e estabelecer as diretrizes operacionais para a etapa conclusiva do mestrado. O texto consolidou a função das três camadas de validação propostas — ancoragem factual, restrição sintática e isolamento de esquemas —, além de mapear as demandas técnicas remanescentes e o cronograma planejado para a defesa da dissertação.

Ao submeter e validar as bases teóricas e a estruturação lógica do modelo neste exame de qualificação, a pesquisa conclui a sua fundamentação conceitual. As etapas subsequentes estarão voltadas à implementação em *software* dos módulos descritos, à condução de testes de estresse em ambientes simulados e à elaboração do manuscrito final. Dessa maneira, espera-se comprovar a viabilidade de acoplar a adaptabilidade dos modelos de linguagem a uma infraestrutura de controle computacionalmente segura e determinística.

---

## Referências Bibliográficas

---

- [Allison et al. 2014]ALLISON, M. et al. Synthesizing interpreted domain-specific models to manage smart microgrids. *The Journal of Systems and Software*, Elsevier, 2014.
- [Bahr et al. 2025]BAHR, L. et al. Knowledge graph enhanced retrieval-augmented generation for failure mode and effects analysis. *Journal of Industrial Information Integration*, v. 45, p. 100807, 2025. ISSN 2452-414X. Disponível em: <<https://www.sciencedirect.com/science/article/pii/S2452414X25000317>>.
- [Bai et al. 2025]BAI, X. et al. Collaboration between intelligent agents and large language models: A novel approach for enhancing code generation capability. *Expert Systems with Applications*, v. 269, p. 126357, 2025.
- [Chen et al. 2026]CHEN, H. et al. Malm-clip: A generative multi-agent framework for multimodal fusion in few-shot industrial anomaly detection. *Information Fusion*, v. 127, p. 103765, 2026.
- [Chen et al. 2025]CHEN, S. et al. Cmp: Composable meta prompt for sam-based cross-domain few-shot segmentation. In: *2025 IEEE International Conference on Image Processing (ICIP)*. [S.l.: s.n.], 2025. p. 737–742.
- [Clarísó e Cabot 2023]CLARISÓ, R.; CABOT, J. Model-driven prompt engineering. In: *2023 ACM/IEEE 26th International Conference on Model Driven Engineering Languages and Systems (MODELS)*. [S.l.: s.n.], 2023. p. 47–54.
- [Eberhardinger et al. 2025]EBERHARDINGER, M. et al. From code to play: Benchmarking program search for games using large language models. *IEEE Transactions on Games*, p. 1–13, 2025.
- [Feng et al. 2025]FENG, X. et al. Crafting user-centric prompts for ui generations based on kansei engineering and knowledge graph. *Advanced Engineering Informatics*, v. 65, p. 103217, 2025.
- [France e Rumpe 2007]FRANCE, R.; RUMPE, B. Model-driven development of complex software: A research roadmap. In: *Future of Software Engineering (FOSE '07)*. [S.l.: s.n.], 2007. p. 37–54.



- [Hopcroft, Motwani e Ullman 2006]HOPCROFT, J. E.; MOTWANI, R.; ULLMAN, J. D. *Introduction to Automata Theory, Languages, and Computation*. 3rd. ed. Boston, MA, USA: Addison-Wesley Professional, 2006.
- [Husein, Aburajouh e Catal 2025]HUSEIN, R. A.; ABURAJOUH, H.; CATAL, C. Large language models for code completion: A systematic literature review. *Computer Standards & Interfaces*, v. 92, p. 103917, 2025.
- [Ji et al. 2023]JI, Z. et al. Survey of hallucination in natural language generation. *ACM Comput. Surv.*, Association for Computing Machinery, New York, NY, USA, v. 55, n. 12, mar. 2023. ISSN 0360-0300. Disponível em: <<https://doi.org/10.1145/3571730>>.
- [Karabiyik 2025]KARABIYIK, M. A. Fault analysis in additive manufacturing: Identifying causes of three-dimensional printer faults using machine learning and large language models. *Journal of Systems and Software*, v. 230, p. 112556, 2025.
- [Kitchenham e Charters 2007]KITCHENHAM, B.; CHARTERS, S. *Guidelines for performing Systematic Literature Reviews in Software Engineering*. UK, 2007.
- [Lewis et al. 2020]LEWIS, P. et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. In: *Proceedings of the 34th International Conference on Neural Information Processing Systems*. Red Hook, NY, USA: Curran Associates Inc., 2020. (NIPS '20). ISBN 9781713829546.
- [Li et al. 2025]LI, Z. et al. Domain adaptation for textual adversarial defense via prompt-tuning. *Neurocomputing*, v. 620, p. 129192, 2025. ISSN 0925-2312.
- [Liu et al. 2023]LIU, Z. et al. Fill in the blank: Context-aware automated text input generation for mobile gui testing. In: *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. [S.l.: s.n.], 2023. p. 1355–1367.
- [Ma et al. 2024]MA, C. et al. An iterative optimizing framework for radiology report summarization with chatgpt. *IEEE Transactions on Artificial Intelligence*, v. 5, n. 8, p. 4163–4175, 2024.
- [Mernik, Heering e Sloane 2005]MERNIK, M.; HEERING, J.; SLOANE, A. M. When and how to develop domain-specific languages. *ACM computing surveys (CSUR)*, ACM New York, NY, USA, v. 37, n. 4, p. 316–344, 2005.
- [Morris et al. 2015]MORRIS, K. A. et al. An adaptive middleware design to support the dynamic interpretation of domain-specific models. *Information and Software Technology*, Elsevier, v. 62, p. 21–41, 2015.

- [Nasiri et al. 2022]NASIRI, M. et al. Prompt-guided scene generation for 3d zero-shot learning. In: *2022 International Conference on Digital Image Computing: Techniques and Applications (DICTA)*. [S.l.: s.n.], 2022. p. 1–8.
- [Ouyang et al. 2022]OUYANG, L. et al. Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems*, v. 35, p. 27730–27744, 2022.
- [Park e Kim 2026]PARK, K.; KIM, D. Aipo: Automatic instruction prompt optimization by model itself with “gradient ascent”. *Computer Speech & Language*, v. 96, p. 101889, 2026.
- [Qin et al. 2025]QIN, Z. et al. Cafgd: Context auto-filling via gradient descent. In: *Proceedings of the 2024 8th International Conference on Computer Science and Artificial Intelligence*. [S.l.: s.n.], 2025. p. 551–557.
- [Shi et al. 2025]SHI, F. et al. A stepwise intelligence generative method for structured maintenance guidance documents based on knowledge graph augmented llm. *Advanced Engineering Informatics*, v. 67, p. 103523, 2025.
- [Sun et al. 2020]SUN, Y. et al. Neural semantic parsing in low-resource settings with back-translation and meta-learning. In: *Proceedings of the AAAI Conference on Artificial Intelligence*. [S.l.: s.n.], 2020. v. 34, n. 05, p. 8960–8967.
- [Umama et al. 2025]UMAMA et al. Llm-based code generation: A systematic literature review with technical and demographic insights. *IEEE Access*, v. 13, p. 194915–194939, 2025.
- [Wang et al. 2023]WANG, B. et al. Grammar prompting for domain-specific language generation with large language models. p. 2837, 2023.
- [Wei et al. 2022]WEI, J. et al. Chain of thought prompting elicits reasoning in large language models. In: *Advances in Neural Information Processing Systems (NeurIPS)*. [S.l.: s.n.], 2022.
- [Wen et al. 2025]WEN, J. et al. Prompt4vfd: A visual analytic system for fault diagnosis of power equipment with llms. *2025 IEEE 2nd International Conference on Big Data Science and Engineering (ICBDSE)*, p. 1–8, 2025.
- [Yan et al. 2025]YAN, Y. et al. Stp: Special token prompt for parameter-efficient tuning of pre-trained language models. *Expert Systems with Applications*, v. 284, p. 127665, 2025.

[Yu et al. 2025]YU, Z. et al. Spec2rtl-agent: Automated hardware code generation from complex specifications using llm agent systems. In: *2025 IEEE International Conference on LLM-Aided Design (ICLAD)*. [S.l.: s.n.], 2025. p. 37–43.

[Zhang et al. 2025]ZHANG, C. et al. Automated data-driven building energy load prediction method based on generative pre-trained transformers (gpt). *Energy*, v. 318, p. 134824, 2025.

[Zhang et al. 2025]ZHANG, T. et al. Knowledge-based cross-modal fusion for long-term forecasting of grouting construction parameters using large language model. *Automation in Construction*, v. 172, p. 106036, 2025.

# Arquitetura, Integração e Viabilidade: Sistema Model-Driven com Grafos de Conhecimento e LLMs

**PPGCC**  
PROGRAMA DE  
PÓS-GRADUAÇÃO EM  
CIÊNCIA DA COMPUTAÇÃO

**INF**  
INSTITUTO DE  
INFORMÁTICA



**UFG**  
UNIVERSIDADE  
FEDERAL DE GOIÁS

## Arquitetura, Integração e Viabilidade

A Arquitetura Neuro-Simbólica CML (*Conceptual Model Learning*)

**Mestrando:** Fernando Souza Furtado Carrilho

**Orientador:** Prof. Dr. Fábio Moreira Costa

Goiânia - GO · 2026

### Resumo Executivo

A proposta apresenta o **Modelo CML**, uma arquitetura neuro-simbólica focada na automação cibernético-física (ex: *Agrodrones*). Baseia-se na integração de Linguagens de Domínio Específico (DSL), Grafos de Conhecimento Dinâmicos e restrição gramatical de LLMs. O objetivo é consolidar o **Fechamento Semântico Mútuo**, garantindo que a intenção extraída de um operador não especializado obedeça rigorosamente às regras físicas do *Esquema de Controle* e seja serializada sob as

amarras sintáticas do *Esquema de Dados*.

## A.1 Validação da Arquitetura e Componentes de Terceiros

### Informação

A arquitetura é tecnicamente sólida e fundamentada na interdependência de Engenharia Dirigida a Modelos (MDE), Recuperação Aumentada por Grafos (*GraphRAG*) e Geração Restrita (*Constrained Decoding*).

### Componentes Selecionados

- **Langium (Modelagem da DSL):** *Framework* principal para a construção da Sintaxe Abstrata (AST) e formalização isolada do Esquema de Controle e do Esquema de Dados.
- **Neo4j (Grafo de Conhecimento):** Núcleo da camada de Verdade Fundamental (*Ground Truth*). Responsável por mapear a ontologia, regras de negócio agrícolas transversais e habilitar o *GraphRAG*.
- **Outlines / Guidance (Grammar Prompting):** Ferramentas de geração restrita que forçam a conformidade estrutural, mascarando os *logits* do LLM em tempo de inferência para obedecer à gramática BNF.
- **Impromptu (Plus / Camada de Adaptação):** Extensão metodológica complementar. Atua opcionalmente como adaptador final (*Just-in-Time*) para traduzir a intenção estruturada para a sintaxe nativa da API alvo.

## A.2 Componentes a Implementar

### Objetivo

Para garantir a fluidez do *pipeline* CML e a independência de modelo LLM (*model-agnostic*), os seguintes módulos de interligação devem ser desenvolvidos.

- **AST-to-Graph Mapper:** Compilador dinâmico que processa a AST validada pelo Langium e a converte em nós, relacionamentos e subgrafos no Neo4j.
- **Semantic Routing Engine (Embedding):** Interpreta a ambiguidade do *prompt* coloquial do usuário (fundido à telemetria de sensores) e ativa subgrafos relevantes via consultas *Cypher* vetorizadas.

- Dynamic Grammar Builder:** Módulo que conecta as permissões do Esquema de Dados da DSL diretamente ao motor de *Grammar Prompting*.

### A.2.1 Body: Semantic Routing Engine

#### Objetivo

Para a operacionalização do motor de roteamento semântico (*Semantic Routing Engine*), a escolha do modelo de *embedding* é um passo fundamental. Modelos de vetorização são substancialmente mais leves e rápidos que os LLMs generativos, atuando como o primeiro filtro de desambiguação do jargão operacional (ex: "veneno" → "defensivo agrícola").

Estes modelos podem ser divididos em duas categorias principais de adoção:

- Modelos via API (Nuvem):**

- text-embedding-3-small (OpenAI):** Considerado um padrão da indústria atual. Destaca-se por sua alta velocidade, baixo custo por *token* e suporte robusto ao jargão regional em português.

- Outras alternativas comerciais:** *Cohere* e *Google Vertex AI*.

- Modelos Locais (*Open-source* e execução *on-premise*):**

- all-MiniLM-L6-v2:** Consiste em um modelo altamente otimizado e leve, executável localmente através da biblioteca *sentence-transformers* em Python. Gera vetores de 384 dimensões e é ideal para missões em áreas remotas (sem internet) e arquiteturas com restrições rígidas de conectividade, típicas da agricultura de precisão.

No contexto desta dissertação, propõe-se a utilização de *Embeddings* integrados à funcionalidade de *Vector Search* nativa do Neo4j para recuperar os subgrafos (culturas, defensivos, regras de estresse hídrico). O subgrafo exato recuperado é repassado ao *Dynamic Grammar Builder*, restringindo o LLM a atuar unicamente como um tradutor semântico encapsulado.

## A.3 Plataforma, Linguagens e Frameworks

#### Motivação

A *stack* prioriza o controle ontológico rigoroso, integração semântica com IAs e portabilidade estrutural para sistemas embarcados/críticos.

- **Linguagem Principal:** Python 3.10+ e TypeScript (para o Langium)
- **Banco de Dados:** Neo4j (GraphRAG e ontologia de regras)
- **Decodificação Restrita:** Outlines / Guidance
- **LLMs Alvo:** GPT-4o ou Llama-3 (agindo como conversores semânticos limitados)
- **API e Protocolos:** FastAPI / MCP Server (Model Context Protocol)
- **Camada de Adaptação (Plus):** Impromptu (opcional para adaptação JIT final)

## A.4 Estratégia do Pipeline CML

Fluxo de resolução processa desde a intenção bruta à especialização em *hardware*.

1. **Definição Formal:** O especialista (agrônomo) modela a DSL, mapeando as regras no Esquema de Controle e as saídas válidas no Esquema de Dados.
2. **Ancoragem:** O AST-to-Graph Mapper traduz as regras para subgrafos no Neo4j.
3. **Input e Embedding:** Usuário leigo emite um comando verbal que, somado à telemetria de sensores, é vetorizado.
4. **Busca (GraphRAG):** Vetores interrogam o Neo4j, extraíndo proibições e obrigações para gerar o *Prompt Semântico*.
5. **Grammar Prompting:** O motor restringe as opções de *tokens* do LLM usando o Esquema de Dados, forçando uma saída JSON matematicamente validada.
6. **Adaptação e Execução:** A camada Impromptu (opcional) adapta o JSON estruturado para *payloads* nativos da *flight controller* do drone agrícola.

## A.5 Plano de Testes e Avaliação

A validação envolverá a comprovação da eliminação de erros sintáticos e alucinações físicas operacionais.

### Métricas e Abordagens

- **Testes Unitários:** Validação da AST → Geração de Cypher (pytest)
- **Recuperação Semântica (GraphRAG):** Context Precision (CP) e Context Recall (CR)
- **Conformidade Estrutural:** Taxa de falha sintática (*Parsing Error Rate*) → **objetivo metodológico CML: 0%**.

## A.6 Análise de Viabilidade

**Viabilidade: Alta**

A arquitetura desloca o esforço computacional do incerto *fine-tuning* de LLMs para a governança sólida de grafos e matemática gramatical. As ferramentas base (Langium e Neo4j) são maduras e robustas para a pesquisa acadêmica.

## A.7 Cronograma

| Semana | Fase          | Atividades Principais                                          |
|--------|---------------|----------------------------------------------------------------|
| 1      | Fundamentação | Modelagem dos Esquemas de Controle/Dados em Langium            |
| 2      | Ancoragem     | Implementação do AST-to-Graph Mapper para o Neo4j              |
| 3      | Vetorização   | Configuração de <i>Embeddings</i> e testes de GraphRAG         |
| 4      | Contenção     | Integração do motor de <i>Grammar Prompting</i>                |
| 5      | Camada Plus   | Conexão opcional com Impromptu e formatação para APIs de Drone |
| 6      | Validação     | Avaliação do Fechamento Semântico e taxa de falhas sintáticas  |

Documento técnico com foco na formalização da Arquitetura Neuro-Simbólica CML.

## A.8 Justificativa da Escolha Tecnológica

A seleção unifica o rigor da Engenharia Dirigida a Modelos (MDE) à vanguarda da Inteligência Artificial Generativa e recuperação ontológica.

- Langium (Engenharia de Linguagem):** Baseado em TypeScript e infraestrutura *Language Server Protocol* (LSP), gera *parsers* nativos de alta interoperabilidade. É substancialmente superior a abordagens antiquadas baseadas exclusivamente em Java/Xtext.
- Neo4j (Banco de Dados de Grafos):** Selecionado por possuir *index-free adjacency*. Supera *VectorDBs* tradicionais por reter a direcionalidade das restrições agronômicas (ex: "Cultura Soja → *proíbe* → Nicosulfuron"), vital para o *Ground Truth*.



- **Outlines / Guidance (Grammar Prompting):** Diferente de *frameworks* baseados em *prompt engineering* que fazem *retry* após falhas, estas ferramentas agem diretamente no nível de máscara de *logits* do modelo neural, assegurando 100% de conformidade sintática sem ciclos de inferência extras.
- **Impromptu (Camada Adaptativa / Plus):** Atuando intencionalmente como uma funcionalidade externa ao núcleo neuro-simbólico, serve de *glue code* final, traduzindo e acomodando o *prompt* purificado às idiossincrasias das APIs nativas.

## A.9 A Instanciação da Linguagem Específica de Domínio

Para assegurar o **Fechamento Semântico Mútuo**, é necessário construir a metassintaxe da i-DSML. O *design* a seguir materializa o **Estudo de Caso do Agrodrome**, separando de forma estrita o que a máquina conhece (Esquema de Controle) de como o LLM tem permissão para responder (Esquema de Dados).

### A.9.1 A Arquitetura Conceitual da DSL

A formulação do modelo de domínio consolida os pilares do CML:

- **Esquema de Controle (A Verdade Factual):** Modela as regras estruturais e de integridade do ambiente físico, registrando interações biológicas (ex: Cultura, Defensivo, e as restrições meteorológicas aplicáveis).
- **Esquema de Dados (As Restrições de Saída):** Define exaustivamente as respostas possíveis (formatos, parâmetros, *strings* exatas) que o LLM tem permissão de formatar na execução final, barrando saídas arbitrárias.

### A.9.2 Ferramentas e Frameworks Selecionados

A implementação desta camada fará uso das seguintes tecnologias para sustentar o fluxo:

### A.9.3 Passo-a-Passo: Especificação CML para o Agrodrome

Detalha-se o fluxo de contenção lógica utilizando a metassintaxe no Langium para o cenário crítico da agricultura de precisão.

#### **Etapas 1: Definição do Esquema de Dados e Controle (BNF)**

A gramática BNF abaixo dita as entidades fáticas (Cultura) no *Esquema de Controle*, mas crucialmente, delimita as **saídas possíveis** (ex: AcaoPermitida) no

| Ferramenta | Camada de Atuação       | Justificativa e Propósito                                                                                                      |
|------------|-------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| Langium    | Construção da DSL       | Estrutura a gramática formal (BNF) e gera a AST para validação estrutural <i>a priori</i> .                                    |
| Neo4j      | Grafo (Eq. de Controle) | Instancia os nós da AST em grafos ontológicos, permitindo a gestão cruzada de regras globais do domínio.                       |
| Impromptu  | Camada Adap. (Plus)     | <i>Framework</i> complementar que isola e converte o JSON validado para <i>payloads</i> e protocolos nativos da máquina final. |

*Esquema de Dados.* O LLM não pode burlar estas saídas, sob pena de bloqueio (*logits masking*).

Exemplo Prático (Domínio Drone Agrícola):

AgroDrone.langium

```
grammar AgroDrone

// Regra inicial (Ponto de ancoragem da AST)
entry OperacaoPulverizacao: (comandos+=ComandoDrone)*;

// Esquema de Controle (O que existe no dominio fisico e regras
// do sistema)
ComandoDrone:
    'atuar' acao=AcaoPermitida 'produto' prod=Produto 'talhao'
    id=ID;

// Esquema de Dados (Restricoes estritas de saidas possiveis
// para o LLM)
Produto returns string: 'Acefato' | 'Glifosato' | '2,4-D' |
'Mancozebe';
Cultura returns string: 'Soja' | 'Milho' | 'Cana';
AcaoPermitida returns string: 'Aplicar' | 'Adiar' |
'Limpar_Tanque';

// Regras lexicais basicas
terminal ID: /[_a-zA-Z][\w_]*/;
```

**title**

1. **Validação da AST:** O código BNF gera a árvore de sintaxe. Se, motivado por um comando ambíguo, o LLM tentar alucinar uma *AcaoPermitida* inexistente (como “Acelerar\_Drone”), o próprio avaliador sintático bloqueia a inferência, garantindo conformidade.
2. **Graph Mapper:** O contexto agroquímico validado é instanciado no Neo4j, onde regras globais e perigosas (como precipitação por mistura de químicos ou limite de estresse térmico da planta) são avaliadas antes da instrução final.
3. **Passagem ao Impromptu:** A intenção 100% segura e formatada no molde da DSL pode então ser injetada no *Impromptu*, envelopando as variáveis extraídas em comandos JSON nativos da controladora de voo (*flight controller*), blindando de ponta-a-ponta a automação industrial.

# Apostila Técnica - Arquitetura Neuro-Simbólica

---

**PPGCC**  
PROGRAMA DE  
PÓS-GRADUAÇÃO EM  
CIÊNCIA DA COMPUTAÇÃO

**INF**  
INSTITUTO DE  
INFORMÁTICA



---

## APOSTILA TÉCNICA

### Arquitetura Neuro-Simbólica: DSL AgroDrone com Langium e Grafos no Neo4j

---

Um guia prático focado no cenário agrícola: configuração de ambientes, modelagem de linguagem de domínio específico, geração de embeddings locais e roteamento semântico integrado a Knowledge Graphs.

**Mestrando:** Fernando Souza Furtado Carrilho

**Orientador:** Prof. Dr. Fábio Moreira Costa

**Universidade Federal de Goiás**

Instituto de Informática (INF) | PPGCC

**Goiânia - GO · 2026**

## B.1 Visão Geral do Cenário AgroDrone

### Objetivo de Aprendizagem

Capacitar o leitor a configurar um ambiente Langium complexo para o domínio agrícola, modelando regras de aplicação química e climática, persistindo-as em grafos (Neo4j) e viabilizando a tomada de decisão via LLMs orientados a telemetria.

O fluxo arquitetural segue uma esteira neuro-simbólica rica e robusta:

### Fluxo da Arquitetura AgroDrone

DSL -> AST -> Grafo + Embeddings (Neo4j) -> (LLM)

## B.2 Setup do Ambiente

Para garantir que todos os componentes funcionem em harmonia, o ambiente de desenvolvimento deve ser configurado com ferramentas modernas e atualizadas.

### B.2.1 Pré-requisitos

- Node.js (LTS):** Motor de execução para o ecossistema Langium.
- VS Code:** Editor de código recomendado.
- Python 3.10+:** Para integrações futuras de inteligência artificial.
- Neo4j:** Banco de dados em grafos rodando localmente.

### B.2.2 Extensões do VS Code

Recomenda-se a instalação das seguintes extensões para suporte completo a tipagem e formatação: Python, ESLint, Prettier e Node.js Extension Pack.

## B.3 Criação do Projeto

### B.3.1 Configuração e Instalação

Para configurar as dependências do projeto, incluindo o motor do Langium, tipagens do TypeScript e o driver do Neo4j, execute os comandos abaixo na raiz do projeto:

**Terminal - Dependências Base**

```
\small
npm install langium
npm install -D typescript ts-node langium-cli @types/node
npm install vscode-languageclient vscode-languageserver
npm install neo4j-driver
npx tsc --init
```

Para a geração de embeddings locais, instale a biblioteca do motor de vetorização:

**Terminal - Embeddings**

```
npm install @xenova/transformers
```

## B.4 Estrutura do Projeto

Mantenha seu código organizado. Uma estrutura limpa facilita a manutenção da linguagem de domínio agrícola e a integração com o VS Code.

**Estrutura Atualizada do Projeto**

```
dsl-project/
  src/
    database/
      neo4j-sync.ts
      semantic-search.ts
    generated/
    language/
      dsl-module.ts
      agrodrome.langium
    language-server/
      main.ts
    test/
      test.ts
    embedding.ts
    extension.ts
  package.json
  tsconfig.json
```

## B.5 Criando Linguagem/Gramática DSL

Para formalizar os conceitos operacionais, as regras de domínio e os limites mecânicos discutidos anteriormente, projetou-se uma Linguagem de Domínio Específico (DSL) utilizando a infraestrutura do *framework* Langium. A especificação da sintaxe concreta e abstrata foi desenvolvida sob o rigor da Notação de Backus-Naur Form (BNF), fornecendo ao ecossistema CML a capacidade de gerar um modelo de controle estrito e tipado. Desse modo, substitui-se o empirismo dos textos livres por uma estrutura computacional previsível, em que cada símbolo possui uma semântica inviolável. O Código B.1 apresenta a especificação gramatical completa da linguagem AgroDrone, detalhando a separação de responsabilidades entre o Esquema de Controle Agrônômico e o Esquema de Dados e Atuação do *hardware*.

Listing B.1: Sintaxe formal da DSL AgroDrone

---

```
grammar AgroDrone

// Ponto de entrada unificado do modelo híbrido
entry AgroModel:
    (elements+=Element)*;

Element:
    CropDef | GlobalRule | MissionCommand | DataSchemaDef;

//
=====

// 1) ESQUEMA DE CONTROLE (Regras de Domínio Agrônômico)
//
=====

CropDef:
    'cultura' name=ID '{'
        (chemicals+=ChemicalDef)*
        (rules+=RuleDef)*
    '}' ;

ChemicalDef:
    type=('herbicida' | 'fungicida' | 'inseticida' | 'defensivo') name=
        ID '-' dosage=STRING '-' timing=STRING;

RuleDef:
    MixingRule | BanRule | WeatherRule | SweepRule | InformRule;

MixingRule:
```

```

    'regra:' 'nao_misturar' chem1=[ChemicalDef:ID] '+' chem2=STRING '('
        reason=STRING ')';

BanRule:
    'regra:' 'proibido' condition=STRING;

WeatherRule:
    'regra_clima:' conditions=STRING;

SweepRule:
    'regra_varredura:' sensor=STRING operator=Operator value=FLOAT '->'
        action=[ActionCommand:ID];

InformRule:
    'informar:' items=STRING;

GlobalRule:
    'regra_global:' description=STRING;

//
=====

// 2) ESQUEMA DE DADOS E ATUAÇÃO (Estados Físicos e Comandos
Mecânicos)
//
=====

DataSchemaDef:
    'esquema_dados' name=ID '{'
        'estados_permitidos' '[' allowedStates+=DroneState (keys+=','
            allowedStates+=DroneState)* ']'
        'acoes_atuador' '[' actuatorActions+=ActuatorAction (keys+=','
            actuatorActions+=ActuatorAction)* ']'
        (commands+=ActionCommand)*
    '}'

// Definição dos estados de movimentação física do hardware
enum DroneState:
    AVANCAR = 'AVANCAR' |
    PARAR = 'PARAR' |
    RETORNAR_BASE = 'RETORNAR_BASE' |
    MUDAR_TALHAO = 'MUDAR_TALHAO' |
    MUDAR_SENTIDO = 'MUDAR_SENTIDO';

// Definição dos comandos do sistema de pulverização e bombas
enum ActuatorAction:

```



```

BOMBA_LIGAR = 'BOMBA_LIGAR' |
BOMBA_DESLIGAR = 'BOMBA_DESLIGAR' |
BATER_VENENO = 'BATER_VENENO' |
AJUSTAR_VAZAO = 'AJUSTAR_VAZAO';

// Comando at mico que vincula um movimento a uma a o de atuador
ActionCommand:
    'acao' name=ID '{'
        'movimento' motion=DroneState
        'atuador' actuator=ActuatorAction
        ('parametro' param=STRING)?
    '}' ;

//
=====

// 3) EXECU O DA MISS O (V nculo entre os Esquemas)
//
=====

MissionCommand:
    'comando' 'executar' missionName=ID 'para' crops+=[CropDef:ID] (
        keys+=',' crops+=[CropDef:ID]) * '{'
        'esquema_referencia' schema=[DataSchemaDef:ID]
        'sequencia_atuacao' '[' sequence+=[ActionCommand:ID] (keys+=','
            sequence+=[ActionCommand:ID]) * ']'
        '}' ;

// Operadores matem ticos e regras l xicas
Operator returns string:
    '>' | '<' | '>=' | '<=' | '==';

terminal ID: /[_a-zA-Z][\w_]*/;
terminal STRING: /"[^"]*"|'[^']*'/;
terminal FLOAT returns number: /[0-9]+(\.[0-9]+)?/;
hidden terminal WS: /\s+/;
hidden terminal ML_COMMENT: /\/*[\s\S]*?\*/;
hidden terminal SL_COMMENT: /\s*\/\s*/;

```

O Código [B.1](#) materializa a estrutura sintática da DSL AgroDrone, evidenciando como a arquitetura CML traduz as fronteiras operacionais do sistema em regras gramaticais estritas. A inclusão do bloco `DataSchemaDef` consolida o Esquema de Dados e Atuação através dos tipos enumerados `DroneState` e `ActuatorAction`, mapeando formalmente os estados de movimentação física do veículo (*AVANCAR*, *PARAR*, *RETORNAR\_BASE*, *MUDAR\_TALHAO*, *MUDAR\_SENTIDO*) e os comandos diretos do sis-

tema de pulverização (*BOMBA\_LIGAR*, *BOMBA\_DESLIGAR*, *BATER\_VENENO*, *AJUSTAR\_VAZAO*). Ao encapsular essas diretrizes em estruturas atômicas vinculadas pela regra *ActionCommand*, a linguagem deixa de depender de descrições textuais abstratas ou livres e passa a fornecer a Notação de Backus-Naur (BNF) exata que alimentará o motor de *Grammar Prompting*. Como resultado, a inferência probabilística do modelo de linguagem é cirurgicamente confinada ao preenchimento dessa sequência tipada em *MissionCommand*, mitigando falhas estruturais de formatação e garantindo que o plano de atuação derivado seja inteiramente inteligível e seguro para o *hardware* de destino.

## B.6 Criando o Modelo da DSL Agrícola

A gramática do AgroDrone define blocos para culturas (*CropDef*), restrições de produtos (*ChemicalDef*), condições climáticas e comandos de missão.

### B.6.1 Exemplo de Modelo no uso da DSL)

O agrônomo ou sistema de retaguarda fornecerá as regras da seguinte maneira:

## Modelo feito por especialista da área

```

cultura Soja {
  herbicida Glifosato - "1.5-2.0 L/ha" - "p s-emerg ncia"
  fungicida Mancozebe - "1.5 kg/ha" - "preventivo ferrugem"
  inseticida Acefato - "0.6 kg/ha" - "lagartas"

  regra: nao_misturar Glifosato + "Mancozebe" ("precipita o")
  regra: nao_misturar Acefato + "Nicosulfuron" ("fitotoxicidade")
  regra_clima: "06h-09h ou 17h-19h (T < 30C, vento < 10 km/h)"
  regra_varredura: "Indice NDVI" < 0.4 -> "alerta pragas"
  informar: "Area afetada (ha), produto aplicado, hora, sensor NDVI"
}

cultura Milho {
  herbicida Atrazina - "1.5-3.0 L/ha" - "pr e p s-emerg ncia"
  fungicida Trifloxistrobina - "0.4 L/ha" - "cercosporiose"
  inseticida Clorpirifos - "0.5 L/ha" - "cigarrinha-do-milho"

  regra: nao_misturar Atrazina + "Clorpirifos"
  regra: proibido "Atrazina em rea < 500m de manancial h drico"
  regra_clima: "somente manh (06h-10h), umidade > 55%"
  regra_varredura: "Sensor umidade foliar" < 60.0 ->
  "risco cigarrinha"
  informar: "Talh o, est dio fenol gico, n vel de infesta o,
  vento"
}

cultura Cana_de_Acucar {
  herbicida Dois_Quatro_D - "1.0-2.0 L/ha" - "p s-emerg ncia"
  fungicida Azoxistrobina - "0.2 L/ha" - "ferrugem laranja"
  inseticida Imidacloprido - "0.5 L/ha" - "broca-da-cana"

  regra: nao_misturar Dois_Quatro_D + "Atrazina"
  regra: proibido "perto de culturas sens veis
  (uva, ma , oliveira)"
  regra_clima: "Vento < 10 km/h; T < 32C; umidade > 55%"
  regra_varredura: "Sensor temperatura foliar" > 38.0 ->
  "estresse hidrico"
  informar: "Coordenada GPS, produto, vaz o (L/ha),
  condicao climatica"
}

regra_global: "Limpar tanque do drone entre culturas diferentes"
regra_global: "Nunca aplicar simultaneamente em talh es adjacentes
com ventos > 8 km/h se culturas diferentes"
comando aplicar Soja, Milho, Cana_de_Acucar usando DroneAgricola01

```

## B.7 Langium - Pipeline de Parsing AgroDrone

A interpretação da linguagem natural até a validação do modelo opera em um pipeline altamente estruturado, conforme ilustra a Figura B.1.

## B.8 Geração de Embeddings Locais

Visando cenários off-grid (zonas rurais), o processamento vetorial ocorre localmente via Node.js utilizando o modelo `all-MiniLM-L6-v2` em formato ONNX. O texto das regras processadas na AST é convertido para um tensor de 384 dimensões.

### Processamento Vetorial *On-Premise* e Segurança

A adoção de modelos de *embedding* executados localmente elimina a dependência de APIs em nuvem e assegura a privacidade ponta a ponta dos dados, garantindo que a transformação semântica da DSL ocorra de forma autônoma e segura antes da persistência no banco de grafos.

## B.9 Persistência no Knowledge Graph (Neo4j)

Com a AST validada e os vetores calculados, o sistema sincroniza os subgrafos de cultura e os nós de regras no banco Neo4j. São criados índices vetoriais (VECTOR INDEX) para permitir o *Semantic Routing*.

### Execução do Banco

```
sudo neo4j start
sudo neo4j status
```

### Índices Vetoriais no Neo4j

O índice `agro_rules_index` é gerado automaticamente no banco, apontando para a propriedade `embedding` do nó de Regra, utilizando `cosine similarity`. Isso prepara o terreno para a inferência em tempo real baseada em telemetria.

## B.10 Motor de Decisão (LLM) + Telemetria

Quando o drone realiza a leitura de sensores em campo (Temperatura, Umidade, Vento), o sistema busca no Neo4j as regras restritas daquela cultura específica, combinadas com as Regras Globais. Esses dados estruturados são injetados em um prompt estrito para um LLM governar a decisão de aplicação ou retorno à base (RTH).

FLUXO DE OPERAÇÃO LANGIUM (AGRODRONE)

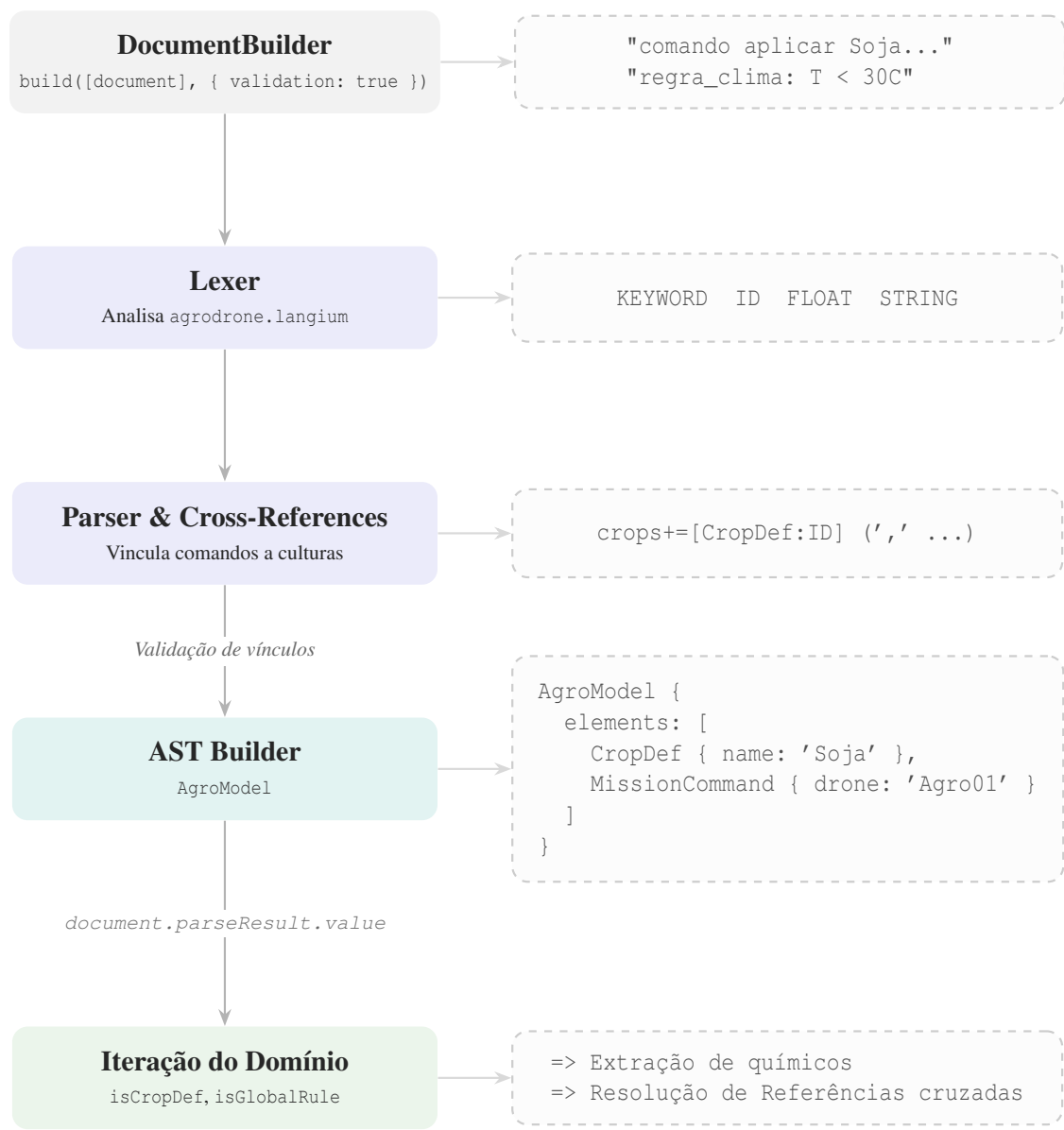


Figura B.1: Pipeline de parse com tratamento de referências cruzadas no modelo Agrícola.

Para testar o fluxo completo, você deve executar:

#### Testar Roteamento Semântico

```
npx tsx src/database/semantic-search.ts
```

#### Conclusão do Módulo

Com este fluxo, unificamos validação sintática restrita (Langium) com o poder computacional de Grafos e IA Generativa (LLM + Embeddings). O AgroDrone adquire capacidade de agir de forma segura, justificada e baseada em regras agronômicas formalizadas.

## B.11 Repositório do Projeto no GitHub

Para garantir a reprodutibilidade da pesquisa e facilitar a exploração prática, todo o ecossistema desenvolvido — incluindo o motor da DSL, a esteira de vetorização e a integração com o *Knowledge Graph* — encontra-se disponível publicamente no GitHub.



### Acesso ao Código-Fonte

Clone o repositório para analisar a arquitetura e executar os testes locais.

<https://github.com/Fernando-EngComputacao/dsl-msc.git>

No repositório da dissertação, os seguintes artefatos estão estruturados e documentados:

- **Motor da DSL AgroDrone:** Implementação completa da gramática, *parser* e validações utilizando o ecossistema Langium.
- **Persistência em Grafos:** Módulos de conexão, criação de índices vetoriais e sincronização com o banco Neo4j.
- **Processamento Vetorial:** Scripts isolados para a geração de *embeddings* locais (*Edge Computing*) otimizados em formato ONNX.
- **Roteamento Semântico:** Implementação do *pipeline* de decisão, cruzando regras simbólicas com inferência via LLMs.

- Cenários de Teste:** Casos de uso práticos com simulação de telemetria de drones para validação da esteira de dados.

A exploração e execução deste código são fortemente recomendadas para uma compreensão aprofundada da Engenharia Dirigida a Modelos (MDE) aplicada, fornecendo uma base sólida para a adaptação da arquitetura a novos requisitos do agronegócio ou outros sistemas críticos.